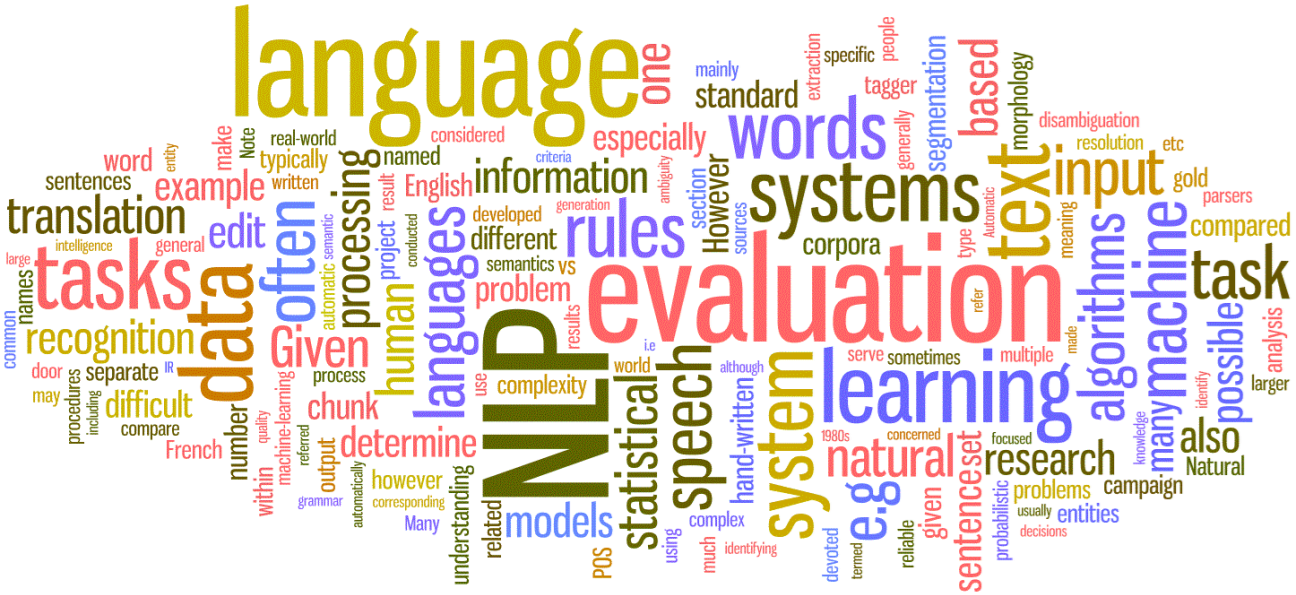


Lecture 8: Parsing I

Xu Ruifeng

Harbin Institute of Technology, Shenzhen

Last Time

- Word Meanings
- Computer Expression of Word Meanings
- Word Sense Disambiguation

Today's class

- Chomsky Hierarchy and Grammar
- Parsing Approaches for Context-Free Grammar (CFG)
  - Top-down Approach
  - Bottom-up Approach
- Parsing Algorithms
  - Shift-reduce Parsing
  - CYK Parsing

Syntactic Analysis

- Computer and Natural Languages are very **productive**. They are also highly **regular** in character. Hence, any grammar which is generated to accept all structurally correct strings must account for this productivity and regularity.

- To describe the regularity and productivity of languages, we provide rules of **syntax**.

- Our rules of syntax should specify the syntactic structure of a string in our language.

Syntactic Analysis

- To provide a syntactic analysis for a string, we must have:

- **A Grammar**: the formal specification of allowable structures.

- **A Parsing Algorithm**: a method of analyzing the sentence to determine its structure.

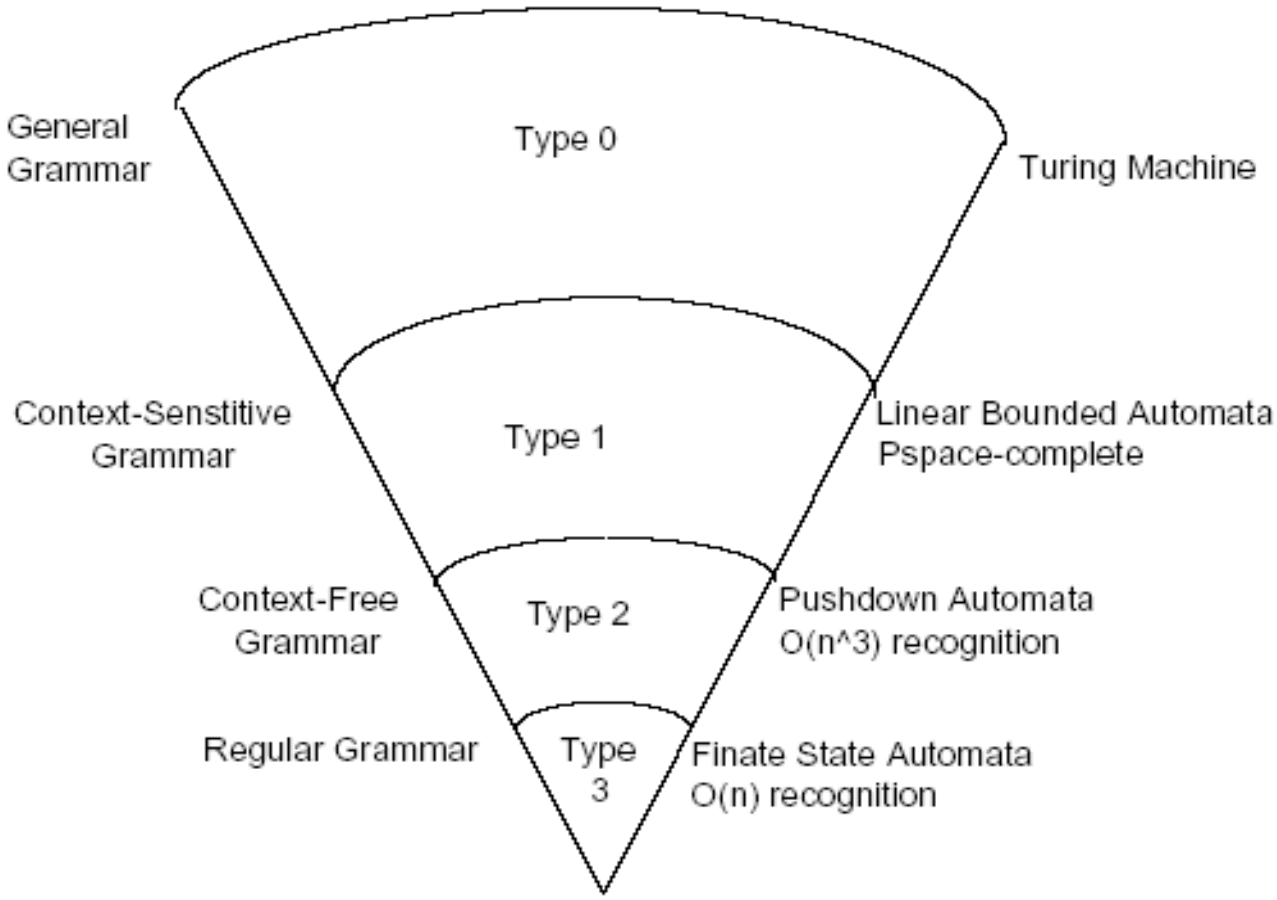
- **Generative grammars** can be thought of as a set of rules that generate valid phrases in a particular language. Noam Chomsky defined four classes of "complexity" for generative grammars, that are commonly organized into the **Chomsky Hierarchy**.

Generative Grammar

- A generative grammar G is expressed as G=(V, T, P, S), where:
  - V is a **finite** set of non-terminals or variables (use capital letters to designate them).
  - T is a **finite** set of terminals or tokens (use small letters to designate them).
  - P is a finite set of productions or rules of the form $\alpha \rightarrow \beta$.
  - S is a special non-terminal called the start symbol, $S \in V$.

- Normally $V \cap T = \varnothing$.
- A grammatical symbol is an element of $V \cup T$.

Chomsky Hierarchy



Regular Grammar (Type 3)

- A Regular Grammar is denoted as G=(V, T, P, S)
  - V are finite set of non-terminal variables
  - T are finite set of terminal variables
  - S are start symbol
  - P is a finite set of productions. Consist of productions like A→β
- Left linear grammar for ab
  - B→Ab A→a
  - Left hand side: one non-terminal symbol
  - Right hand side: empty string / a terminal symbol/a non-terminal sysbol following a terminal symbol
- Right linear grammar
  - A→aB B→b

Context-free Grammars (Type 2)

- A Context-free Grammar is denoted as G=(V, T, P, S)
  - V are finite set of non-terminal variables
  - T are finite set of terminal variables
  - S are start symbol
  - P is a finite set of productions. Consist of productions like $A \rightarrow \alpha$
  - Only one non-terminal symbol on the left hand side

- Push down Automata

Example

- Example Grammar, G:

  V = {S, VP, NP, Name, Det, Noun, Pro, Verb}

  T = {*Nyssa, chases, cat, cats, the, he, she*}

  S = {S}

  P = {S → NP VP
  VP → Verb NP
  NP → Name | Det Noun | Pro
  Name → *Nyssa*
  Det → *the*
  Noun → *cat* | *cats*
  Pro → *he* | *she*
  Verb → *chases*}

Example

- To generate phrases in the language of grammar G, repeatedly apply productions to non-terminals beginning with the start symbol.

- If we apply $A \rightarrow \beta$ to the string $\alpha A \gamma$, we obtain $\alpha\beta\gamma$, obtaining a string in the language generated by G, L(G).

- L(G) is the set of strings such that:

1. Each string consists **solely** of terminals.
2. Each string is derived from S by applying the productions in P. Below is a string of derivations:
   $\alpha_1 \rightarrow \alpha_2 \rightarrow \alpha_3 \rightarrow \ldots \rightarrow \alpha_n$

Example

- An important aspect of the CFGs is that the derivations involve variables that are independent of what surrounds them; hence, they are context independent.

- Importance of CFGs:

1. They are powerful enough to handle much of the productivity of computer and natural languages.

2. They are restrictive enough to construct efficient parsers, $O(n^3)$ generally, $O(n)$ for deterministic grammars (generally true for computer languages).

Context-Sensitive Grammars (Type 1)

- A Context-Sensitive Grammar is denoted as G=(V, T, P, S)
  - V are finite set of non-terminal variables
  - T are finite set of terminal variables
  - S are start symbol
  - P is a finite set of productions. Consist of productions like $\alpha \rightarrow \beta$
  - α and β are strings of symbol in (VUT)* and β is at least as long as α
  - NO productions of $AB \rightarrow C$ or $AB \rightarrow$ empty
  - $\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$ where β can't be empty "A becomes β in the context of α_1 and α_1"

General Grammar (Type 0)

- A General Grammar is denoted as G=(V, T, P, S)
  - V are finite set of non-terminal variables
  - T are finite set of terminal variables
  - S are start symbol
  - P is a finite set of productions WITHOUT RESTRICATIONS
- Type 0 grammar are recursively enumerable and are accepted by a Turing Machine.
- Too complex a speciation for computer languages.

Parsing

- Parsing is the process of determining whether a string is in a grammar G.
- Without context, ambiguities occurs in natural language frequently. With more contextual information, most ambiguities can be removed
- The core of parsing natural language is to resolve the ambiguities in the syntactic structures.
- The goal of sentence processing is to understand the representations people form as they understand a sentence (syntax, meaning…).
- Parsing adds information about sentence structure and constituents
- Allows us to see what constructions words enter into
  - eg, transitivity, passivization, argument structure for verbs
- Allows us to see how words function relative to each other
  - eg, what words can modify / be modified by other words

Parsing: difficulties

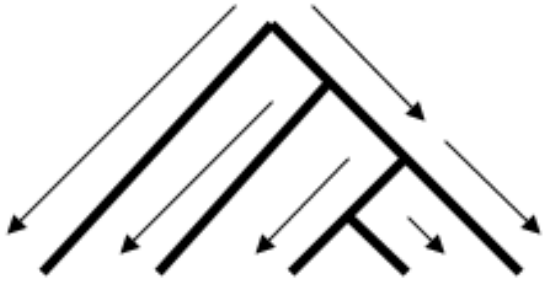
- Besides lexical ambiguities (usually resolved by tagger), language can be structurally ambiguous
  - global ambiguities due to ambiguous words and/or alternative possible combinations
  - local ambiguities, especially due to attachment ambiguities, and other combinatorial possibilities
  - sheer weight of alternatives available in the absence of (much) knowledge
- Broad coverage necessary for parsing corpora of real text
- Long sentences:
  - structures are very complex
  - ambiguities proliferate
- Difficulty (even for human) to verify if parse is correct
  - because it is complex
  - because it may be genuinely ambiguous

Parsing

- Parsing Approaches
  - Top down parsers
  - Bottom up parsers
  - Left corner parsers

Top-down Parsing

- Begin with the start symbol S and produce the right-hand side (RHS) of the rule
- Match the left-hand side (LHS) of CFG rules to the non-terminals in the string, replacing them with the right-hand side of the rule.
- Continue until all the non-terminals are replaced by terminals, such that they correspond to the symbols in the sentence.
- The parse succeeds when all wor generated.



Top-down Parsing

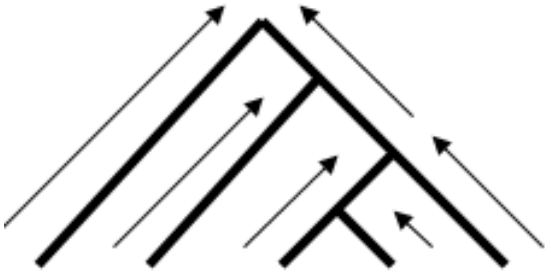
- This is a left-most derivation for the sentence; each production is applied to the left-most non-terminals

- Based on Prediction
  - Produce the prediction for following component, then verify the prediction by matching words
  - If the prediction is verified, the target string will be parsed according to the prediction
  - If no verified, use other predictions (backtracking)
  - If all predictions are rejected, the sentence is not valid → parser fails

Bottom-up Parsing

- Begin with Words.
- Apply the right-hand side RHS of the rules to the words to generate non-terminals from the LHS.
- The parse succeeds when the start non-terminal is generated.

- Based on Reduction
  - Match the words in the sentence
  - Generate possible non-terminal symbols iteratively
  - If S is generated, Parse succeeds
  - If S is not generated, use other symbols (backtracking)



Rule Set

Rule	Dictionary
(7) NP → R	(1) R→我
(8) NP → N	(2) N →县长
(9) NP → Sϕ de	(3) V →是
(10) VPϕ → V V	(4) V →派
(11) VP → V NP	(5) V →来
(12) Sϕ → NP VPϕ	(6) de →的
(13) S → NP VP	

Top-Down Parsing Illustration -1

Rule

(7) NP → R

(8) NP → N

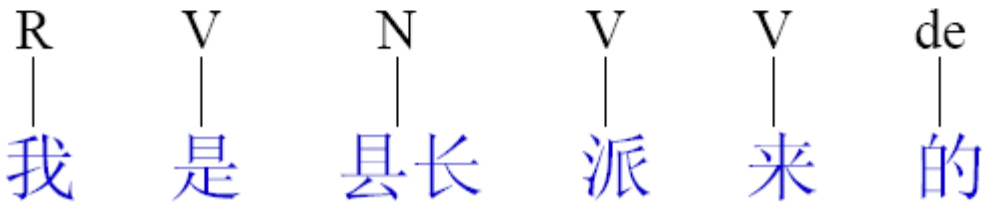
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



2

Rule

(7) NP → R

(8) NP → N

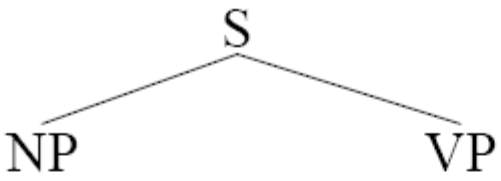
(9) NP → Sϕ de

(10) VPϕ → V V

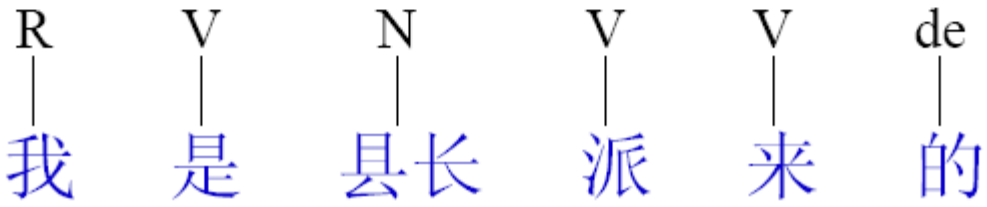
(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



S → NP VP



3

Rule

(7) NP → R

(8) NP → N

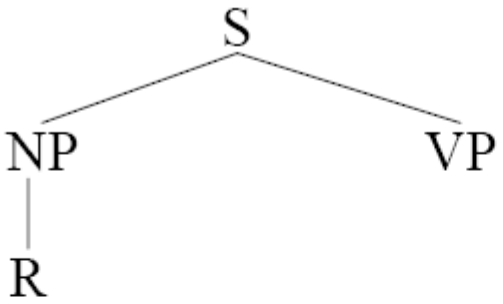
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



NP→R

R V N V V de

我 是 县长 派 来 的

4

Rule

(7) NP → R

(8) NP → N

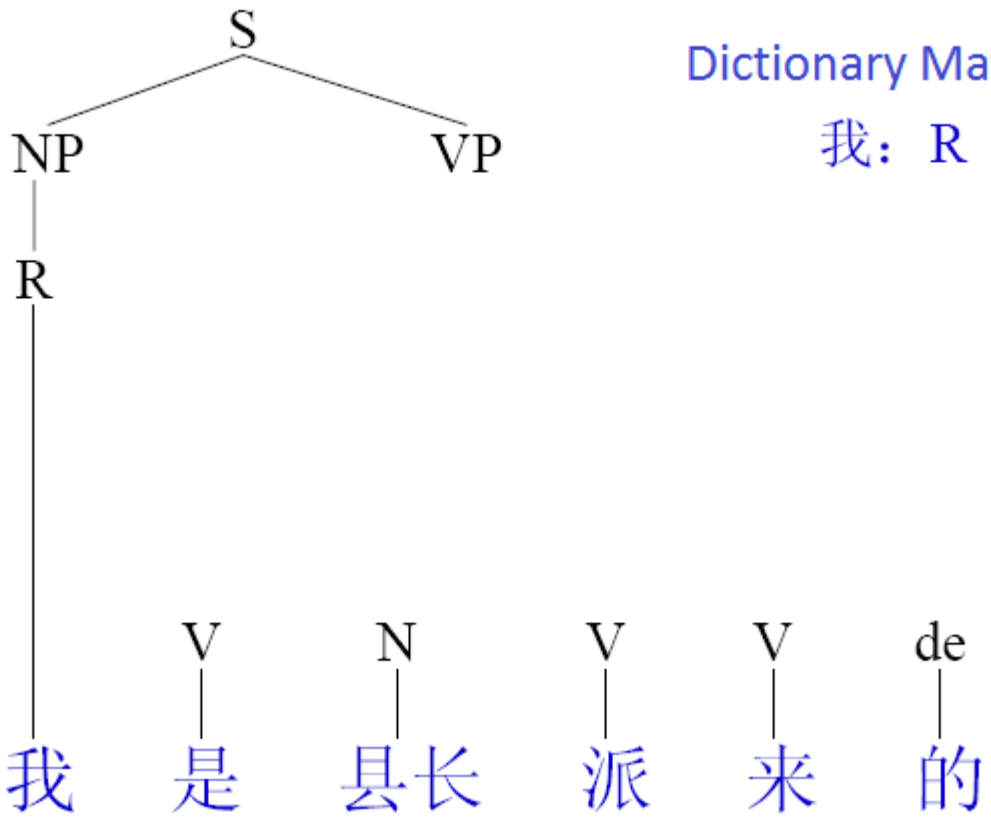
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary Match Succeeds

我：R

5

Rule

(7) NP → R

(8) NP → N

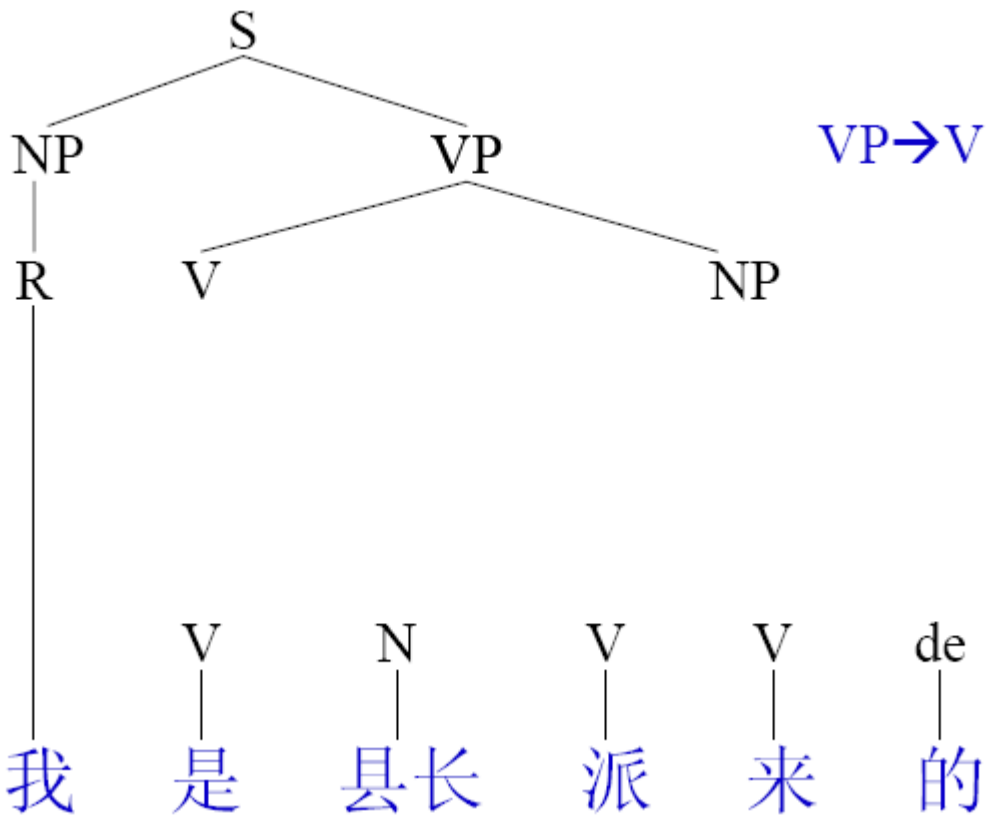
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



VP→V NP

6

Rule

(7) NP → R

(8) NP → N

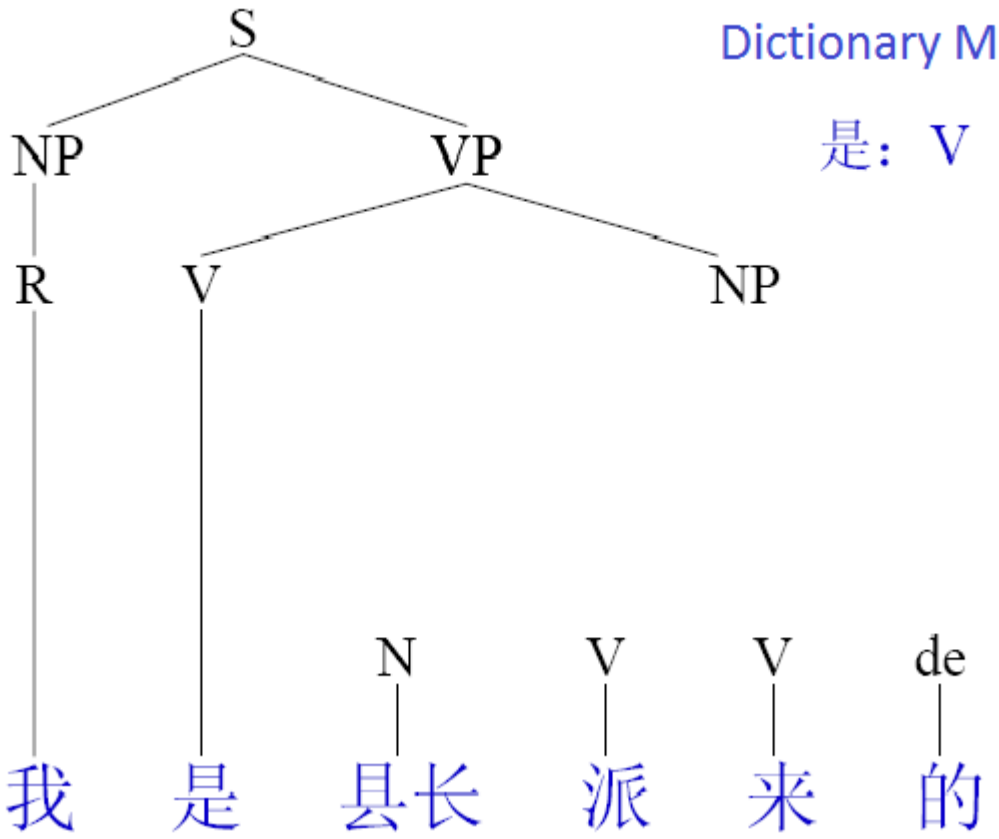
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary Match Succeeds

是：V

Rule

(7) NP → R

(8) NP → N

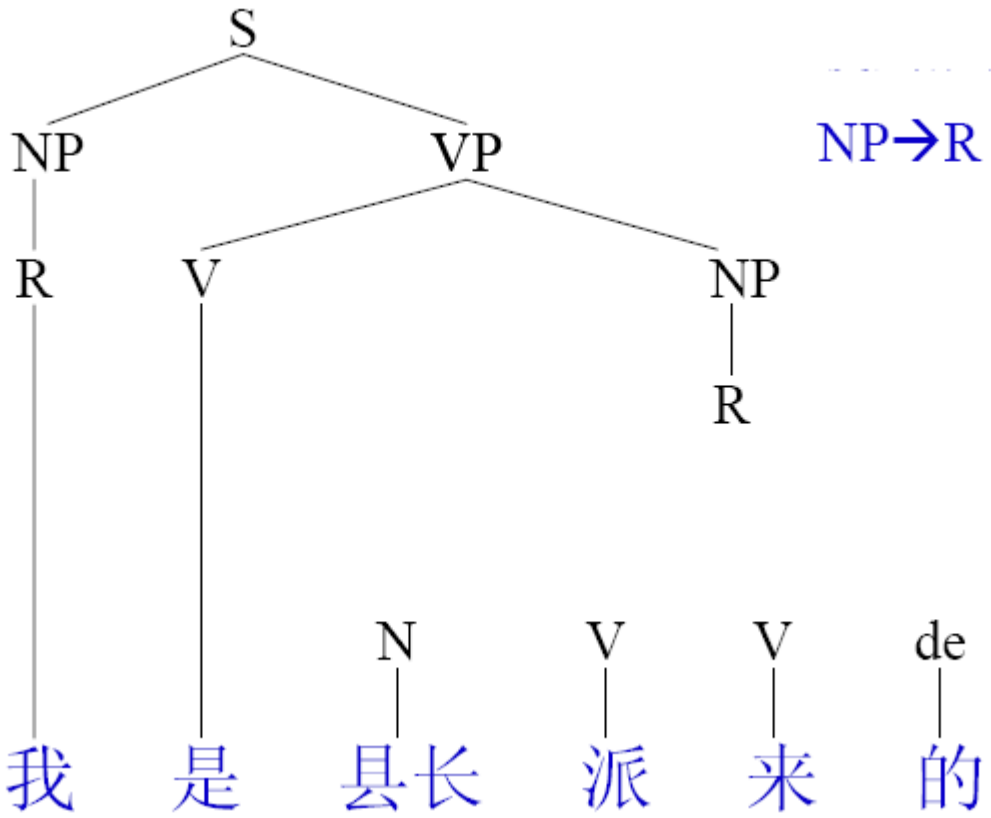
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



NP→R

Rule

(7) NP → R

(8) NP → N

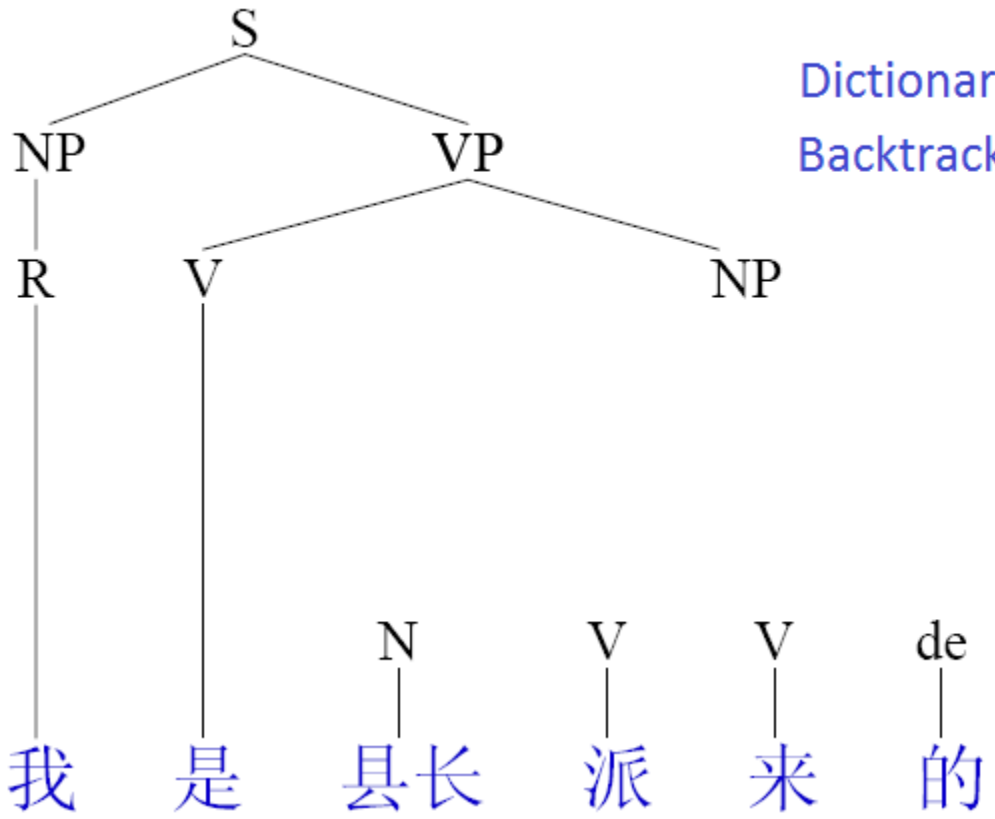
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary Match Fails

Backtracking

Rule

(7) NP → R

(8) NP → N

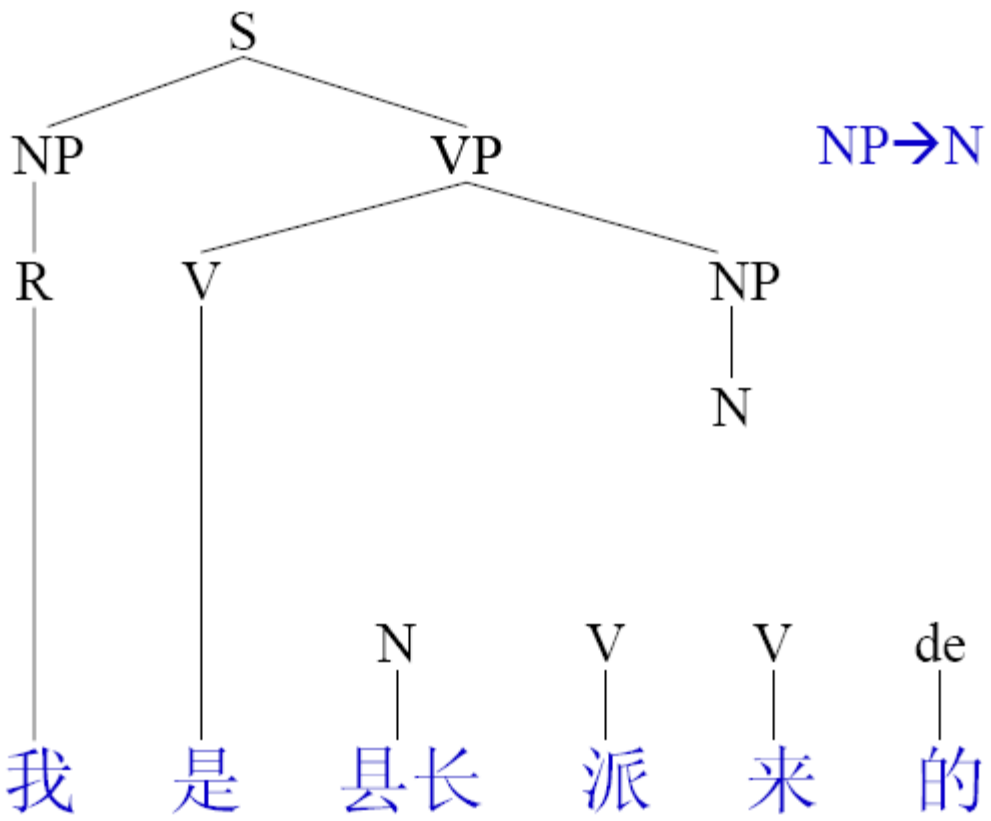
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



NP→N

Rule

(7) NP → R

(8) NP → N

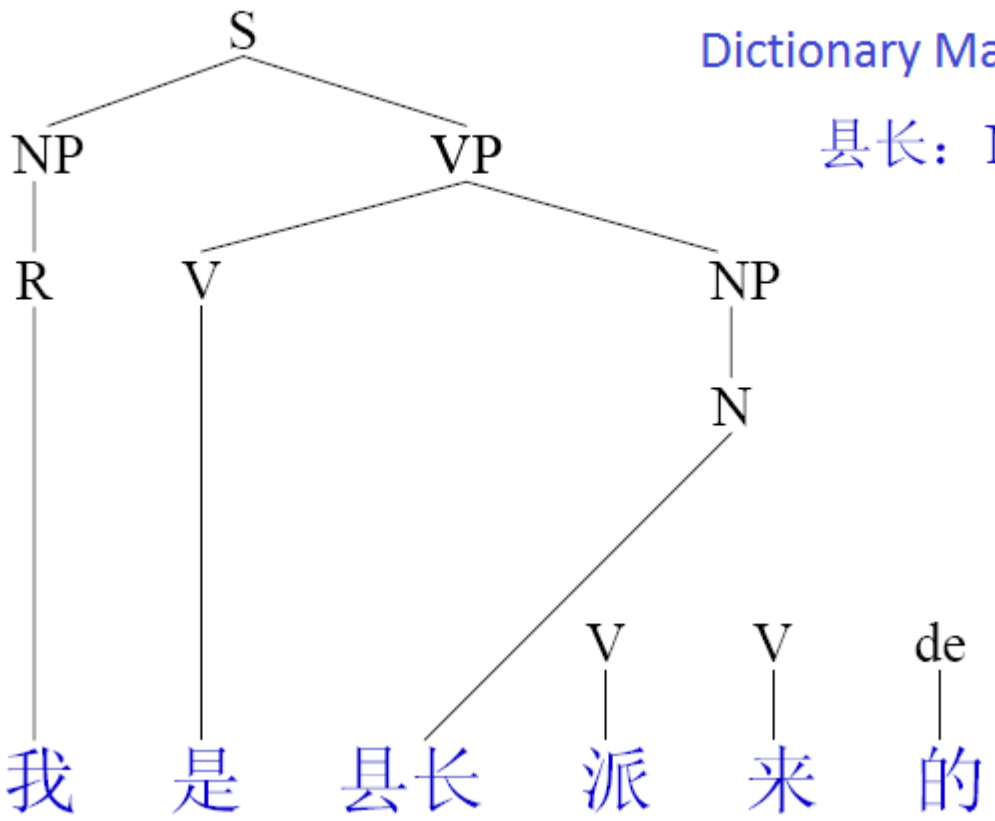
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary Match Succeeds

县长：N

Rule

(7) NP → R

(8) NP → N

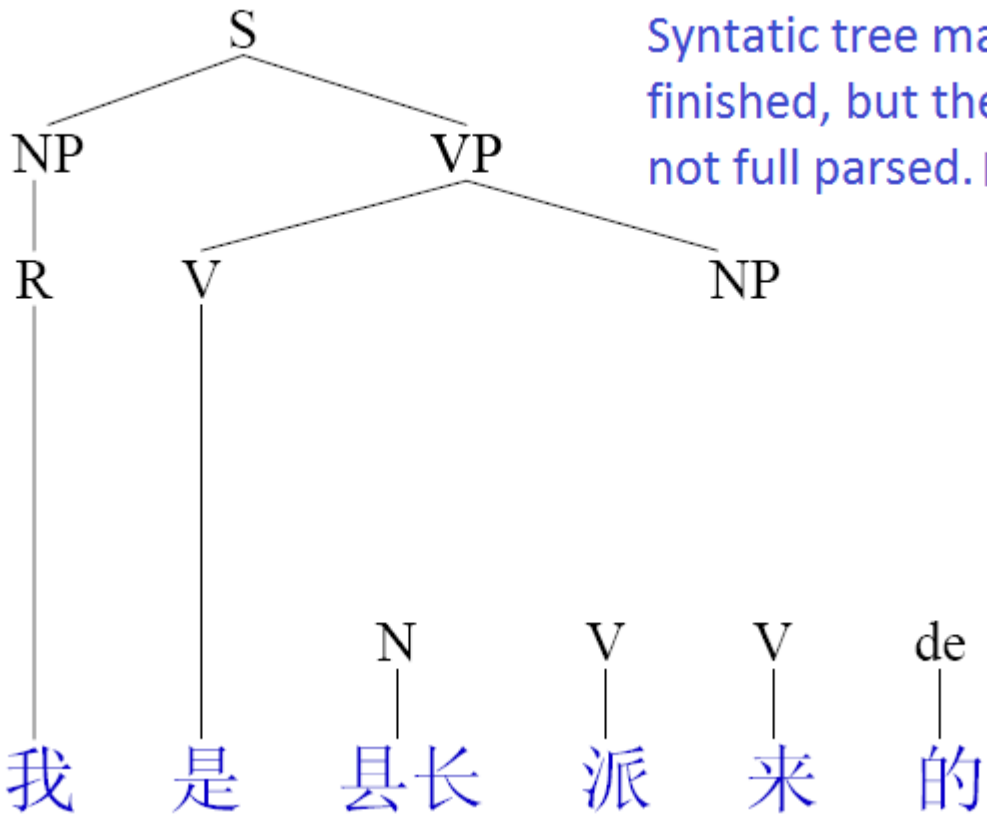
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Syntatic tree matching finished, but the sentence is not full parsed. Backtracking

Rule

(7) NP → R

(8) NP → N

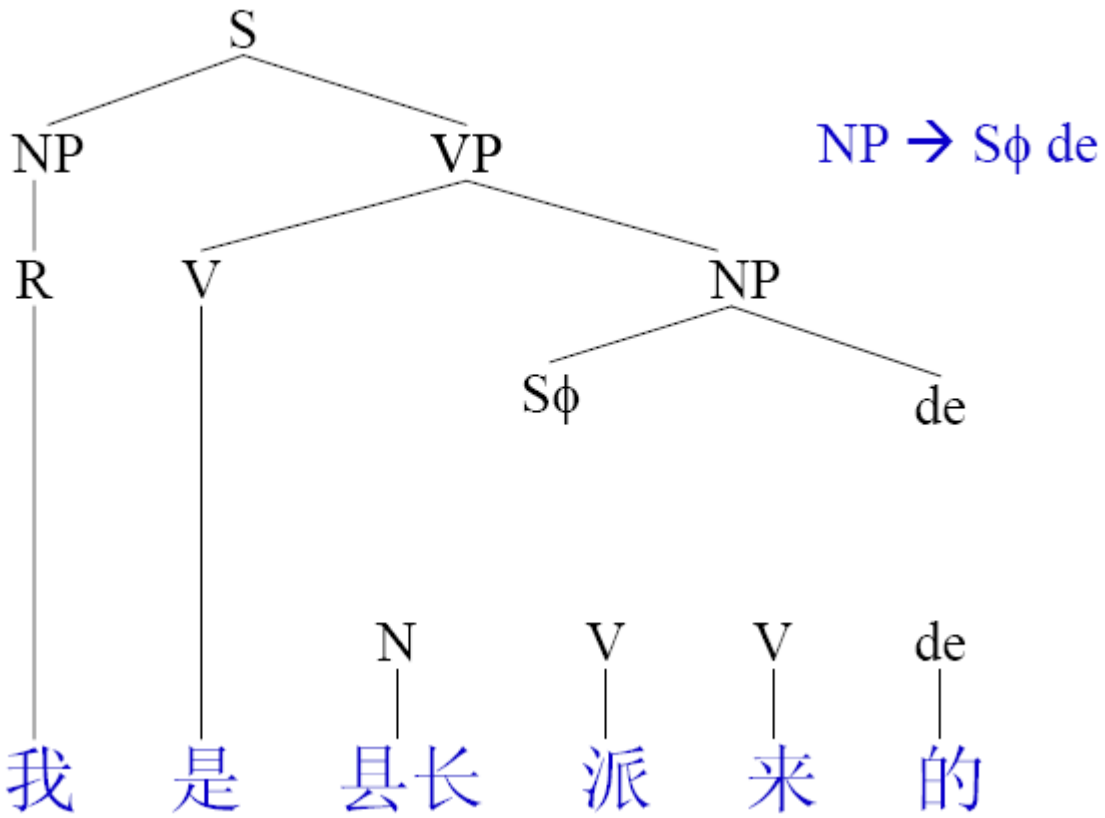
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



NP → Sϕ de

Rule

(7) NP → R

(8) NP → N

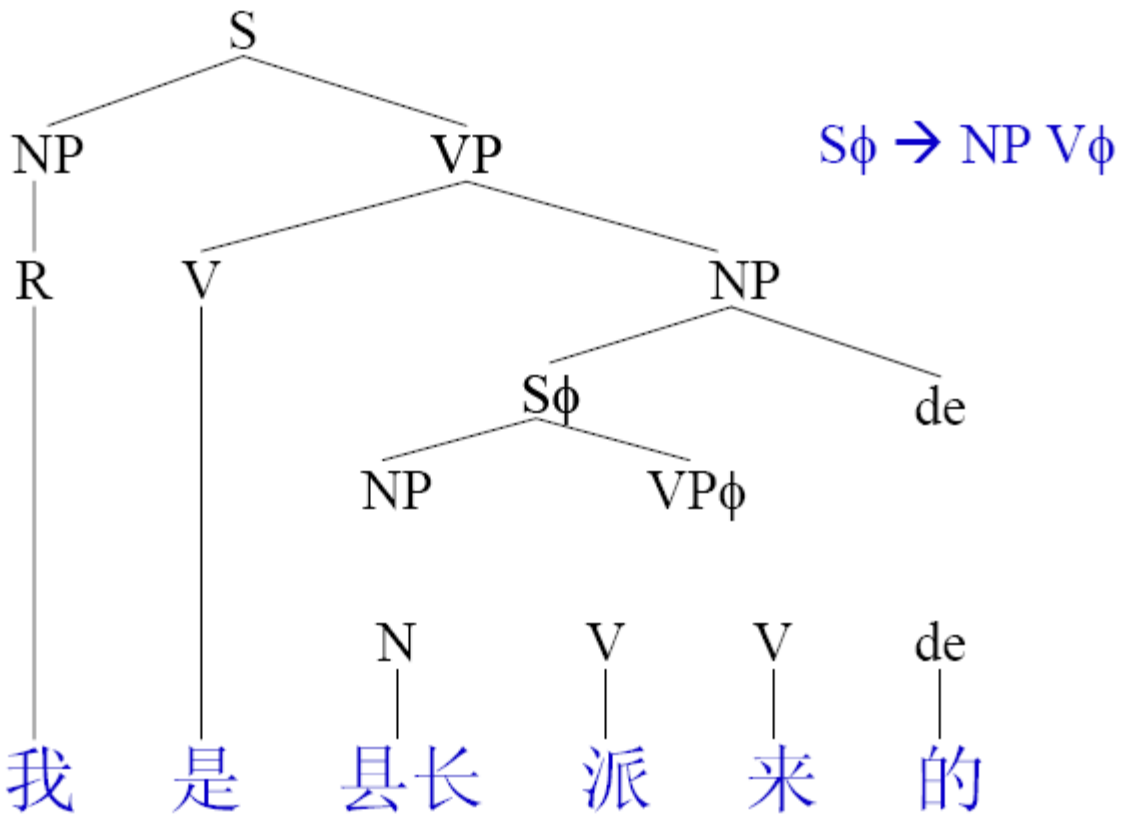
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Sϕ → NP Vϕ

Rule

(7) NP → R

(8) NP → N

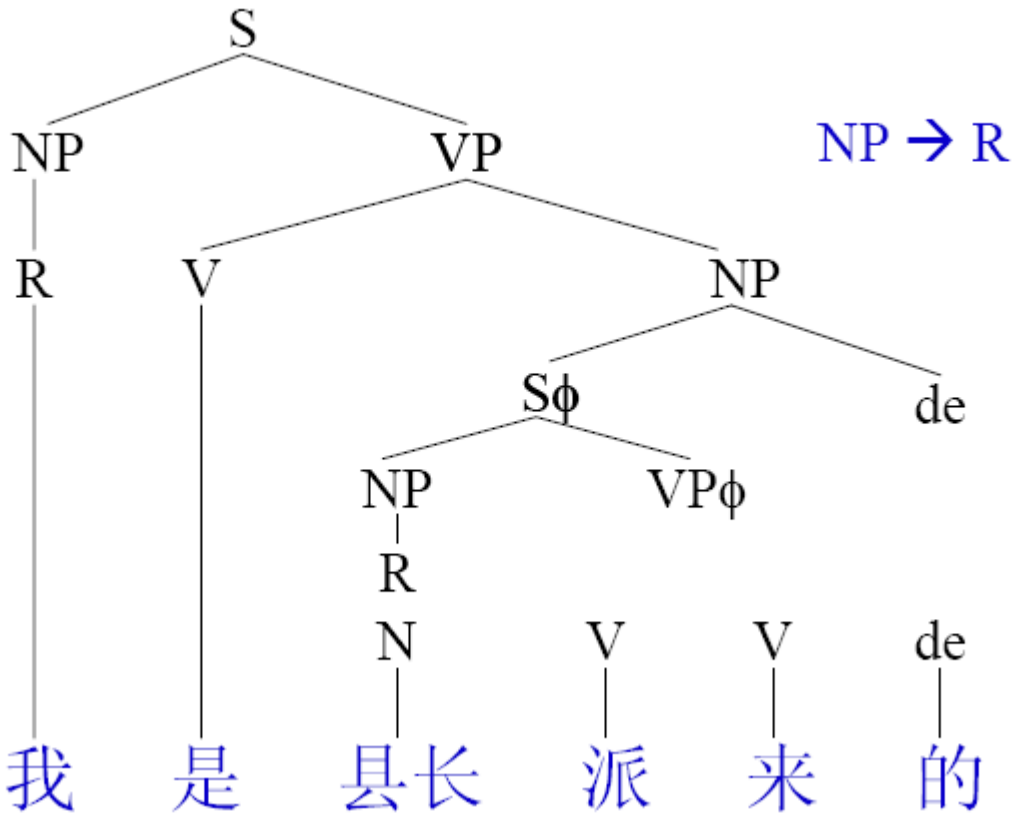
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



NP → R

Rule

(7) NP → R

(8) NP → N

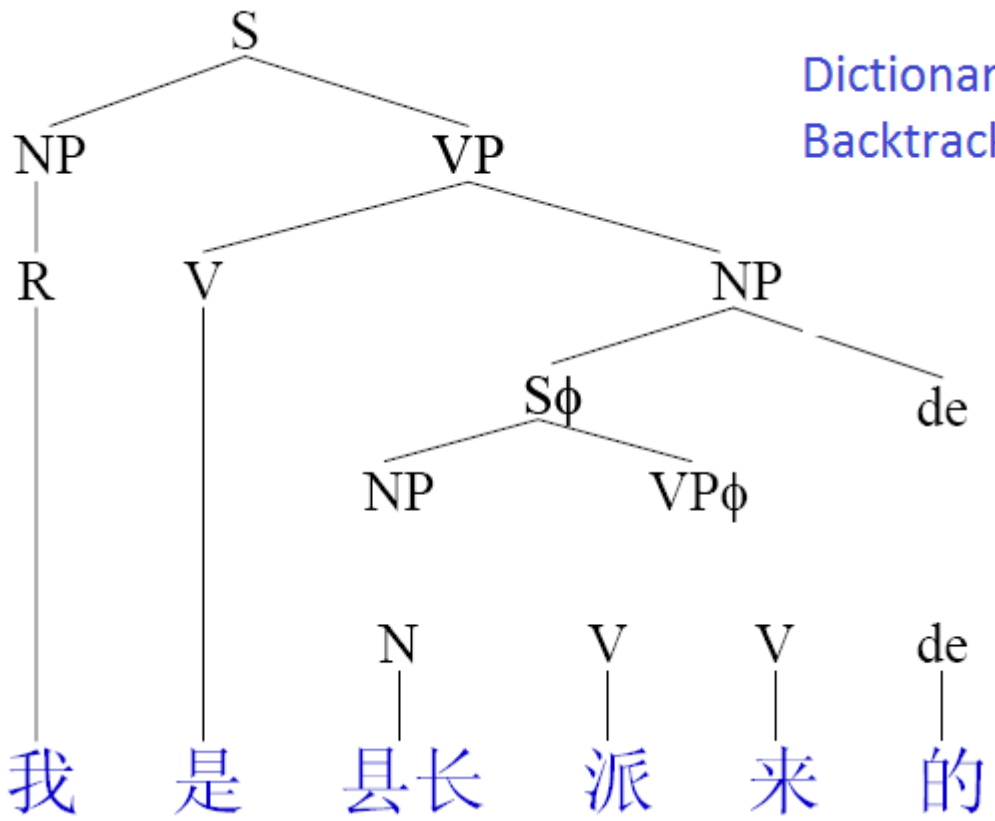
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary match fails

Backtracking

Rule

(7) NP → R

(8) NP → N

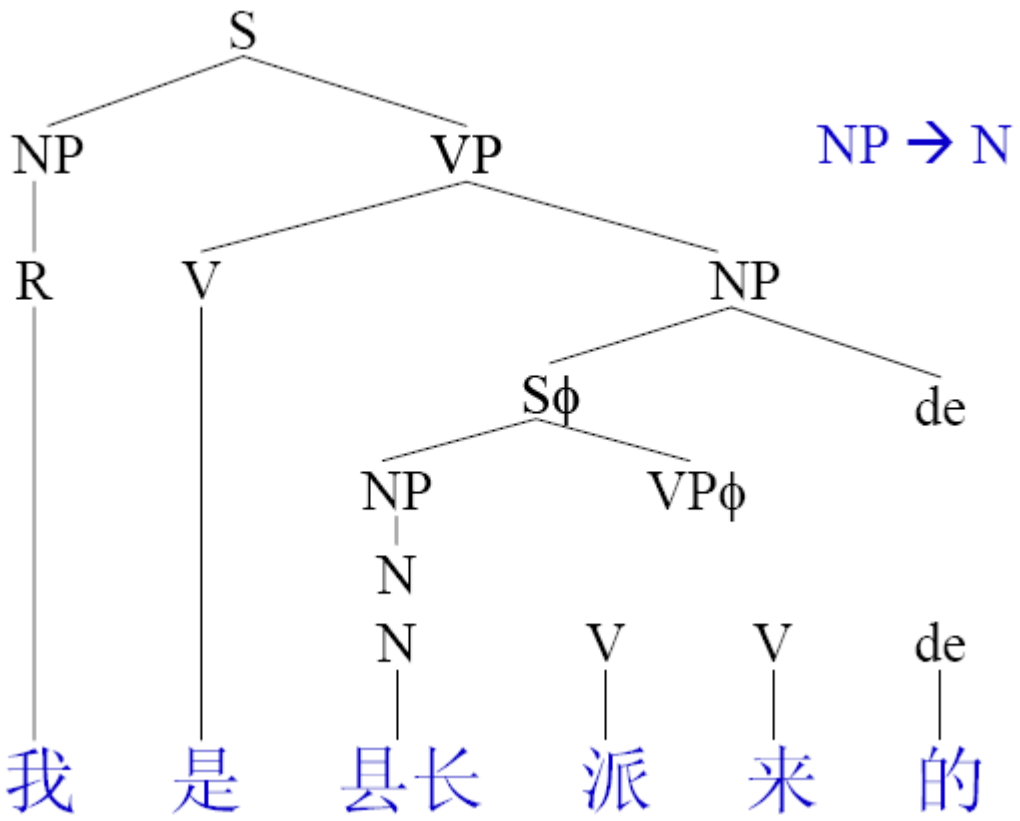
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



NP → N

Rule

(7) NP → R

(8) NP → N

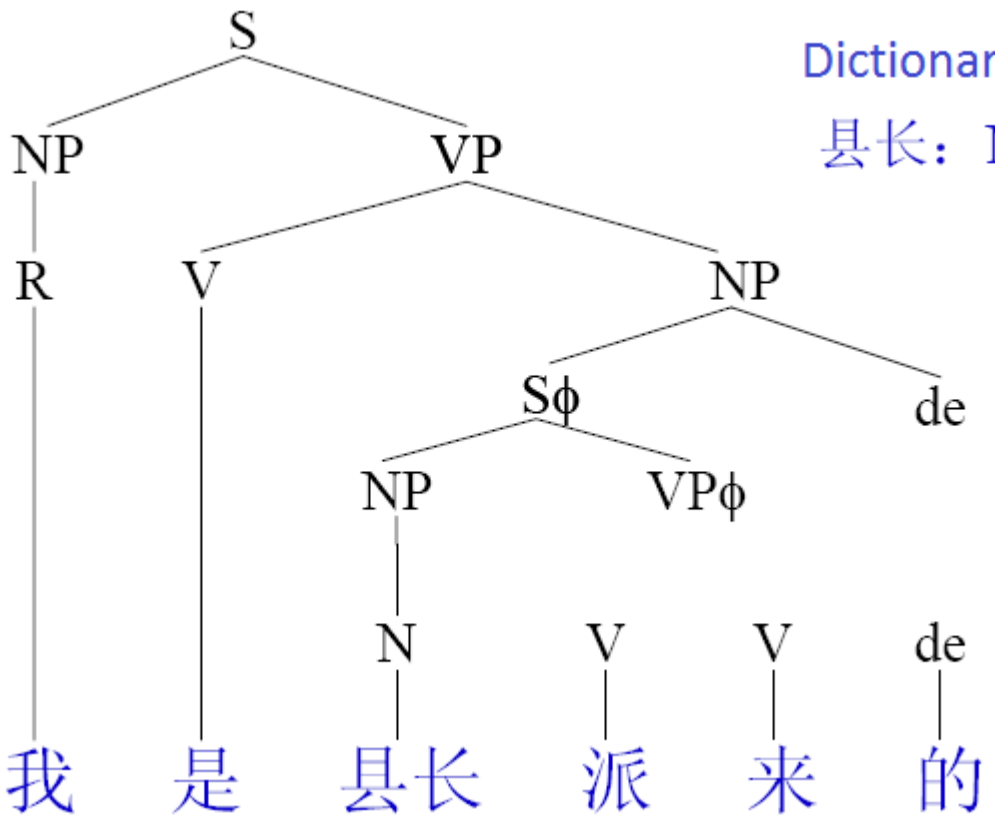
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary matched

县长：N

Rule

(7) NP → R

(8) NP → N

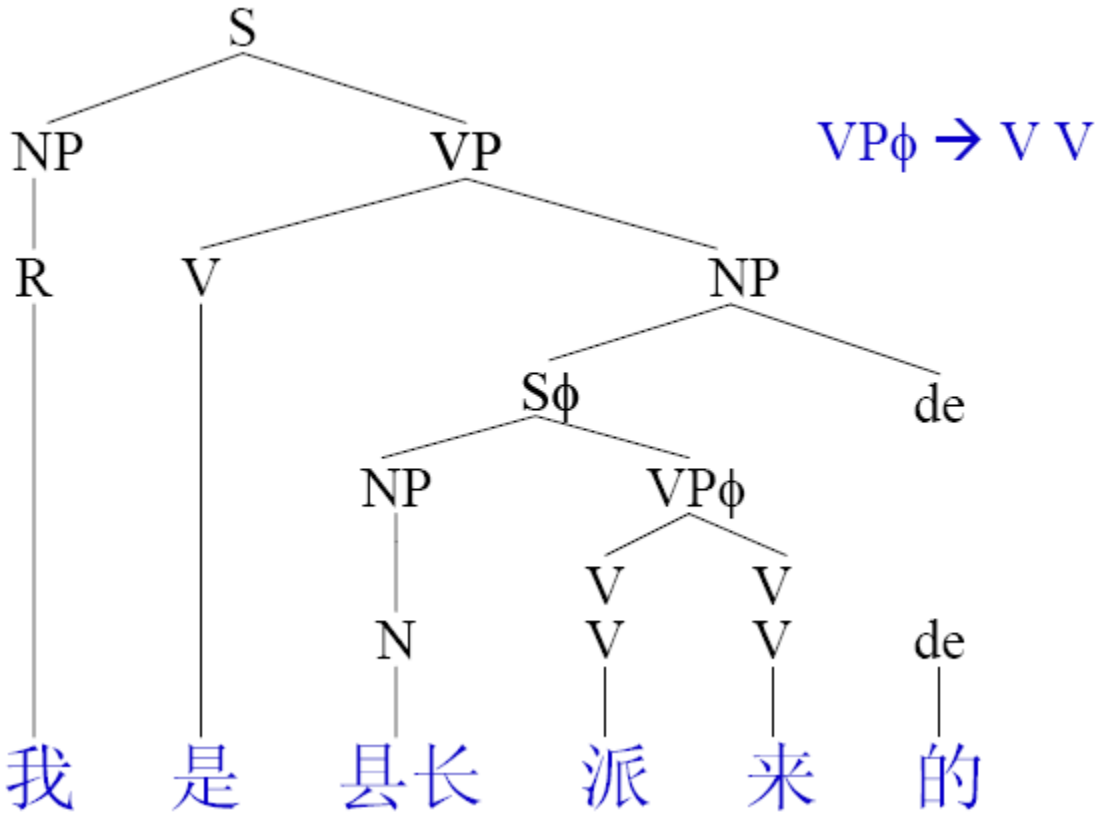
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



VPϕ → V V

Rule

(7) NP → R

(8) NP → N

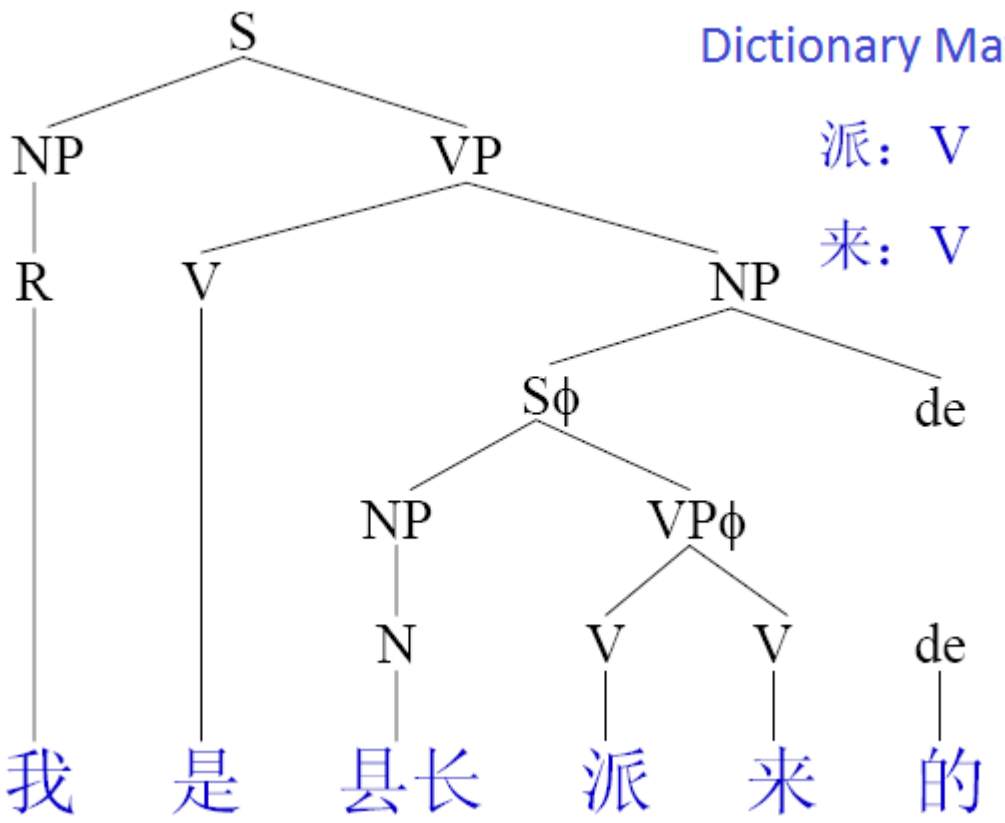
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary Matched

派：V

来：V

Rule

(7) NP → R

(8) NP → N

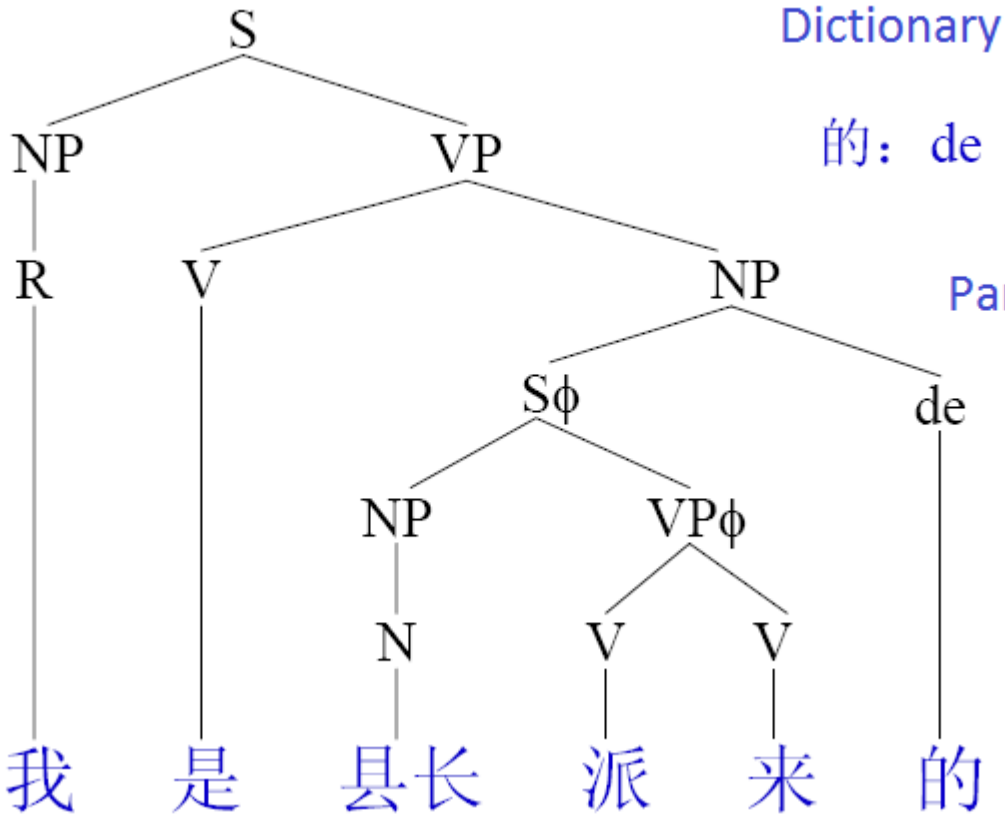
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Dictionary Matched

的：de

Parse Succeeds

Bottom-Up Parsing Illustration -1

Rule

(7) NP → R

(8) NP → N

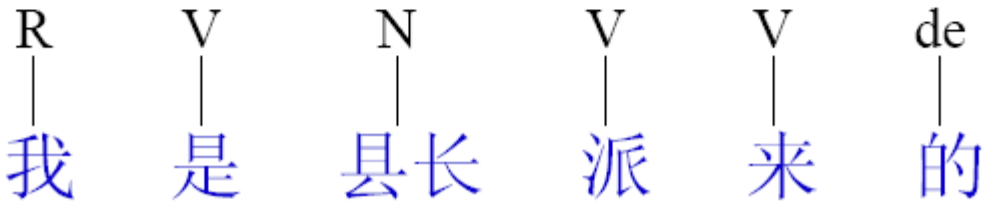
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



2

Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

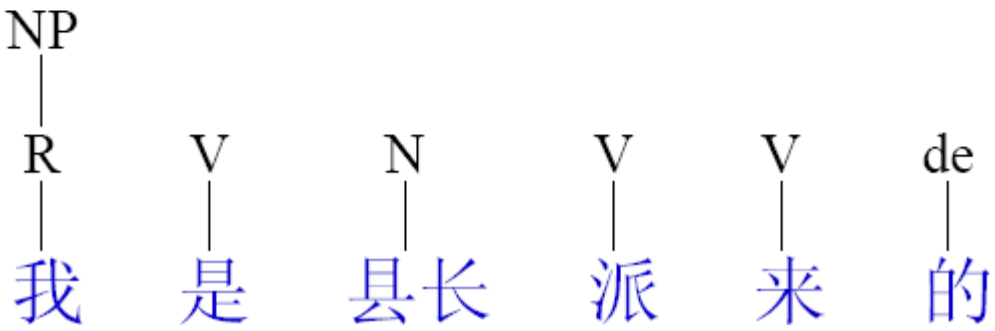
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

NP→R



3

Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

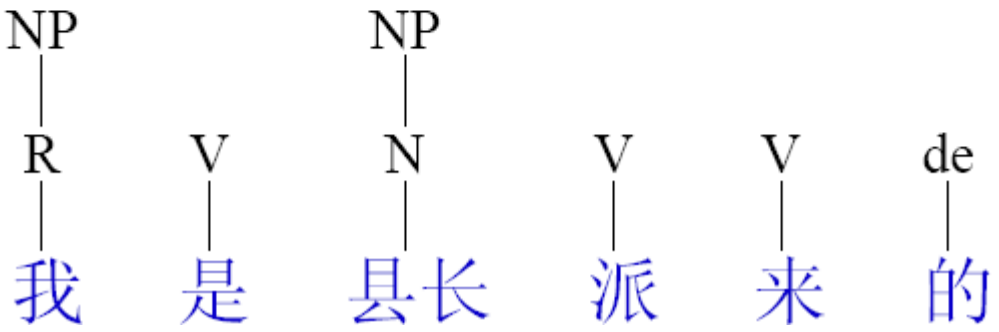
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

NP→N



4

Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

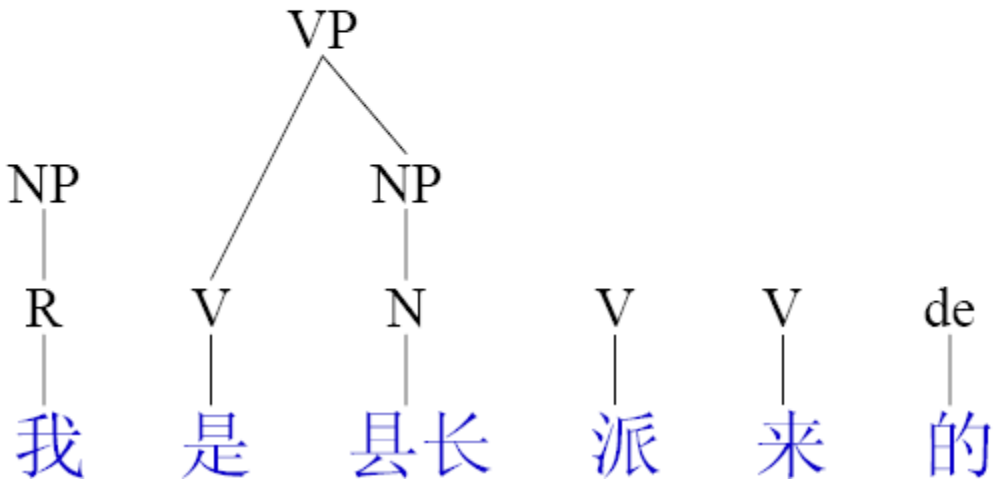
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

VP→V NP



5

Rule

(7) NP → R

(8) NP → N

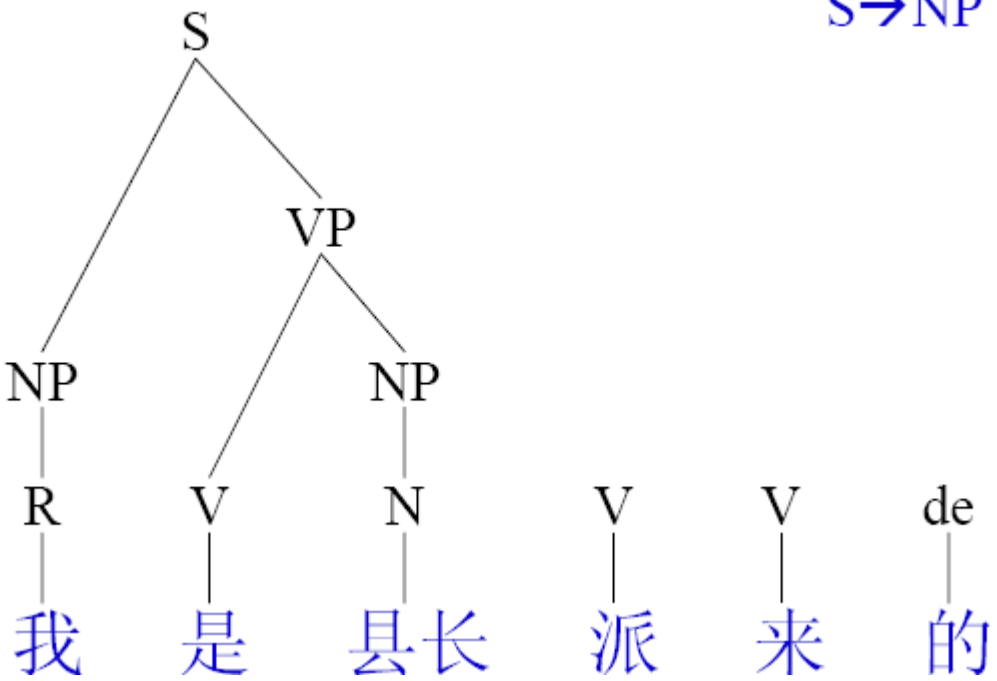
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



S→NP VP

6

Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

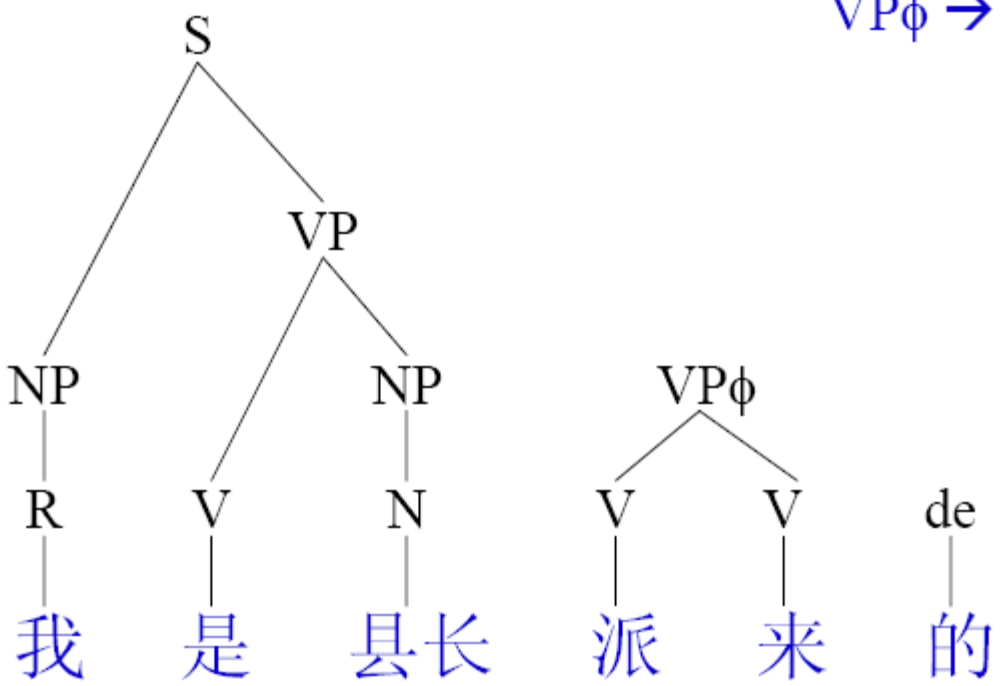
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

VPϕ →V V



Rule

(7) NP → R

(8) NP → N

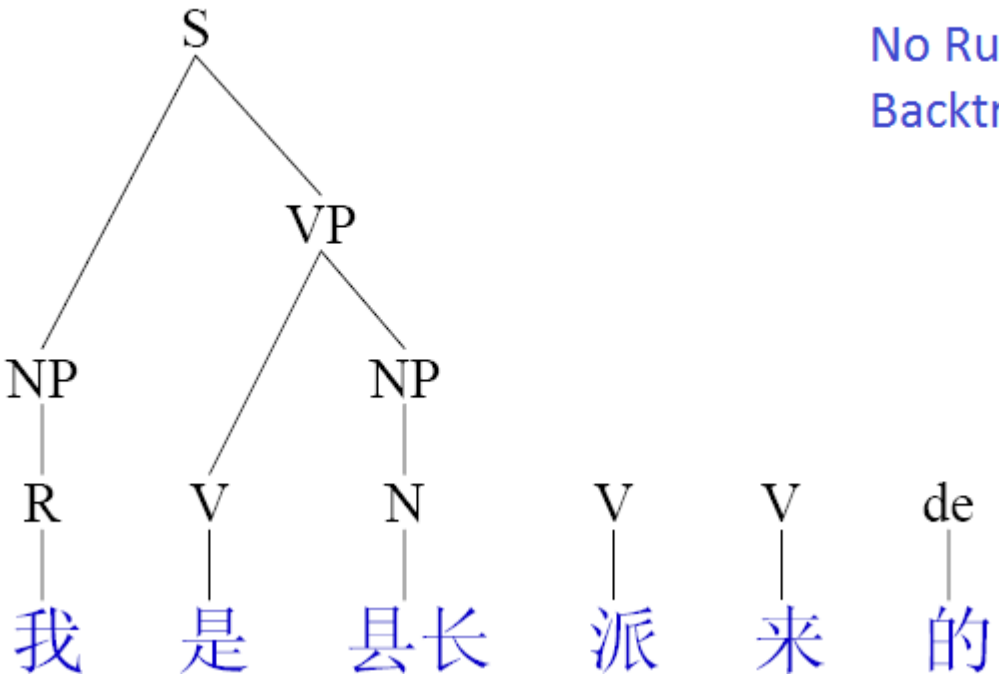
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



No Rule Apply
Backtracking

Rule

(7) NP → R

(8) NP → N

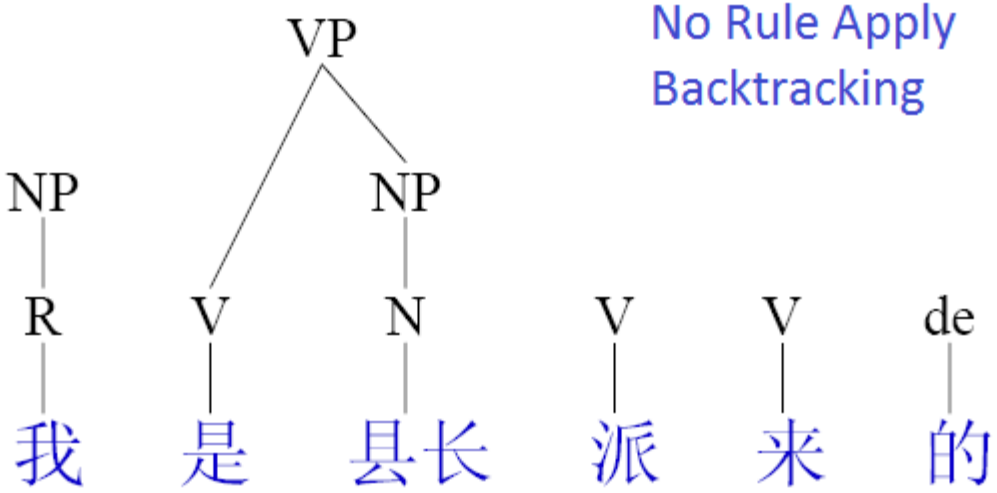
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



No Rule Apply

Backtracking

Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

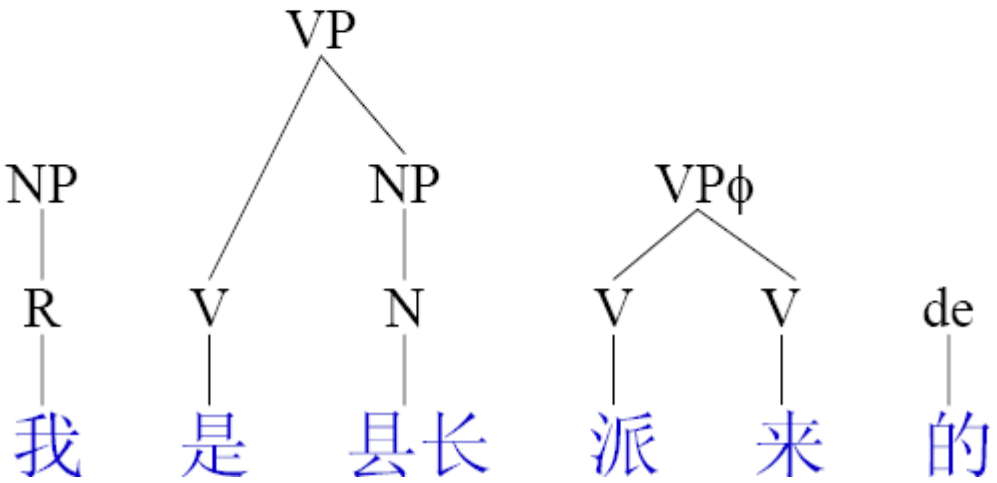
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

VPϕ →V V



Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

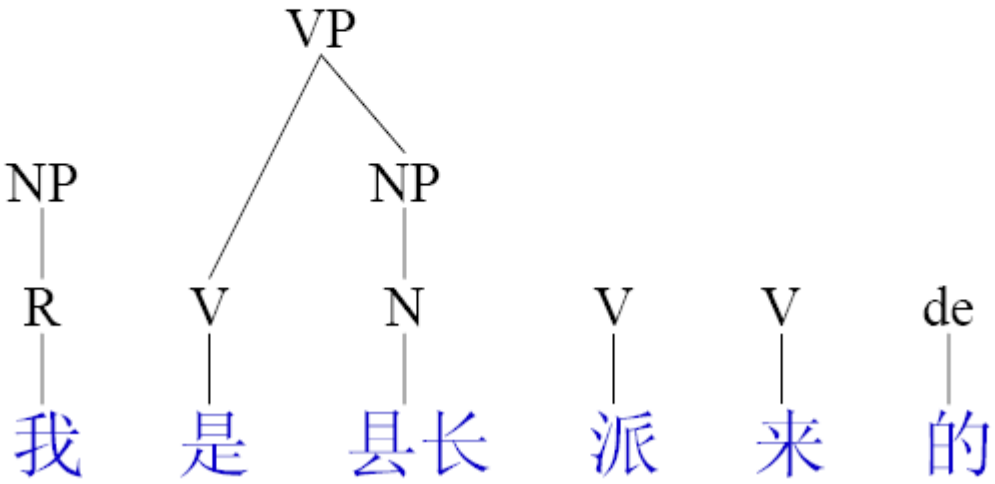
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

No Rule Apply
Backtracking



Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

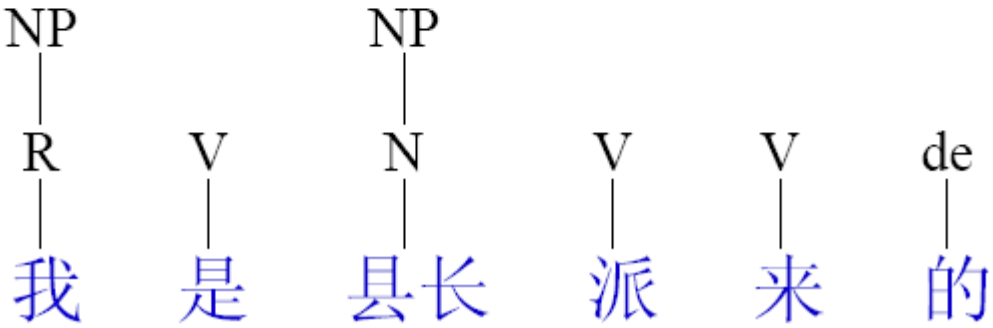
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

No Rule Apply
Backtracking



Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

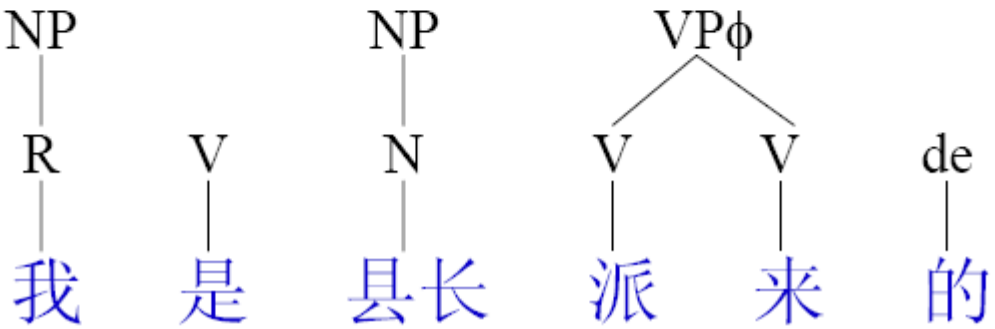
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

VPϕ →V V



Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

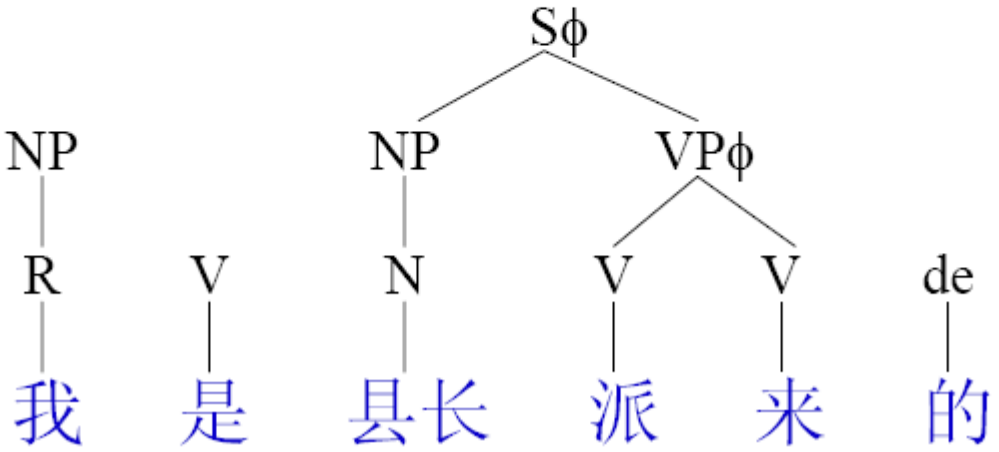
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

Sϕ →NP VPϕ



Rule

(7) NP → R

(8) NP → N

(9) NP → Sϕ de

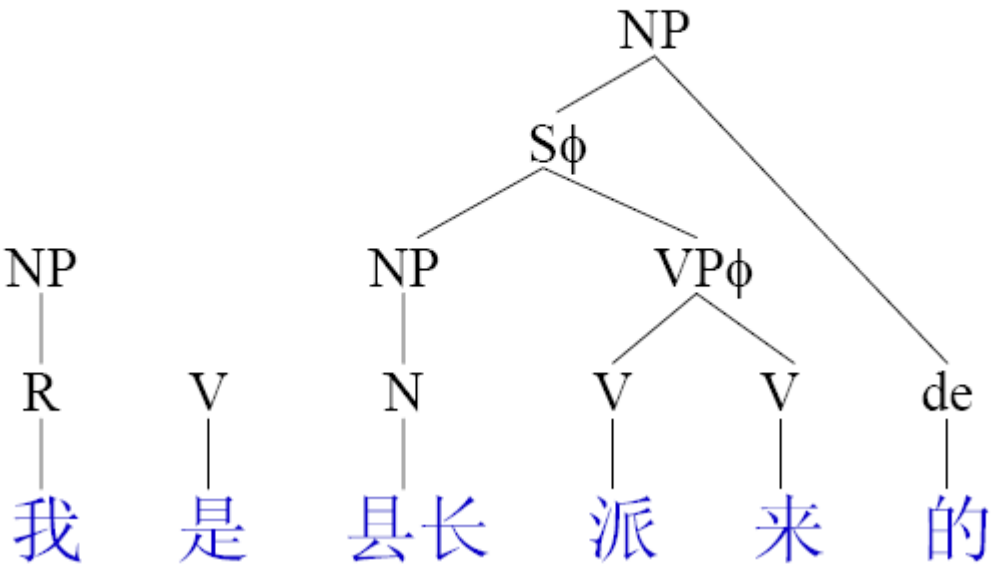
(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP

NP → Sϕ de



Rule

(7) NP → R

(8) NP → N

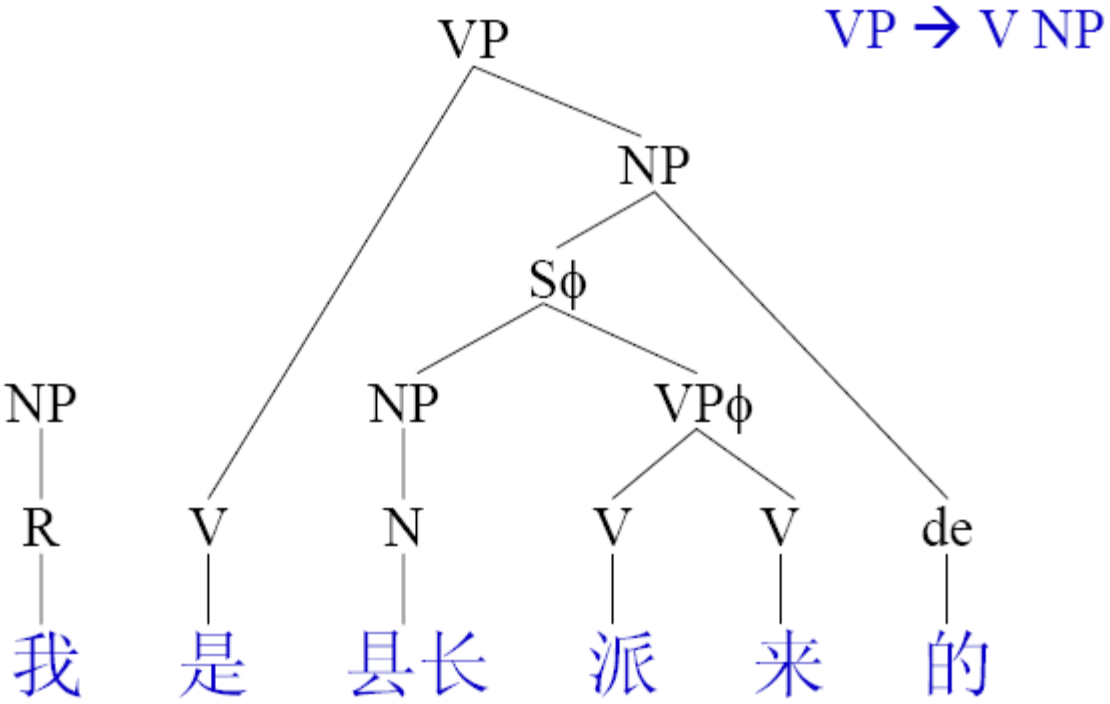
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



Rule

(7) NP → R

(8) NP → N

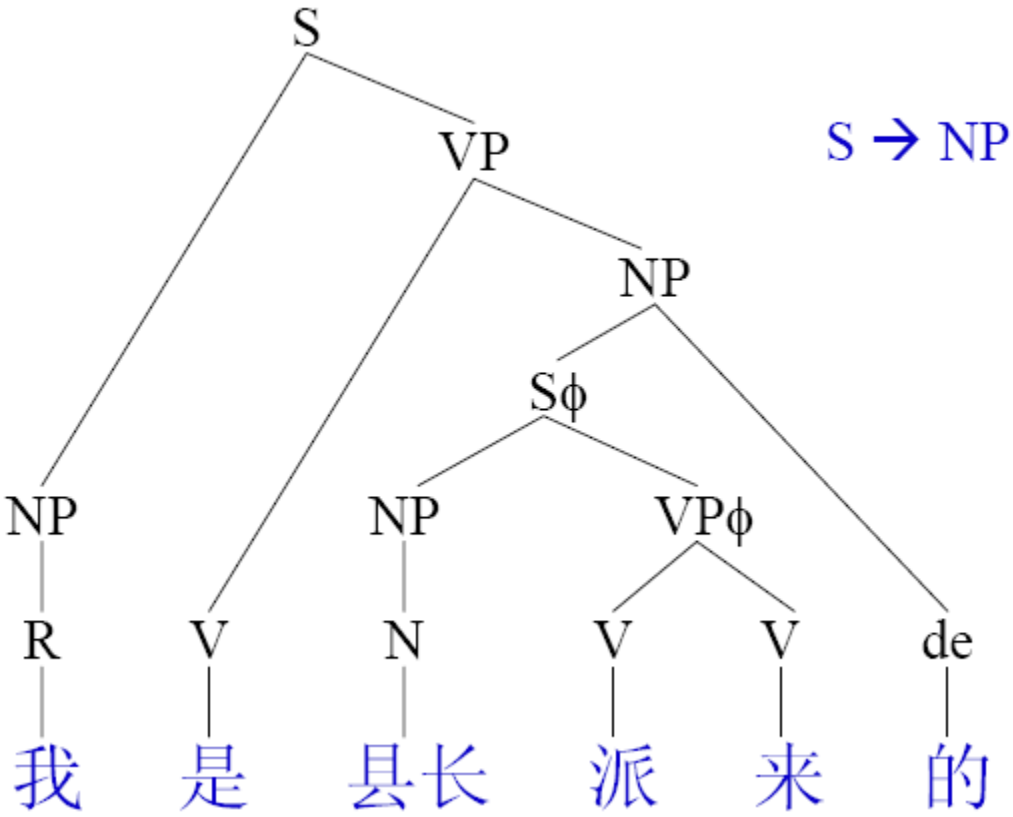
(9) NP → Sϕ de

(10) VPϕ → V V

(11) VP → V NP

(12) Sϕ → NP VPϕ

(13) S → NP VP



S → NP VP

Top-Down Parsing - Remarks

- Top-down parsers do well if there is useful grammar-driven control: search can be directed by the grammar.
- Not too many different rules for the same category.
- Not too much distance between non-terminal and terminal categories.
- Top-down is unsuitable for rewriting parts of speech (pre-terminals) with words (terminals). In practice that is always done bottom-up as lexical lookup.

Bottom-Up Parsing - Remarks

- It is data-directed: it attempts to parse the words that are there.
- Does well, e.g. for lexical lookup.
- Does badly if there are many rules with similar RHS categories.
- Insufficient when there is great lexical ambiguity (grammar driven control might be helpful here)
- Empty categories: termination problem unless rewriting of empty constituents is somehow restricted (but it's generally incomplete)

CFG Parsing Algorithms

- Shift-Reduce Algorithm
- CYK Algorithm
- Marcus Analysis Algorithm
- Early Algorithm (Top-down)
- Tomita Algorithm
- Chart Algorithm

Shift-Reduce Algorithm

- Similar to Push Down Automata
- Basic Data structure: Stack
- Four Operations in Shift-Reduced Algorithm
  - Shift 移进:
    - push next input on to top of stack
    - = Shift a terminator to top of stack from left of sentence
  - Reduce 归约 R:
    - top of stack should match RHS of rule
    - replace top of stack with LHS of rule
    - = replace several symbols on the top of stack to one symbol
  - Accept (shift EOF & can reduce what remains on stack to start symbol)
  - Error (shift all words to the stack, but stack have symbols more than S and cannot be reduced)

Shift-Reduce Algorithm

	Stack	Input	Operation	Rule
1	#	我 是 县长	Shift	
2	# 我	是 县长	Reduce	R→ 我
3	# R	是 县长	Reduce	NP→R
4	# NP	是 县长	Shift	
5	# NP 是	县长	Reduce	V→是
6	# NP V	县长	Shift	
7	# NP V 县长		Reduce	N→县长
8	# NP V N		Reduce	NP→N
9	# NP V NP		Reduce	VP→V NP
10	# NP VP		Reduce	S→NP VP
11	# S		Accept	

Ambiguities in Shift-Reduce Algorithm

- Two kinds of Ambiguities
  - Shift-Reduce ambiguity: The word can be both shifted and reduced.
  - Reduce-Reduce ambiguity: The word can be reduced by different rules.

- Backtracking
  - For the ambiguous operations, give a selection order.
  - Breakpoint Information: Maintain the non-terminator in stack and breakpoint information including the stack information and optional operations on the breakpoint.

Backtracking

- Backtracking Strategy
  - Shift-Reduce Ambiguity: Reduce first, Shift second.
  - Reduce-Reduce Ambiguity: Sort the rules according to their priority. Apply the high priority rule first.

- Breakpoint Information
  - Current Applied Rule.
  - Candidate Rules.
  - Substituted node during reduce operation.

Shift-Reduce Algorithm: Illustration

- Sort the rule set

Rule	Dictionary
(7) NP → R	(1) R→我
(8) NP → N	(2) N →县长
(9) NP → Sϕ de	(3) V →是
(10) VPϕ → V V	(4) V →派
(11) VP → V NP	(5) V →来
(12) Sϕ → NP VPϕ	(6) de →的
(13) S → NP VP	

2

	Stack	Input	Opr	Rule
1	#	我 是 县长 派 来 的	Shift	
2	# 我	是 县长 派 来 的	Reduce	(1) R→ 我
3	# R (1)	是 县长 派 来 的	Reduce	(7) NP→R
4	# NP (7)	是 县长 派 来 的	Shift	
5	# NP (7) 是	县长 派 来 的	Reduce	(3) V→是
6	# NP (7) V (3)	县长 派 来 的	Shift	
7	# NP (7) V (3)县长	派 来 的	Reduce	(2) N→县长
8	# NP (7) V (3) N (2)	派 来 的	Reduce	(8) NP→N
9	# NP (7) V (3) NP (8)	派 来 的	Reduce	(11) VP→V NP
10	# NP (7) VP (11)	派 来 的	Reduce	(13) S→NP VP
11	# S (13)	派 来 的	Shift	

3

	Stack	Input	Opr	Rule
12	# S (13) 派	来 的	Reduce	(4) V→派
13	# S (13) V(4)	来 的	Shift	
14	# S (13) V(4) 来	的	Reduce	(5) V→来
15	# S (13) V(4) V(5)	的	Reduce	(10) VPϕ → V V
16	# S (13) VPϕ (10)	的	Shift	
17	# S (13) VPϕ (10) 的		Reduce	(6) de→的
18	# S (13) VPϕ (10) de(6)		Back	
19	# S (13) VPϕ (10) 的		Back	
20	# S (13) VPϕ (10)	的	Back	
21	# S (13) V(4) V (5)	的	Back	
22	# S (13) V(4) 来	的	Back	

4

	Stack	Input	Opr	Rule
23	# S (13) V(4)	来 的	Back	
24	# S (13) 派	来 的	Back	
25	# S (13)	派 来 的	Back	
26	# NP (7) VP (11)	派 来 的	Shift	(13) S→NP VP
27	# NP (7) VP (11) 派	来 的	Reduce	(4) V→派
28	# NP (7) VP (11) V(4)	来 的	Shift	
29	# NP (7) VP (11) V(4) 来	的	Reduce	(5) V→来
30	# NP (7) VP (11) V(4) V(5)	的	Reduce	(10) VPϕ → V V
31	# NP (7) VP (11) VPϕ (10)	的	Shift	
32	# NP (7) VP (11) VPϕ (10) 的		Reduce	(6) de→的
33	# NP (7) VP (11) VPϕ (10) de(6)		Back	

5

	Stack	Input	Opr	Rule
34	# NP (7) VP (11) VPϕ (10) 的		Back	
35	# NP (7) VP (11) VPϕ (10)	的	Back	
36	# NP (7) VP (11) V(4) V(5)	的	Back	
37	# NP (7) VP (11) V(4) 来	的	Back	
38	# NP (7) VP (11) V(4)	来 的	Back	
39	# NP (7) VP (11) 派	来 的	Back	
40	# NP (7) VP (11)	派 来 的	Back	
41	# NP (7) VP (11)	派 来 的	Back	
42	# NP (7) V (3) NP (8)	派 来 的	Shift	
43	# NP (7) V (3) NP (8) 派	来 的	Reduce	(4) V→派
44	# NP (7) V (3) NP (8) V(4)	来 的	Shift	

6

	Stack	Input	Opr	Rule
45	# NP (7) V (3) NP (8) V(4)来	的	Reduce	(5) V→来
46	# NP (7) V (3) NP (8) V(4) V(5)	的	Reduce	(10) VPϕ → V V
47	# NP (7) V (3) NP (8) VPϕ (10)	的	Reduce	(12) Sϕ → NP VPϕ
48	# NP (7) V (3) Sϕ (12)	的	Shift	
49	# NP (7) V (3) Sϕ (12) 的		Reduce	(6) de→的
50	# NP (7) V (3) Sϕ (12) de (6)		Reduce	(9) NP → Sϕ de
51	# NP (7) V (3) NP (9)		Reduce	(11) VP→V NP
52	# NP (7) VP (11)		Reduce	(13) S→NP VP
53	# S (13)		Accept	

Summary of Shift-Reduce Algorithm

- Shift-Reduce algorithm is a bottom-up analysis .

- In order to obtain all possible parsing results, force backtracking is conducted for each success analysis, which leads to lots of redundant operations .

- Revisions:
  - Introduce conditional operation rule.
  - Introduce contextual operation rule.
  - Introduce Cache (Marcus Algorithm).

CKY Parsing

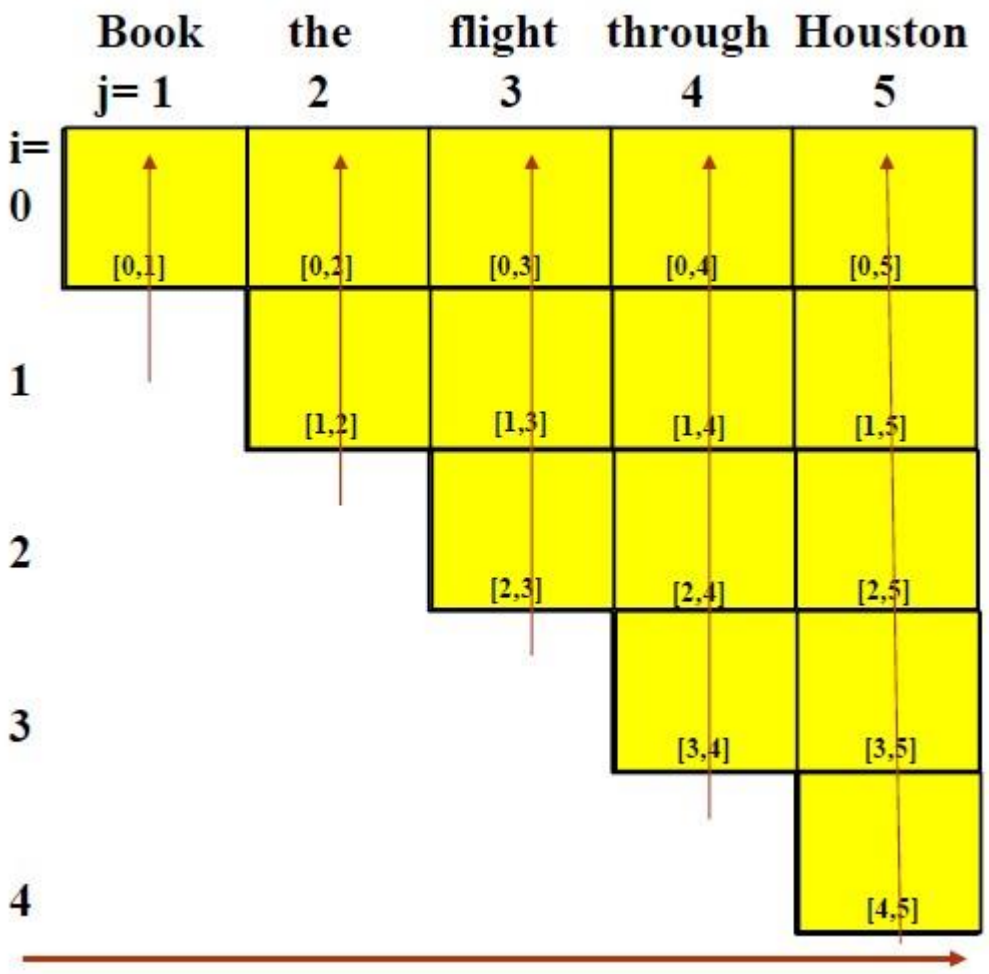
- CKY (Cocke-Kasami-Younger) algorithm.
- Bottom-Up Parsing.
- The productive rules must be formalized.
- Dynamic Parsing (*O(n³)*).
- The rules must be formalized to Chomsky normal.
- form (CNF)， where the RHS of this rule must be 2 non-terminal or 1 terminal.
- Parsing from Bottom in a table.
- The parsing for a string depends on the parsing for its sub-strings

CYK Parsing: Rule Formalization

Original Grammar	Chomsky Normal Form
S → NP VP	S → NP VP
S → Aux NP VP	S → X1 VP
	X1 → Aux NP
S → VP	S → book | include | prefer
	S → Verb NP
	S → VP PP
NP → Pronoun	NP → I | he | she | me
NP → Proper-Noun	NP → Houston | NWA
NP → Det Nominal	NP → Det Nominal
Nominal → Noun	Nominal → book | flight | meal | money
Nominal → Nominal Noun	Nominal → Nominal Noun
Nominal → Nominal PP	Nominal → Nominal PP
VP → Verb	VP → book | include | prefer
VP → Verb NP	VP → Verb NP
VP → VP PP	VP → VP PP
PP → Prep NP	PP → Prep NP

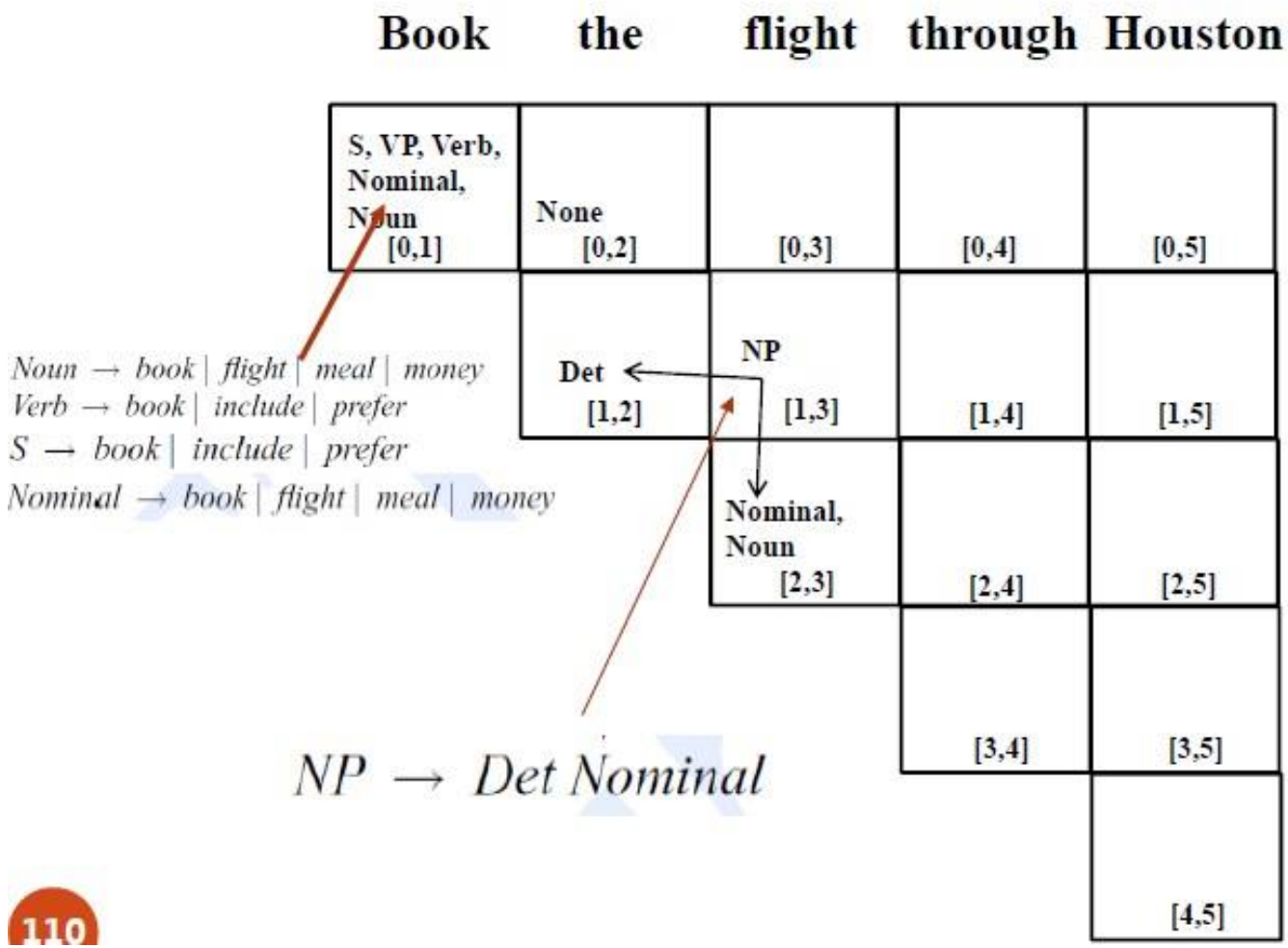
Det → *that* | *this* | *a*
Noun → *book* | *flight* | *meal* | *money*
Verb → *book* | *include* | *prefer*
Pronoun → *I* | *she* | *me*
Proper-Noun → *Houston* | *TWA*
Aux → *does*
Preposition → *from* | *to* | *on* | *near* | *through*

CYK Parsing

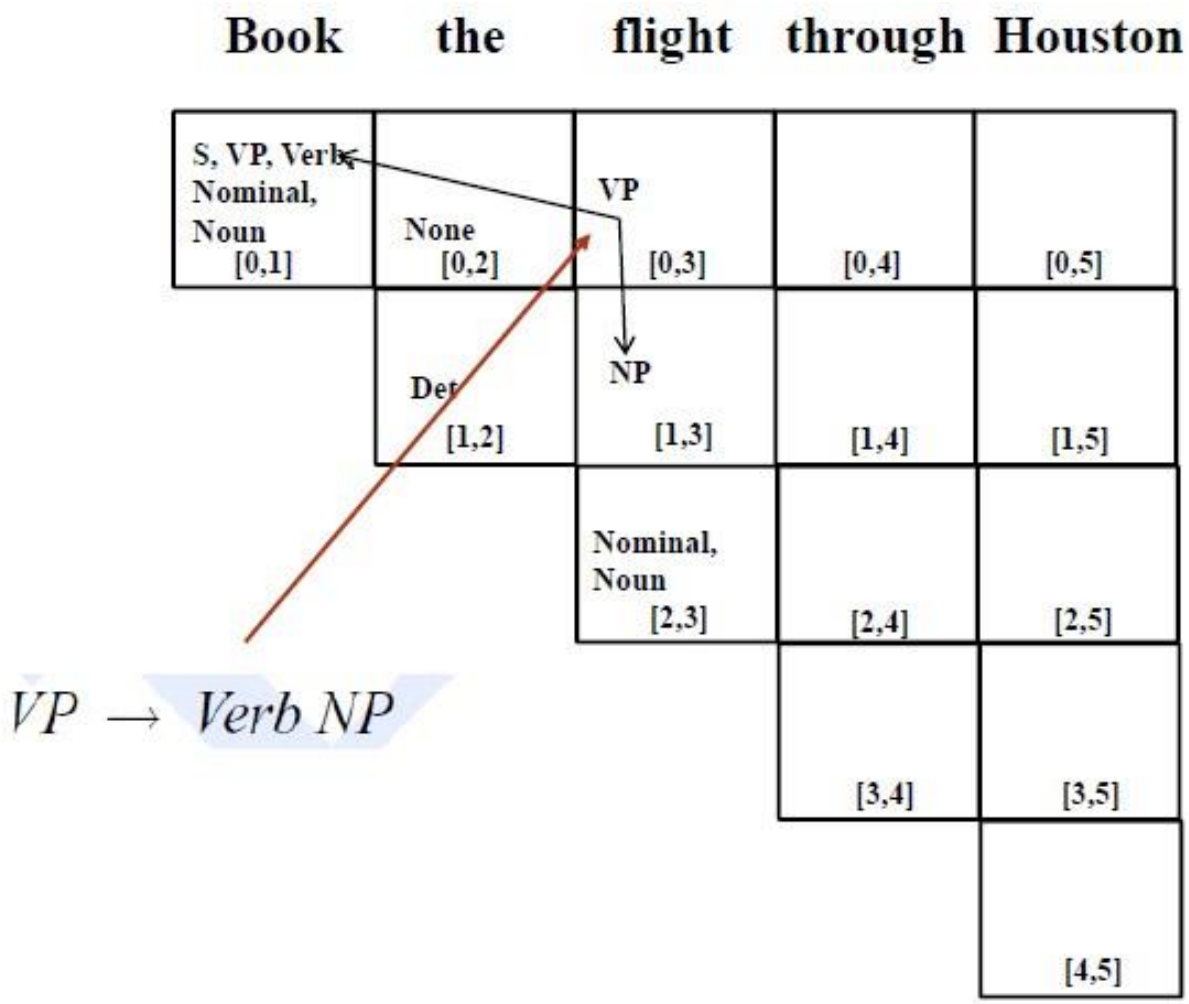


Cell[i,j] contains all of Non-terminals which covers $Word_{i+1}$ to $Word_j$

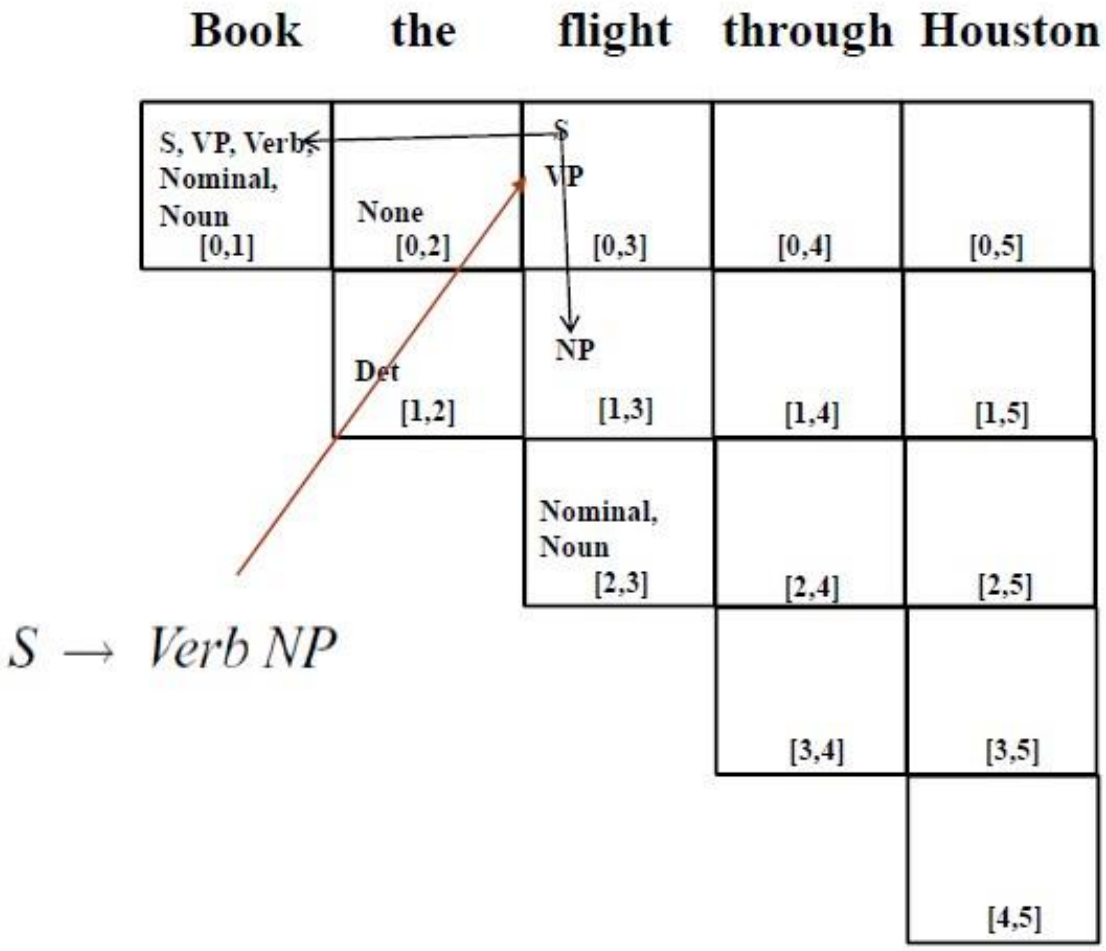
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
[0,3]
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
[1,4]
[1,5]
Nominal, Noun
[2,3]
[2,4]
[2,5]
[3,4]
[3,5]
[4,5]
Noun → book | flight | meal | money
Verb → book | include | prefer
S → book | include | prefer
Nominal → book | flight | meal | money
NP → Det Nominal

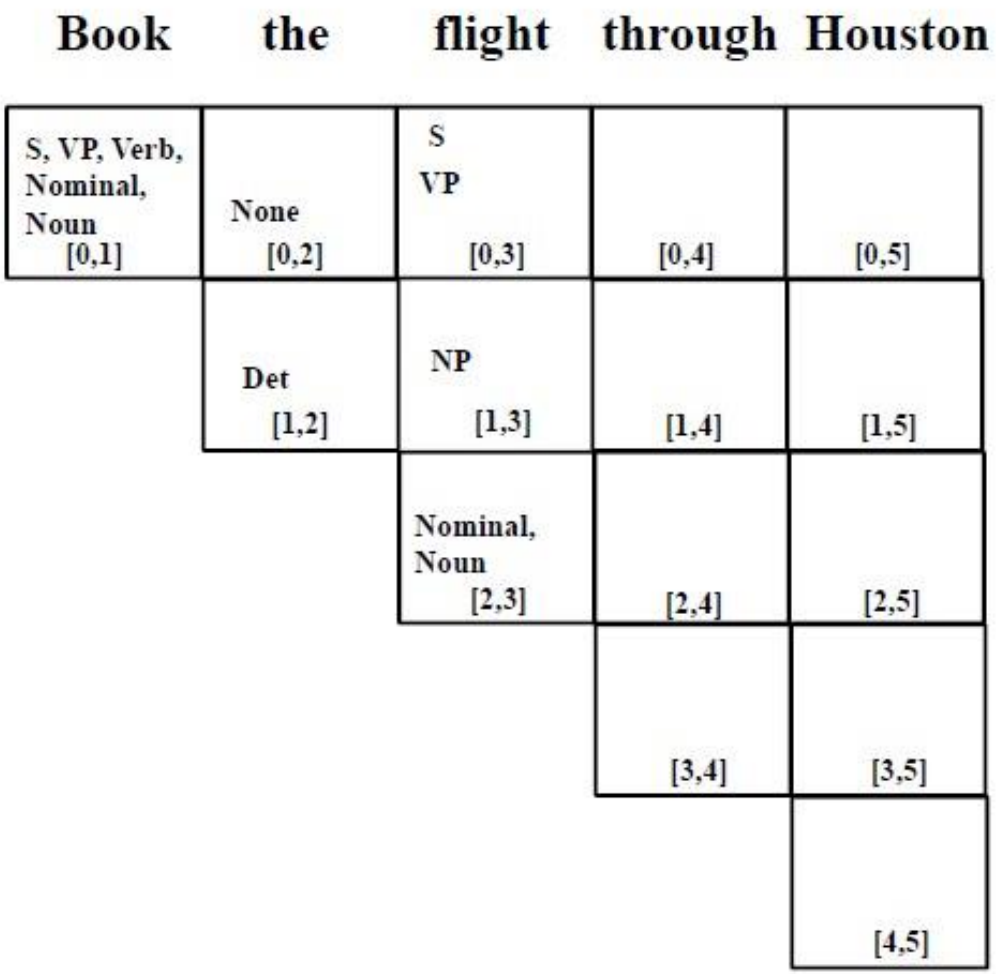
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
VP
[0,3]
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
[1,4]
[1,5]
Nominal, Noun
[2,3]
[2,4]
[2,5]
[3,4]
[3,5]
[4,5]
VP → Verb NP

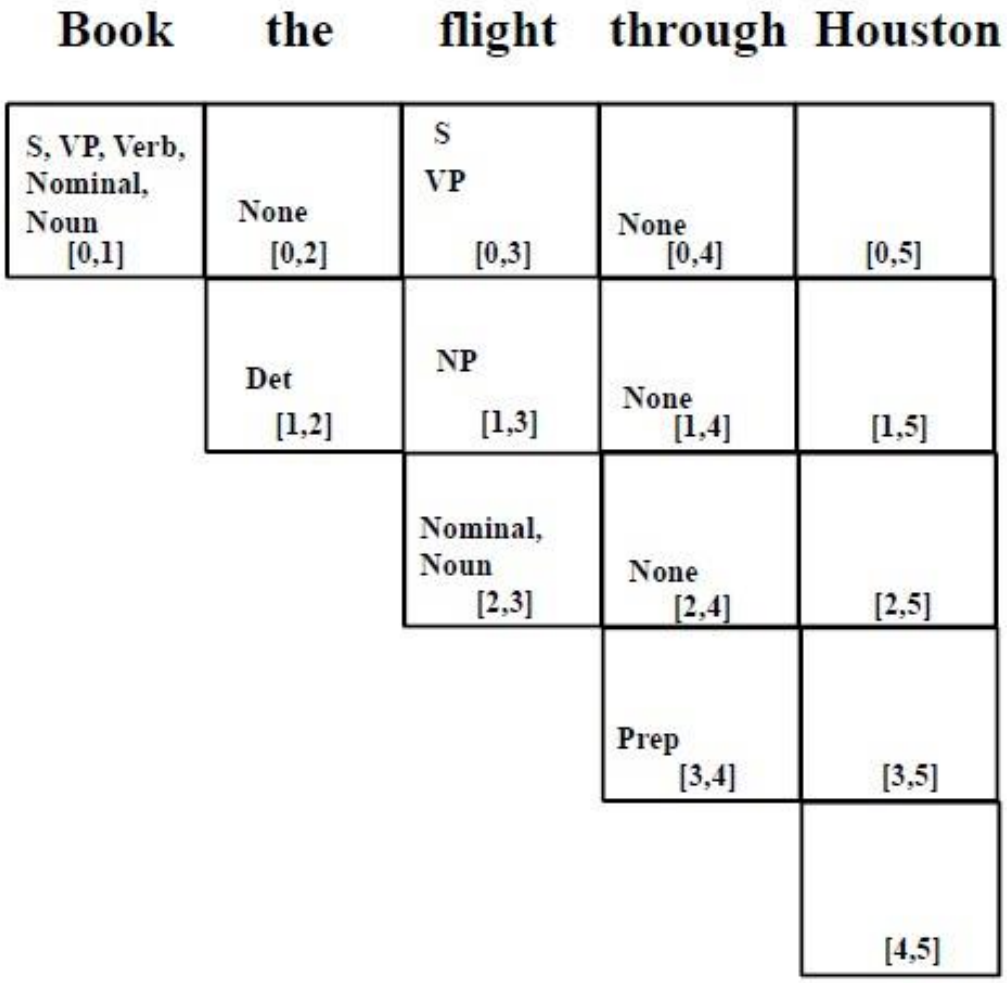
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
[1,4]
[1,5]
Nominal, Noun
[2,3]
[2,4]
[2,5]
[3,4]
[3,5]
[4,5]
$S \rightarrow Verb\ NP$

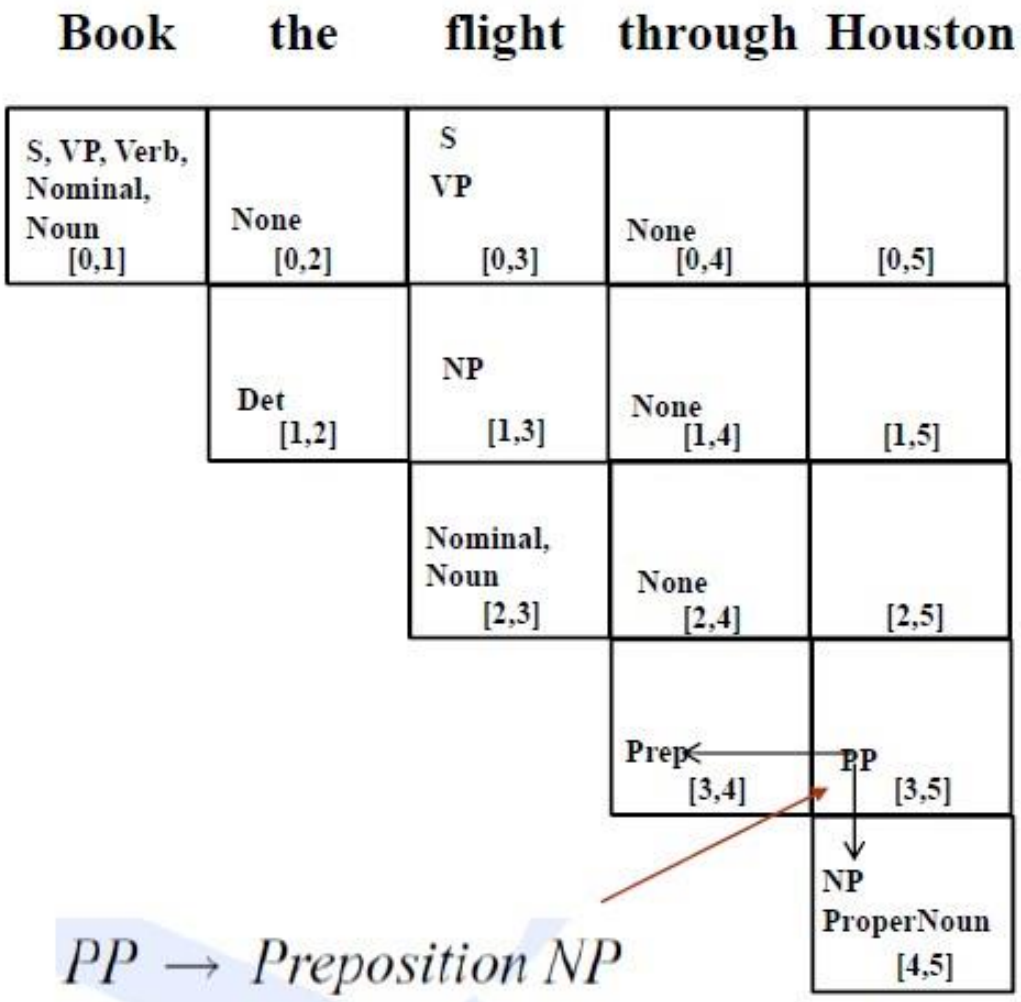
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
[1,4]
[1,5]
Nominal, Noun
[2,3]
[2,4]
[2,5]
[3,4]
[3,5]
[4,5]

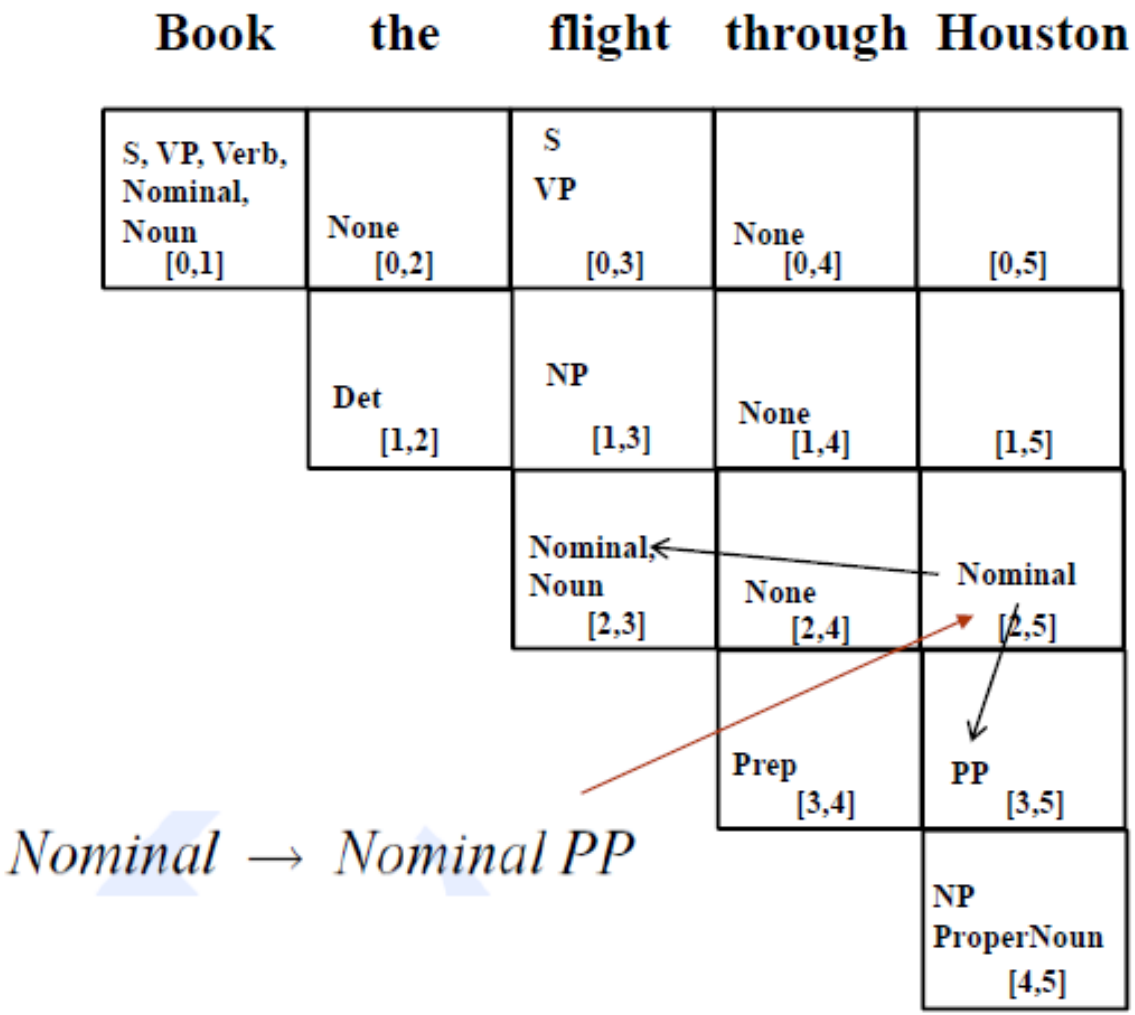
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
[2,5]
Prep
[3,4]
[3,5]
[4,5]

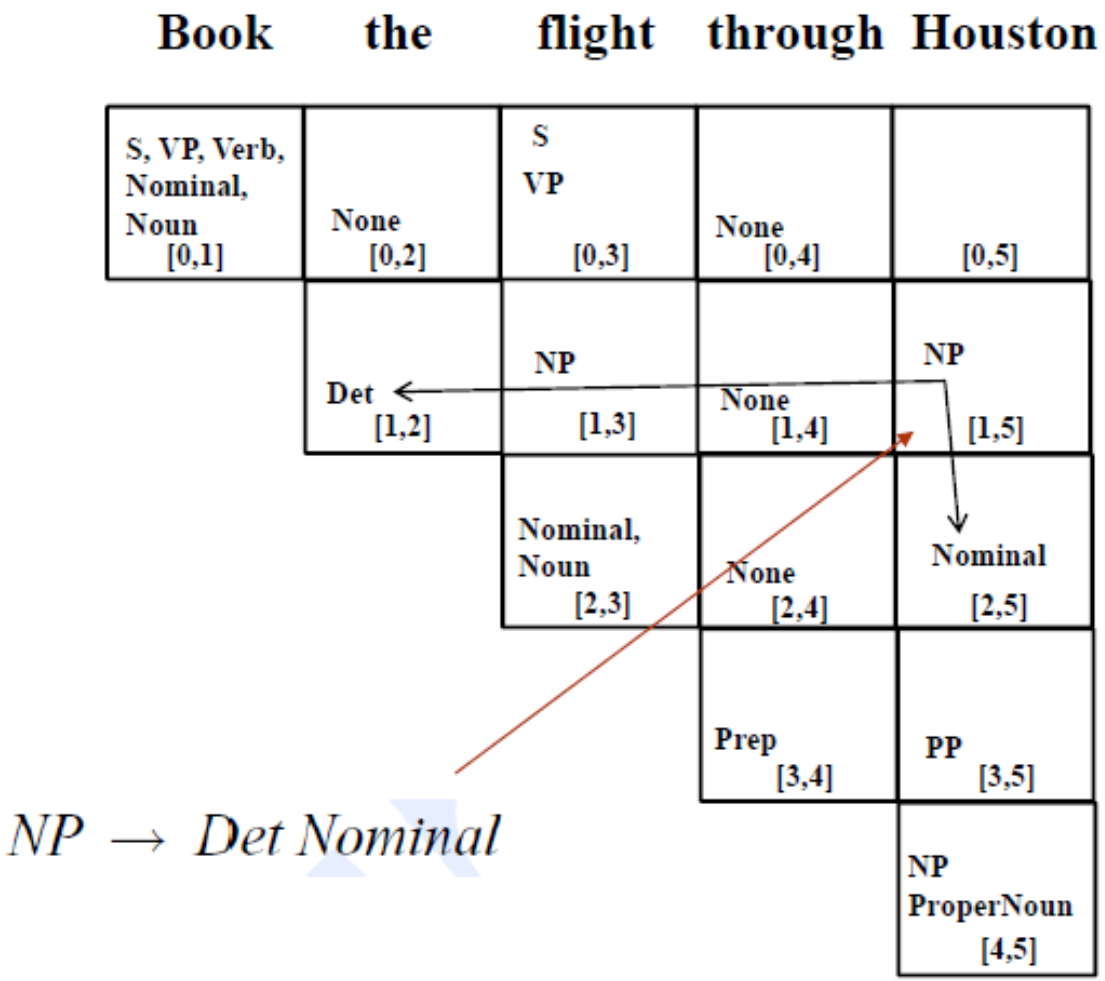
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]
PP → Preposition NP

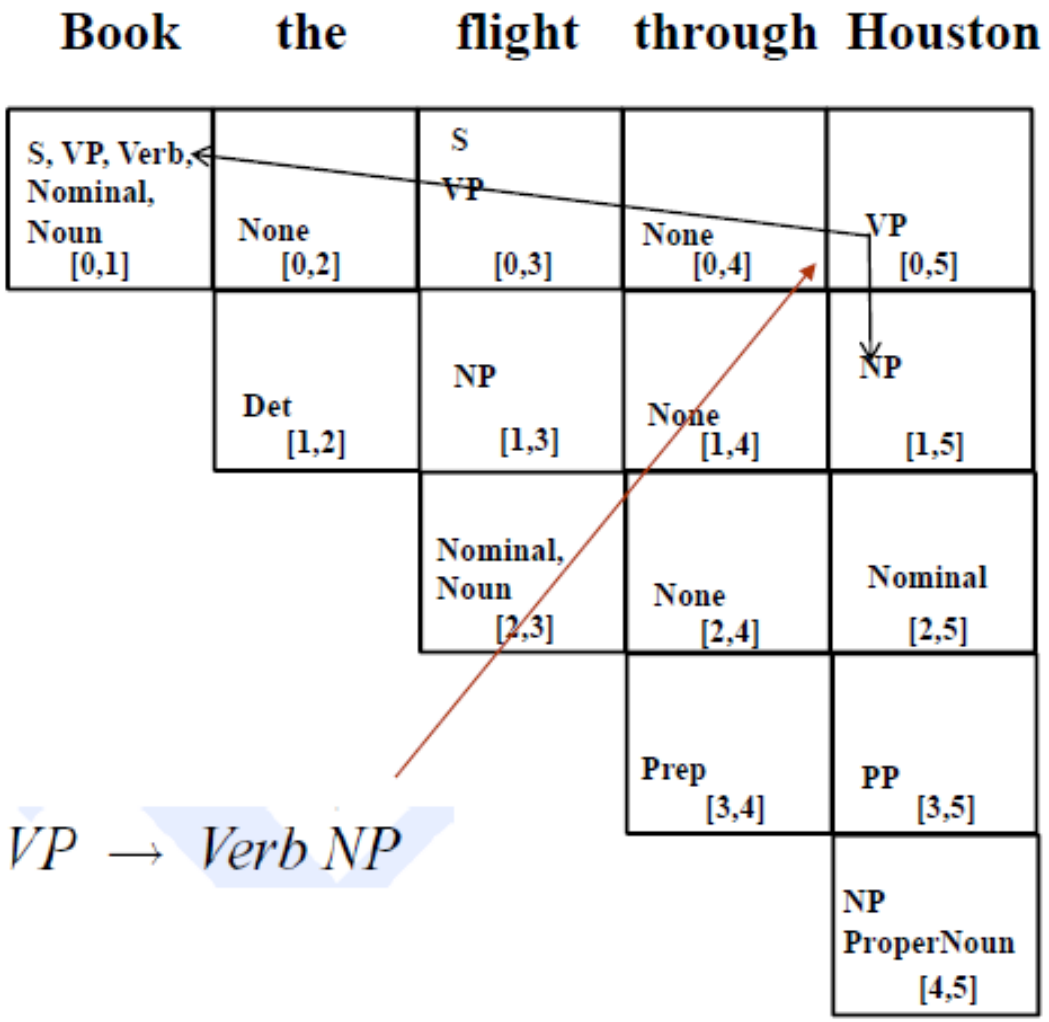
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
Nominal
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]
Nominal → Nominal PP

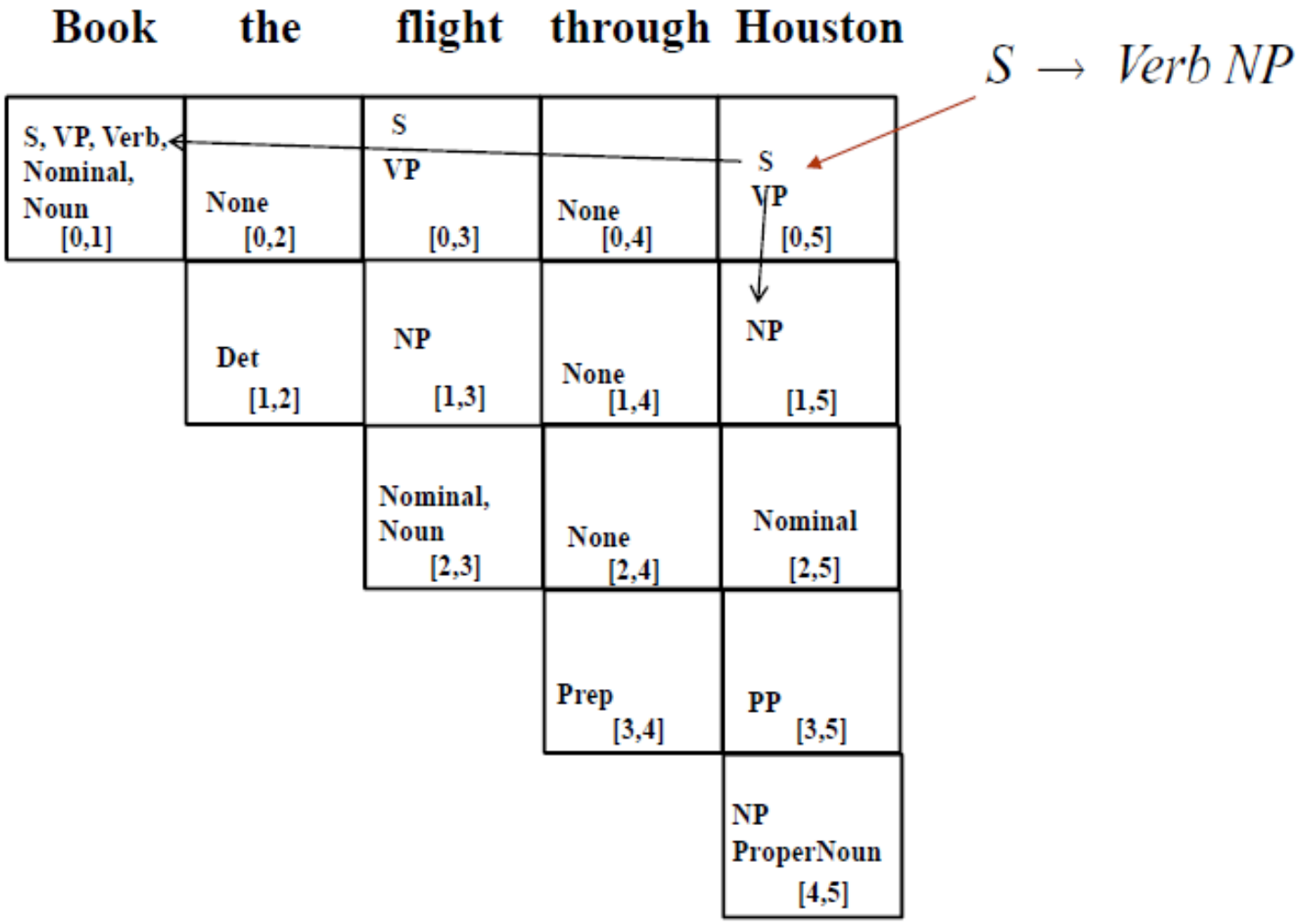
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
NP
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
Nominal
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]
NP → Det Nominal

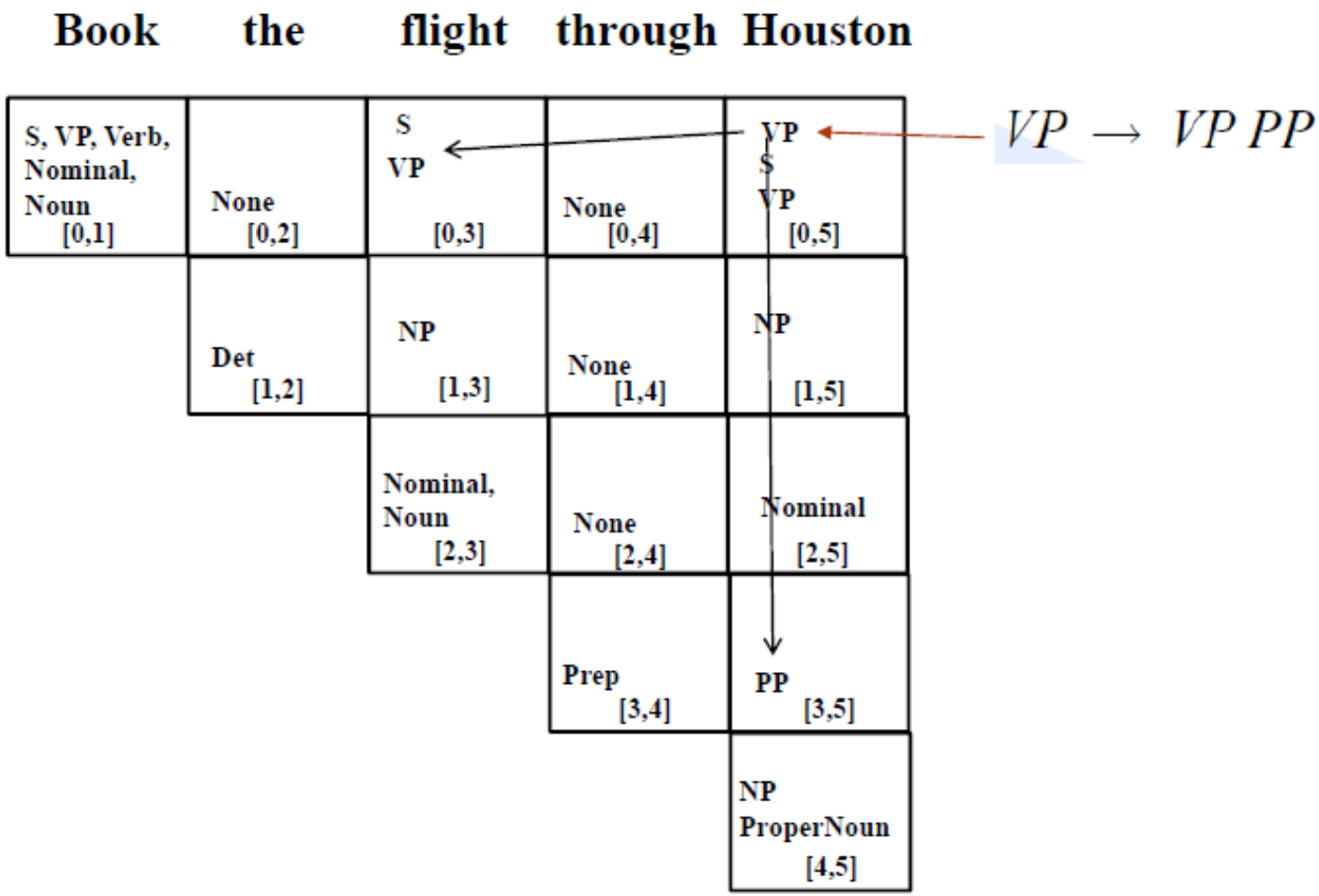
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
VP
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
NP
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
Nominal
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]
VP → Verb NP

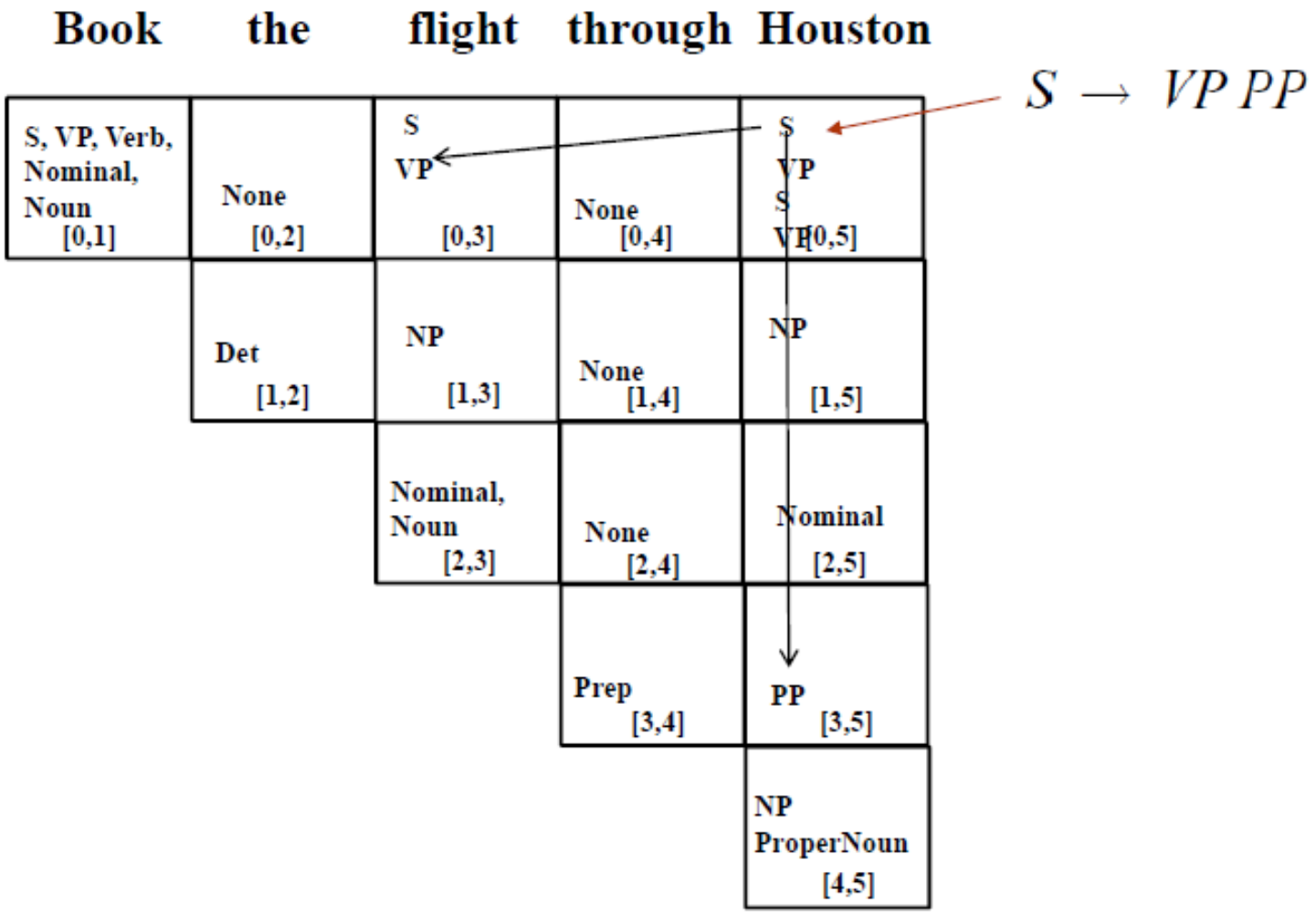
CYK Parsing


Book
the
flight
through
Houston
$S \rightarrow Verb\ NP$
S, VP, Verb, Nominal, Noun [0,1]
None [0,2]
S VP [0,3]
None [0,4]
S VP [0,5]
Det [1,2]
NP [1,3]
None [1,4]
NP [1,5]
Nominal, Noun [2,3]
None [2,4]
Nominal [2,5]
Prep [3,4]
PP [3,5]
NP ProperNoun [4,5]

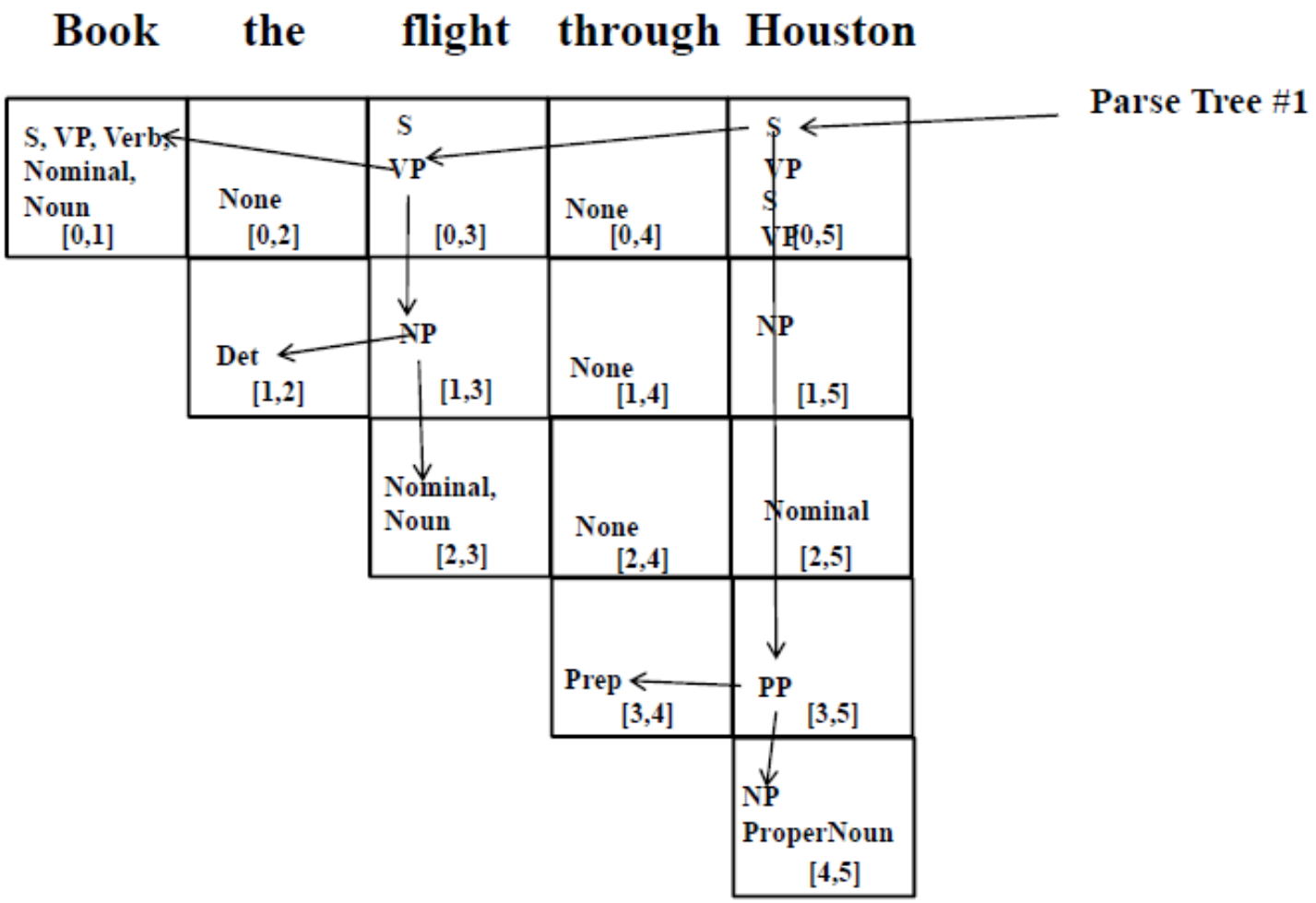
CYK Parsing


Book
the
flight
through
Houston
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
VP
S
VP
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
NP
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
Nominal
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]
VP → VP PP

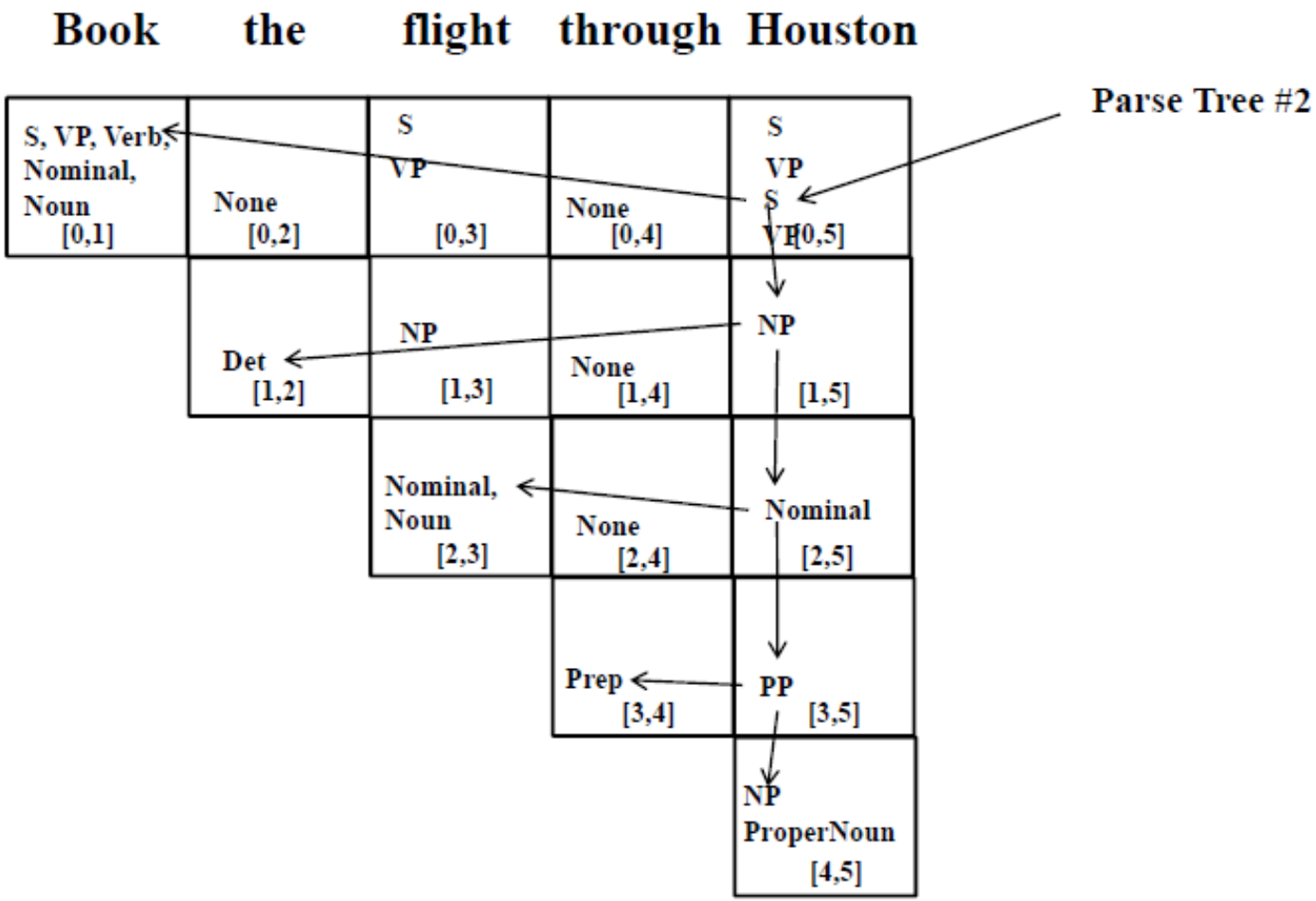
CYK Parsing


Book the flight through Houston
S, VP, Verb, Nominal, Noun [0,1]
None [0,2]
S VP [0,3]
None [0,4]
S VP S VP [0,5]
Det [1,2]
NP [1,3]
None [1,4]
NP [1,5]
Nominal, Noun [2,3]
None [2,4]
Nominal [2,5]
Prep [3,4]
PP [3,5]
NP ProperNoun [4,5]
$S \rightarrow VP\ PP$

CYK Parsing

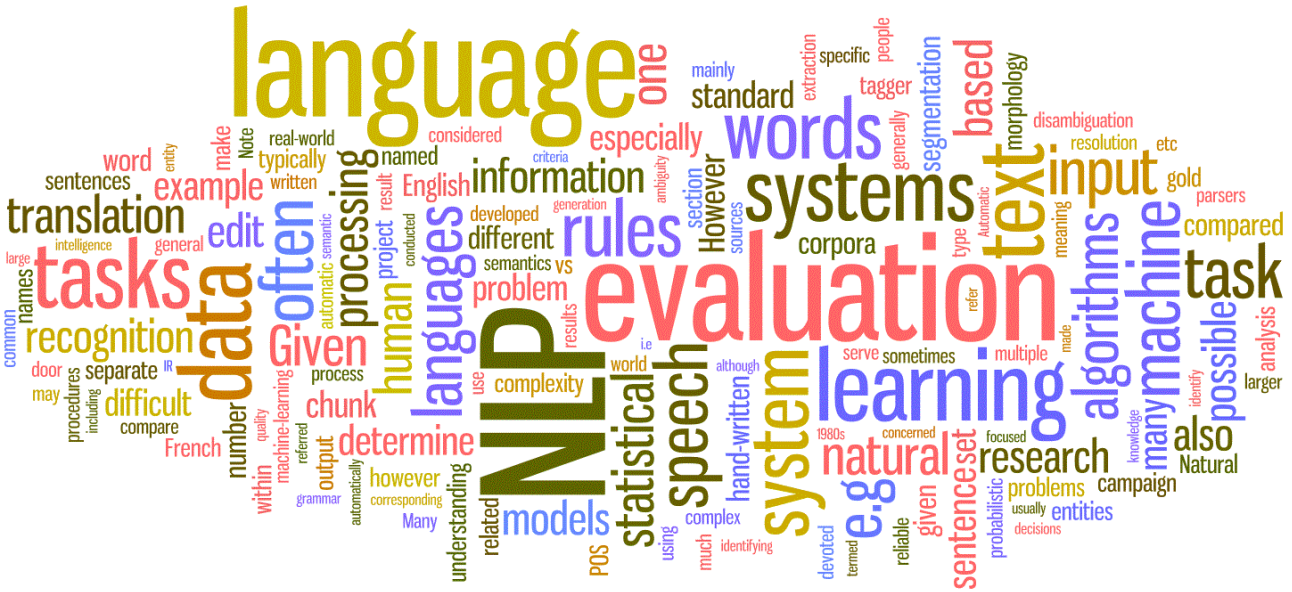

Book the flight through Houston
Parse Tree #1
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
S
VP
S
VP
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
NP
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
Nominal
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]

CYK Parsing


Book the flight through Houston
Parse Tree #2
S, VP, Verb, Nominal, Noun
[0,1]
None
[0,2]
S
VP
[0,3]
None
[0,4]
S
VP
S
VP
[0,5]
Det
[1,2]
NP
[1,3]
None
[1,4]
NP
[1,5]
Nominal, Noun
[2,3]
None
[2,4]
Nominal
[2,5]
Prep
[3,4]
PP
[3,5]
NP
ProperNoun
[4,5]

CYK Parsing

- Generate all possible parsing tree.
- Complexity: $O(n^3)$
- Dynamic Parsing.
- No disambiguation capability.



Lecture 13: Parsing II

Xu Ruifeng

Harbin Institute of Technology, Shenzhen

Today's Class

- Probabilistic Context-Free Grammar (PCFG)
- Lexicalized PCFG
- Dependency Syntax
- Dependency Parsing
- Neural-based Dependency Parsing

Statistical Parsing

- Statistical parsing uses a probabilistic model of syntax in order to assign probabilities to each parse tree.
- Provides a principled approach to resolving syntactic ambiguity.
- Allows supervised learning of parsers from tree-banks of parse trees provided by human linguists.
- Also allows unsupervised learning of parsers from raw text, but the accuracy of such parsers has been limited.

Probabilistic Context Free Grammar (PCFG)

- A PCFG is a probabilistic version of a CFG where each production has a probability.

- Probabilities of all productions rewriting a given non-terminal must add to 1, defining a distribution for each non-terminal.

$$\sum_{a} P(A -> \alpha) = 1$$

- String generation is now probabilistic where production probabilities are used to non-deterministically select a production for rewriting a given non-terminal.

PCFG

- Assumption 1: Place invariance.
- Assumption 2: Context-free.
- Assumption 3: Ancestor-free.

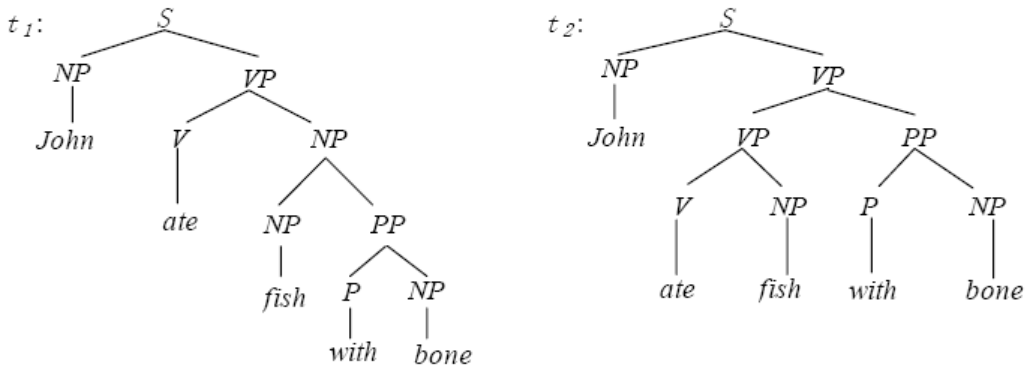
- The parse tree probability is the production of the probability of all applied parsing rules.

Simple PCFG Example

- Assume productions for each node are chosen independently.
- Probability of derivation is the product of the probabilities of its productions.
- Resolve ambiguity by picking most probable parse tree.

Simple PCFG Example

sentence = *"John ate fish with bone"*



$S \rightarrow NP\ VP$	1.0	$NP \rightarrow NP\ PP$	0.4
$PP \rightarrow P\ NP$	1.0	$NP \rightarrow John$	0.1
$VP \rightarrow V\ NP$	0.7	$NP \rightarrow bone$	0.18
$VP \rightarrow VP\ PP$	0.3	$NP \rightarrow star$	0.04
$P \rightarrow with$	1.0	$NP \rightarrow fish$	0.18
$V \rightarrow ate$	1.0	$NP \rightarrow telescope$	0.1

$$P(t_1) = 1.0 \times 0.1 \times 0.7 \times 1.0 \times 0.4 \times 0.18 \times 1.0 \times 1.0 \times 0.18 = 0.0009072$$

$$P(t_2) = 1.0 \times 0.1 \times 0.3 \times 0.7 \times 1.0 \times 0.18 \times 1.0 \times 1.0 \times 0.18 = 0.0006804$$

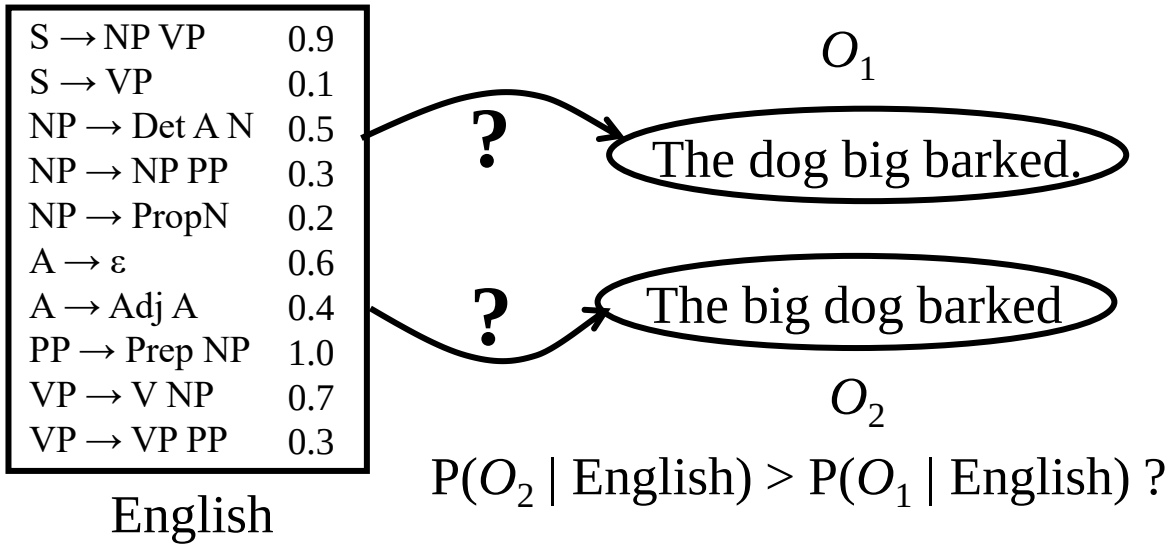
$$P(sentence) = P(t_1) + P(t_2) = 0.0015876$$

Three PCFG Tasks

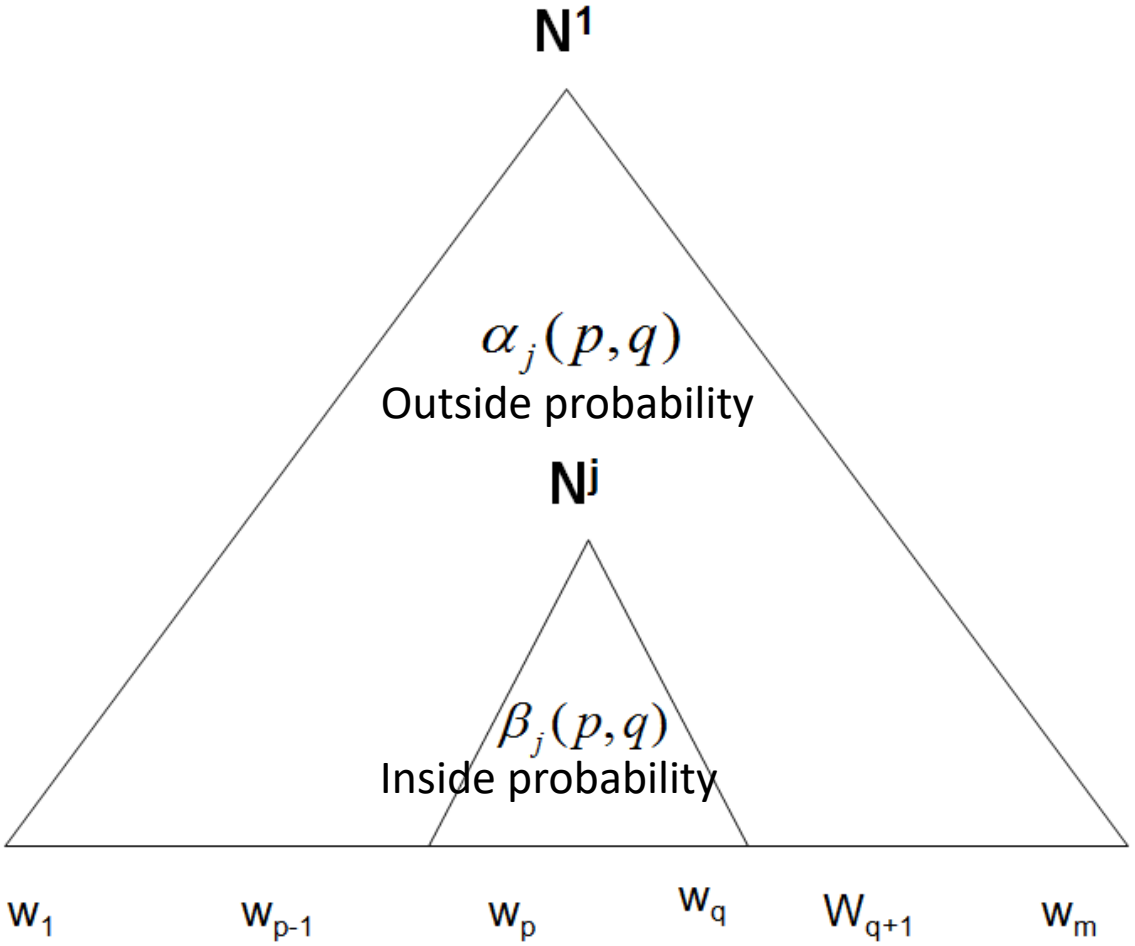
- Observation likelihood: To classify and order sentences. P(W|G) for sentence W=$w_1 w_2 \ldots w_n$
  - Inside Algorithm
- Most likely derivation: To determine the most likely parse tree for a sentence.
  - Viterbi Algorithm
- Maximum likelihood training: To train a PCFG to fit empirical training data, i.e. maximize P(W|G)

PCFG: Observation Likelihood

- There is an analog to Forward algorithm for HMMs called the Inside algorithm for efficiently determining how likely a string is to be produced by a PCFG.



Notation


N1
$\alpha_j(p,q)$
Outside probability
Nj
$\beta_j(p,q)$
Inside probability
w1
wp-1
wp
wq
Wq+1
wm

Inside and Outside Probability

- Inside probability: total probability of generating words $w_p \ldots w_q$ from non-terminal N^j.

$$\beta_j(p,q) = P(w_{pq} \mid N^j_{pq})$$

- Outside probability: total probability of beginning with the start symbol N^1 and generating N^j_{pq} and all the words outside $w_p \ldots w_q$

$$\alpha_j(p,q) = P(w_{1(p-1)}, N^j_{pq}, w_{(q+1)m})$$

- When p>q,

$$\alpha_j(p,q) = \beta_j(p,q) = 0$$

Probabilistic CKY Algorithm

- Revised CKY with probabilistic information.
- Set and maintain production probability during transfer grammar rule to CNF.
- Each cell provide a probability for each non-terminal.
- Cell[i,j]: the most likely parsing contains all of Non-terminals which covers $Word_{i+1}$ to $Word_j$ and its probability.

Probabilistic CKY Algorithm

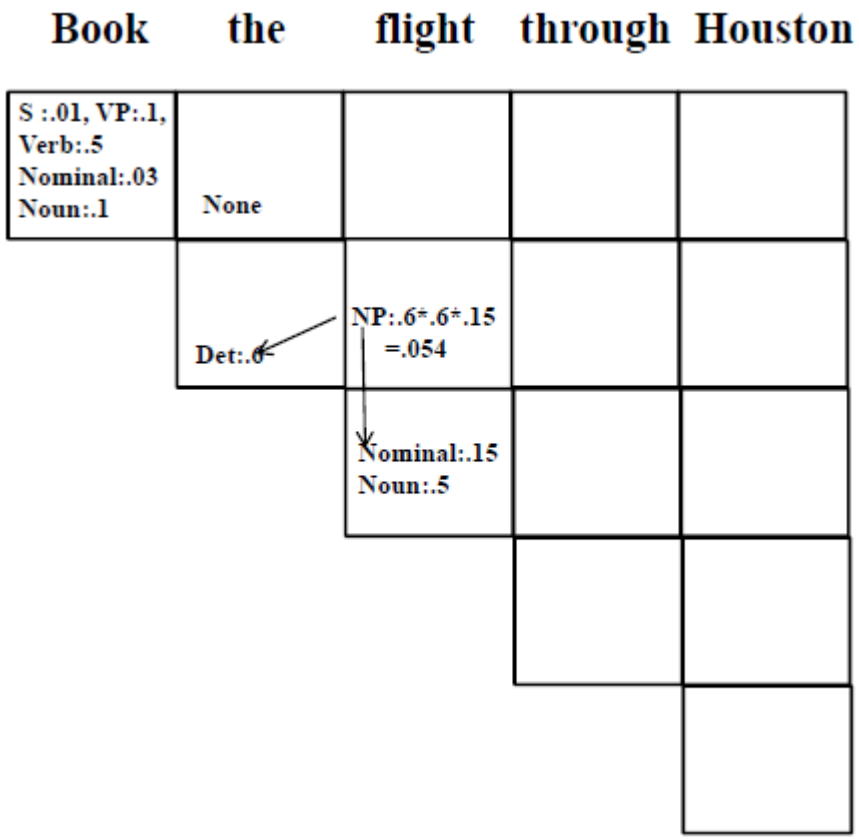
Original Grammar

S → NP VP	0.8
S → Aux NP VP	0.1
S → VP	0.1
NP → Pronoun	0.2
NP → Proper-Noun	0.2
NP → Det Nominal	0.6
Nominal → Noun	0.3
Nominal → Nominal Noun	0.2
Nominal → Nominal PP	0.5
VP → Verb	0.2
VP → Verb NP	0.5
VP → VP PP	0.3
PP → Prep NP	1.0

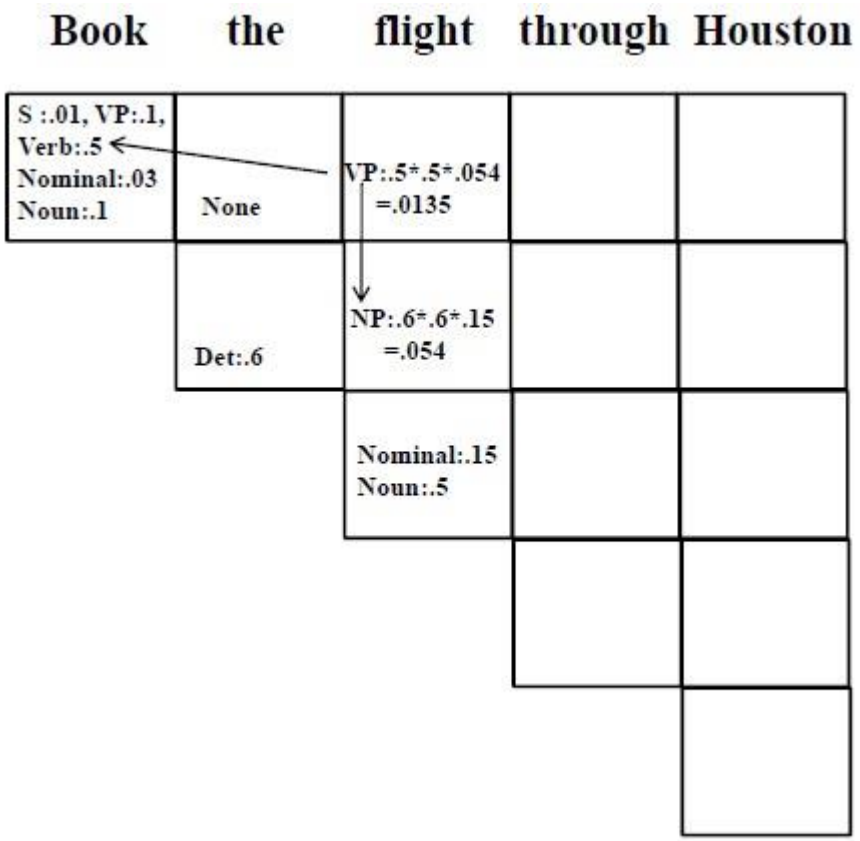
Chomsky Normal Form

S → NP VP	0.8
S → X1 VP	0.1
X1 → Aux NP	1.0
S → book | include | prefer 0.01 0.004 0.006	
S → Verb NP	0.05
S → VP PP	0.03
NP → I | he | she | me 0.1 0.02 0.02 0.06	
NP → Houston | NWA 0.16 .04	
NP → Det Nominal	0.6
Nominal → book | flight | meal | money 0.03 0.15 0.06 0.06	
Nominal → Nominal Noun	0.2
Nominal → Nominal PP	0.5
VP → book | include | prefer 0.1 0.04 0.06	
VP → Verb NP	0.5
VP → VP PP	0.3
PP → Prep NP	1.0

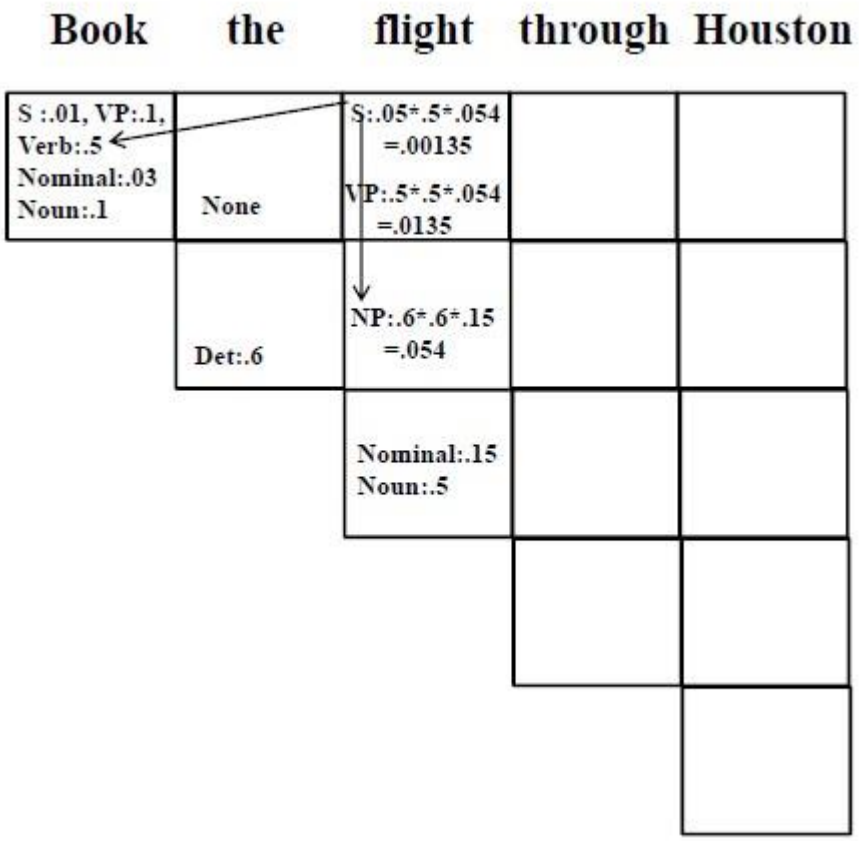
Probabilistic CKY Algorithm



Probabilistic CKY Algorithm



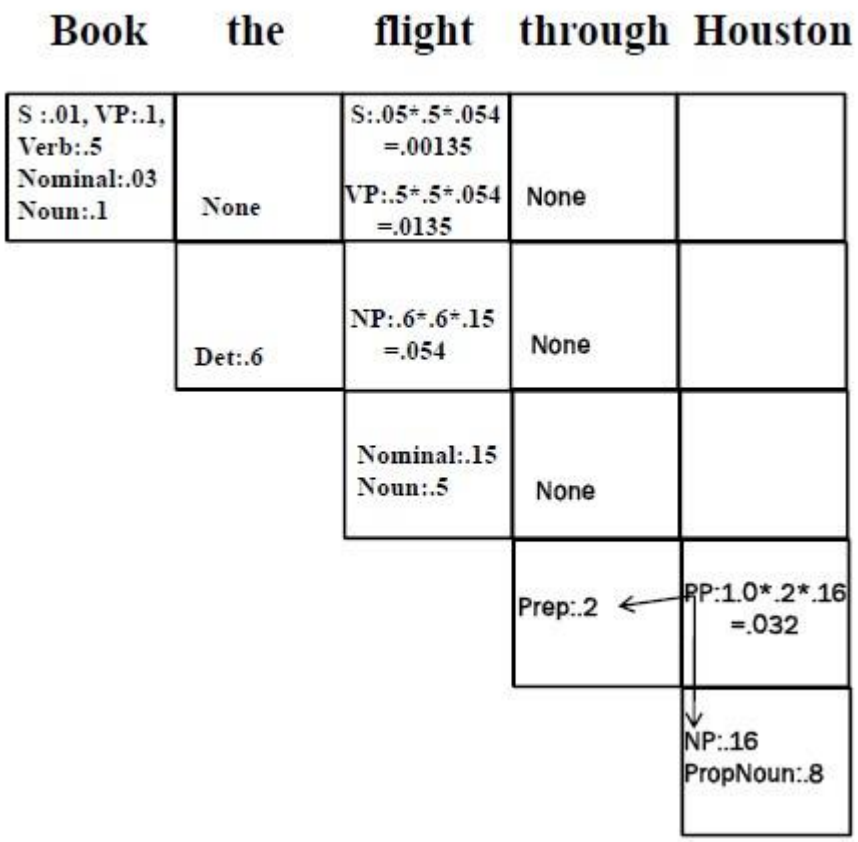
Probabilistic CKY Algorithm



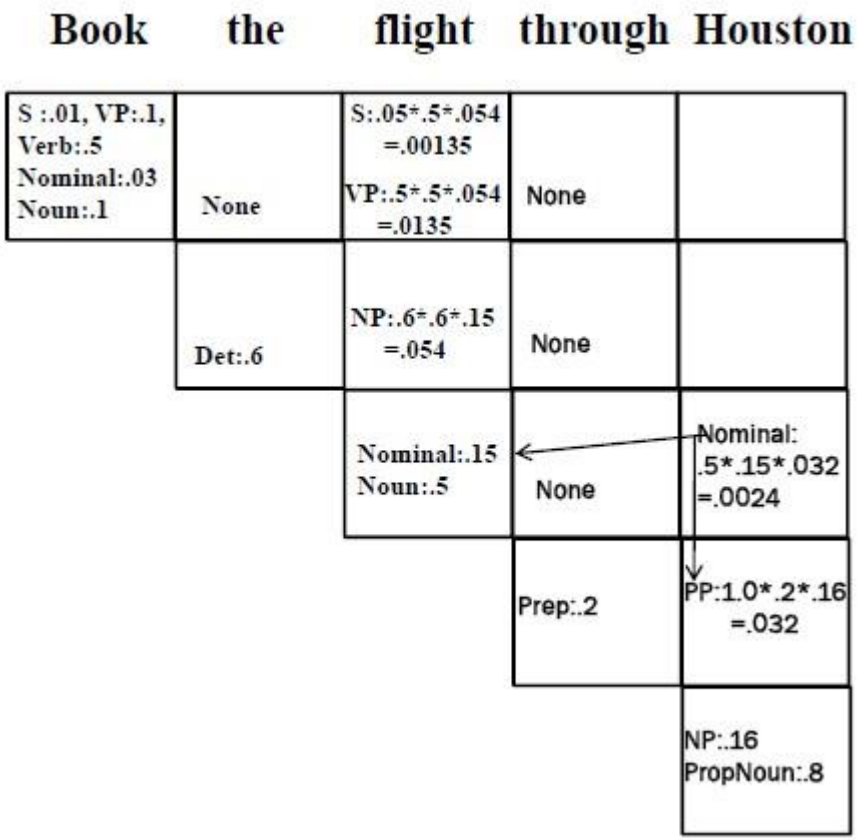
Probabilistic CKY Algorithm

Book	the	flight	through	Houston
S :.01, VP:.1, Verb:.5 Nominal:.03 Noun:.1	None	S:.05*.5*.054 =.00135 VP:.5*.5*.054 =.0135	None	
	Det:.6	NP:.6*.6*.15 =.054	None	
		Nominal:.15 Noun:.5	None	
			Prep:.2	

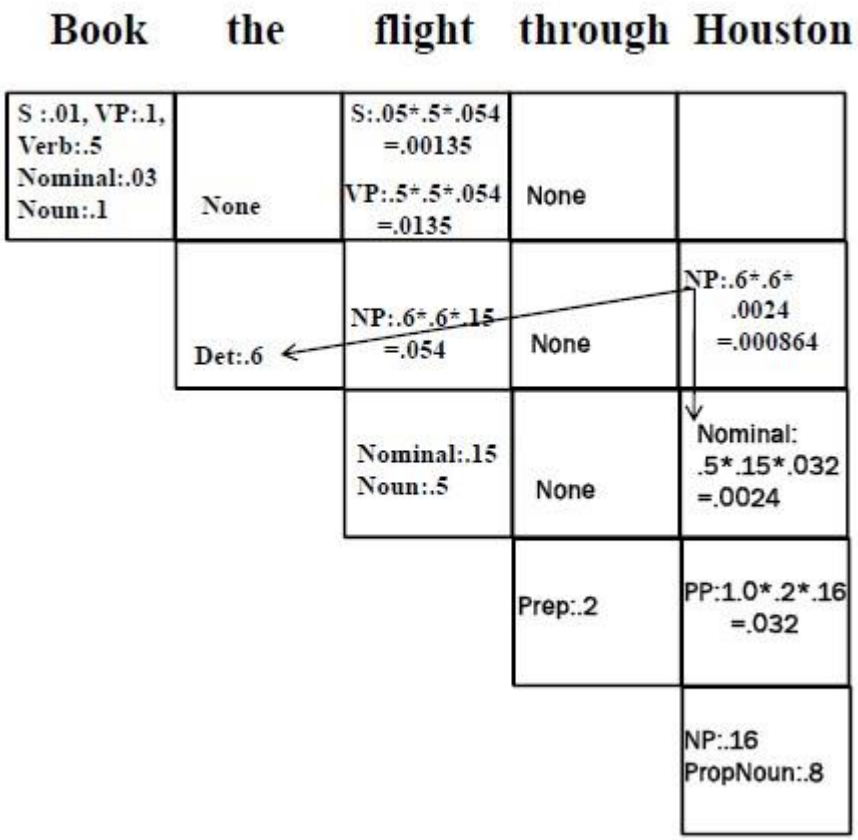
Probabilistic CKY Algorithm



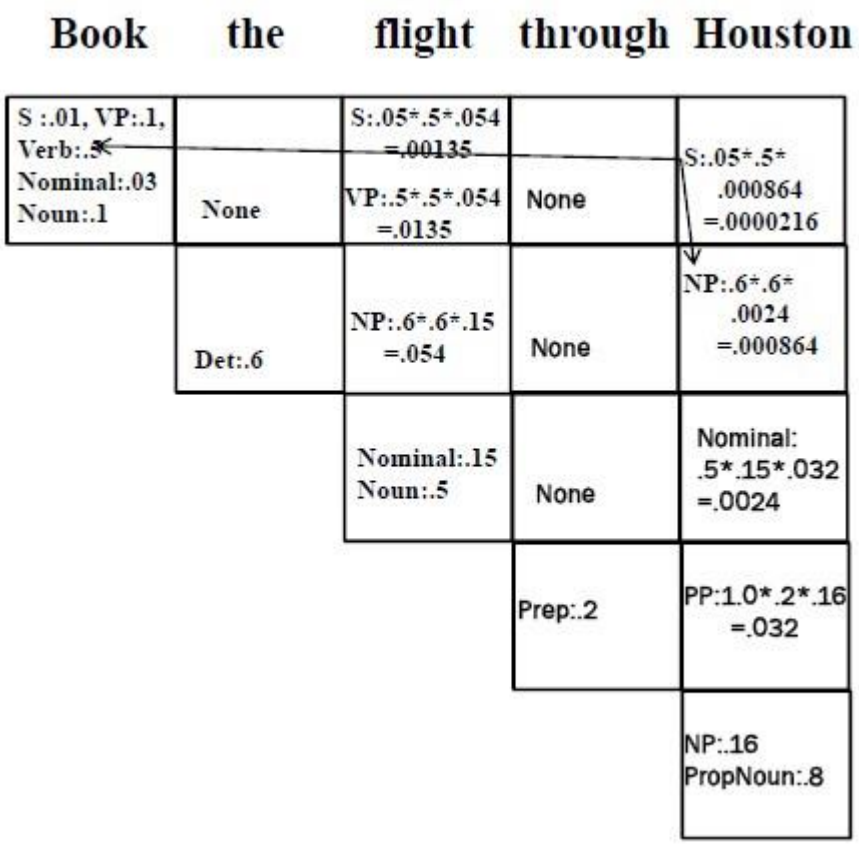
Probabilistic CKY Algorithm



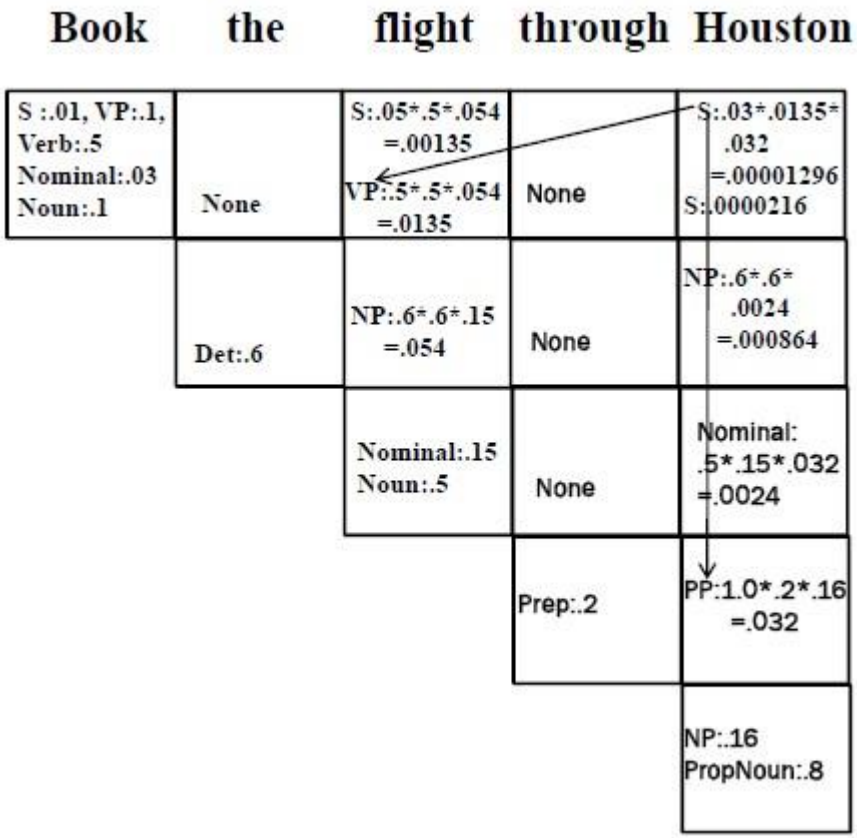
Probabilistic CKY Algorithm



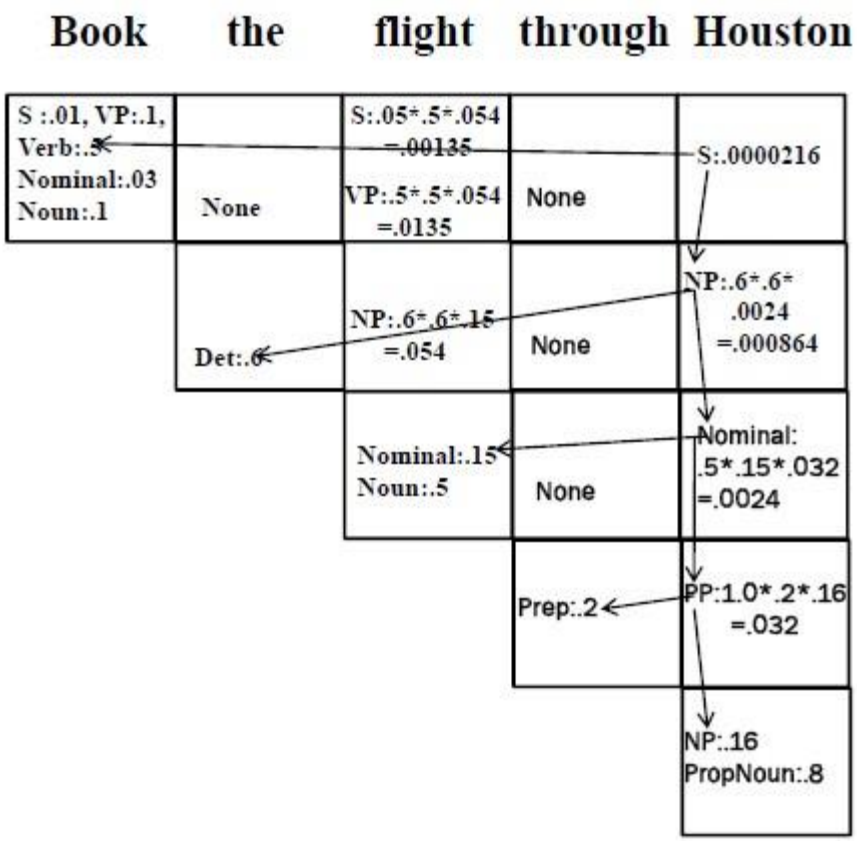
Probabilistic CKY Algorithm



Probabilistic CKY Algorithm

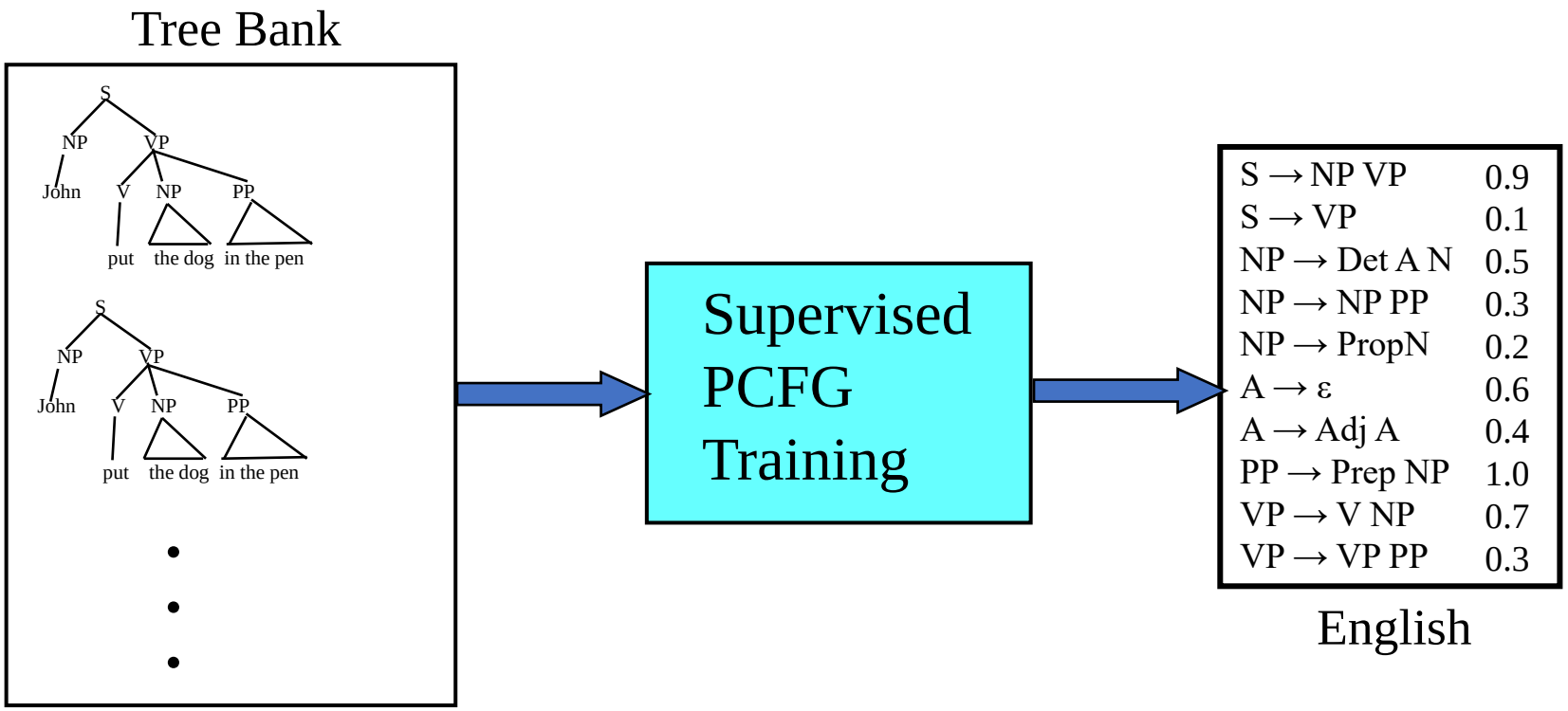


Probabilistic CKY Algorithm



PCFG: Supervised Training

- If parse trees are provided for training sentences, a grammar and its parameters can be estimated directly from counts accumulated from the tree-bank (with appropriate smoothing).



Estimating Production Probabilities

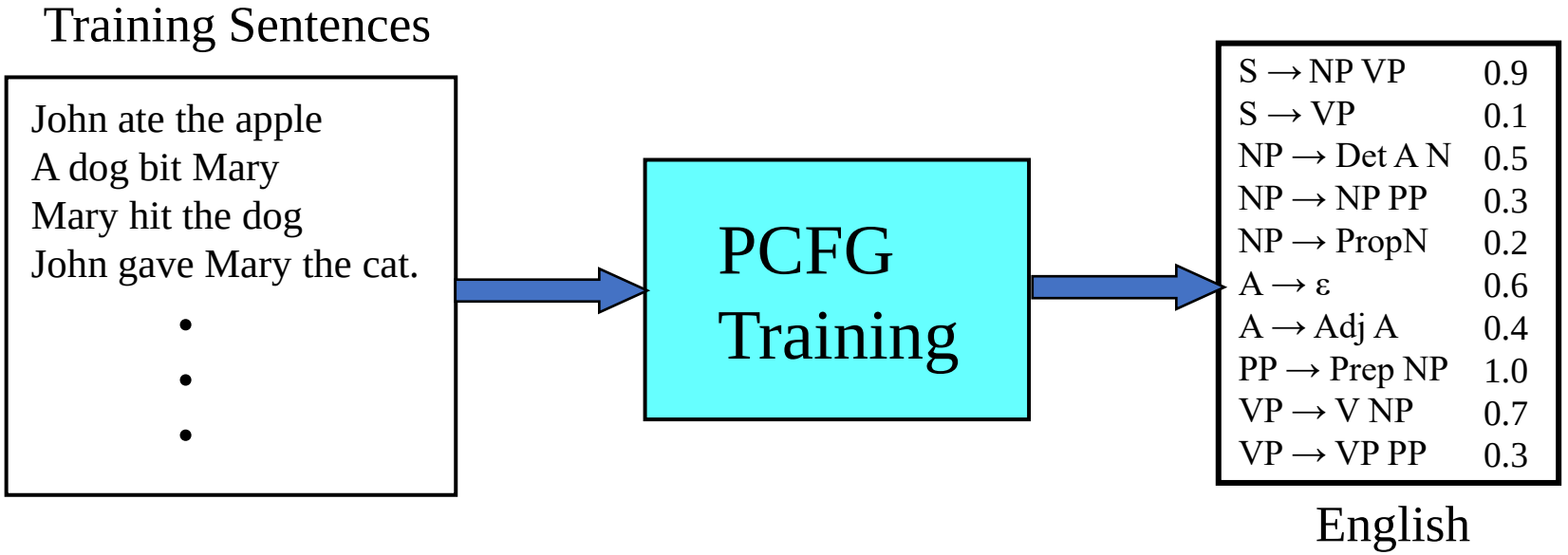
- A set of production rules can be taken directly from the set of rewrites in the treebank.

- Parameters can be directly estimated from frequency counts in the treebank.

$$P(\alpha \rightarrow \beta \mid \alpha) = \frac{\text{count}(\alpha \rightarrow \beta)}{\sum_{\gamma} \text{count}(\alpha \rightarrow \gamma)} = \frac{\text{count}(\alpha \rightarrow \beta)}{\text{count}(\alpha)}$$

PCFG: Maximum Likelihood Training

- Given a set of sentences, induce a grammar that maximizes the probability that this data was generated from this grammar.
- Assume the number of non-terminals in the grammar is specified.
- Only need to have an unannotated set of sequences generated from the model. Does not need correct parse trees for these sentences. In this sense, it is unsupervised.



Inside-Outside Algorithm

- The Inside-Outside algorithm is a version of EM for unsupervised learning of a PCFG.
  - Analogous to Baum-Welch (forward-backward) for HMMs.
- Given the number of non-terminals, construct all possible CNF productions with these non-terminals and observed terminal symbols.
- Use EM to iteratively train the probabilities of these productions to locally maximize the likelihood of the data.
- Experimental results are not impressive, but recent work imposes additional constraints to improve unsupervised grammar learning.

Summary of PCFG

- Pros
  - Disambiguation of structurally different parses.
  - Speed syntactic analysis by eliminating low probability structures.
  - Improve the robustness of analyzer by use of low probabilities.
  - Helpful to grammar induction.

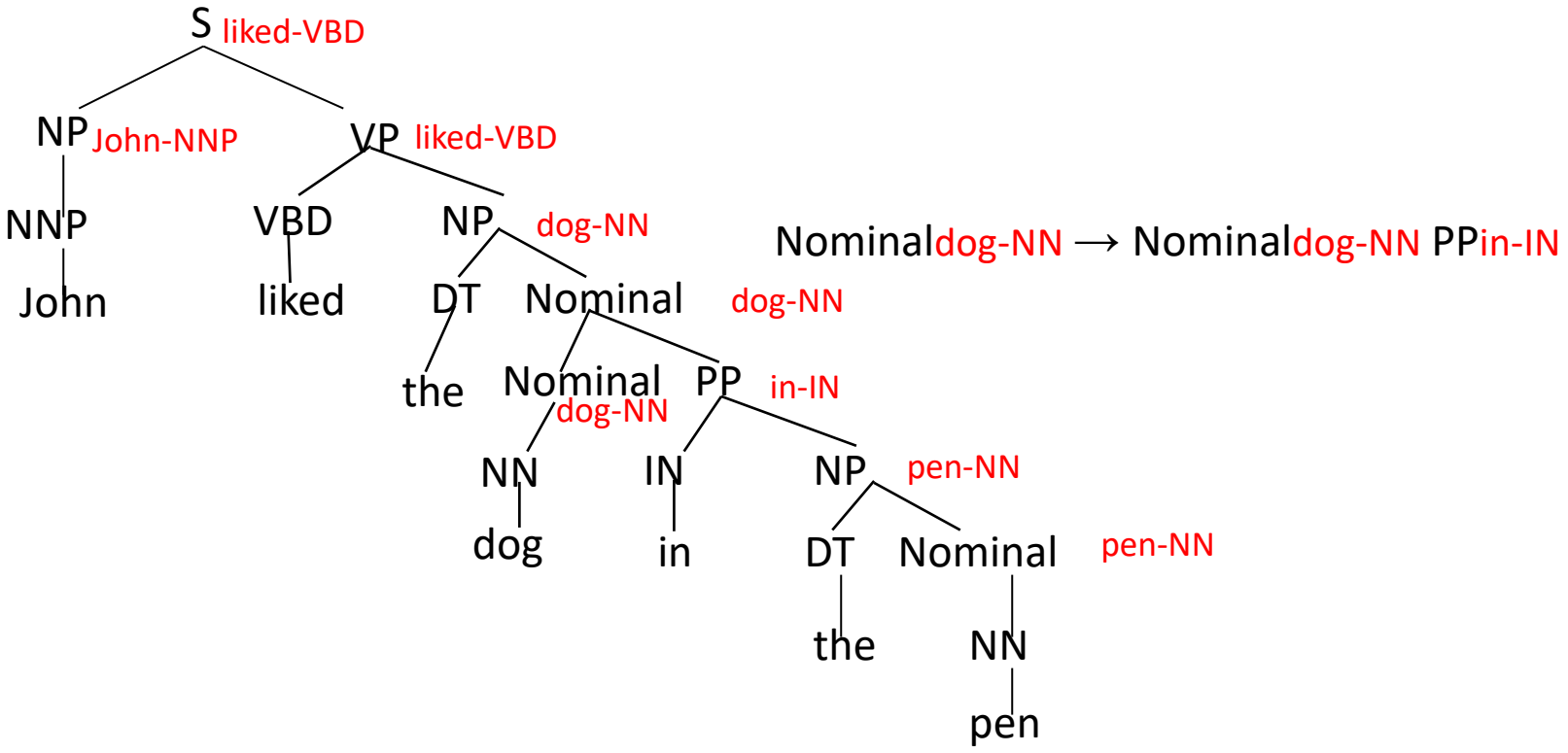
- Cons
  - Since probabilities of productions do not rely on specific words or concepts, only general structural disambiguation is possible.
  - cannot resolve syntactic ambiguities that require semantics to resolve, e.g. ate with fork vs. meatballs.

Lexicalized PCFG

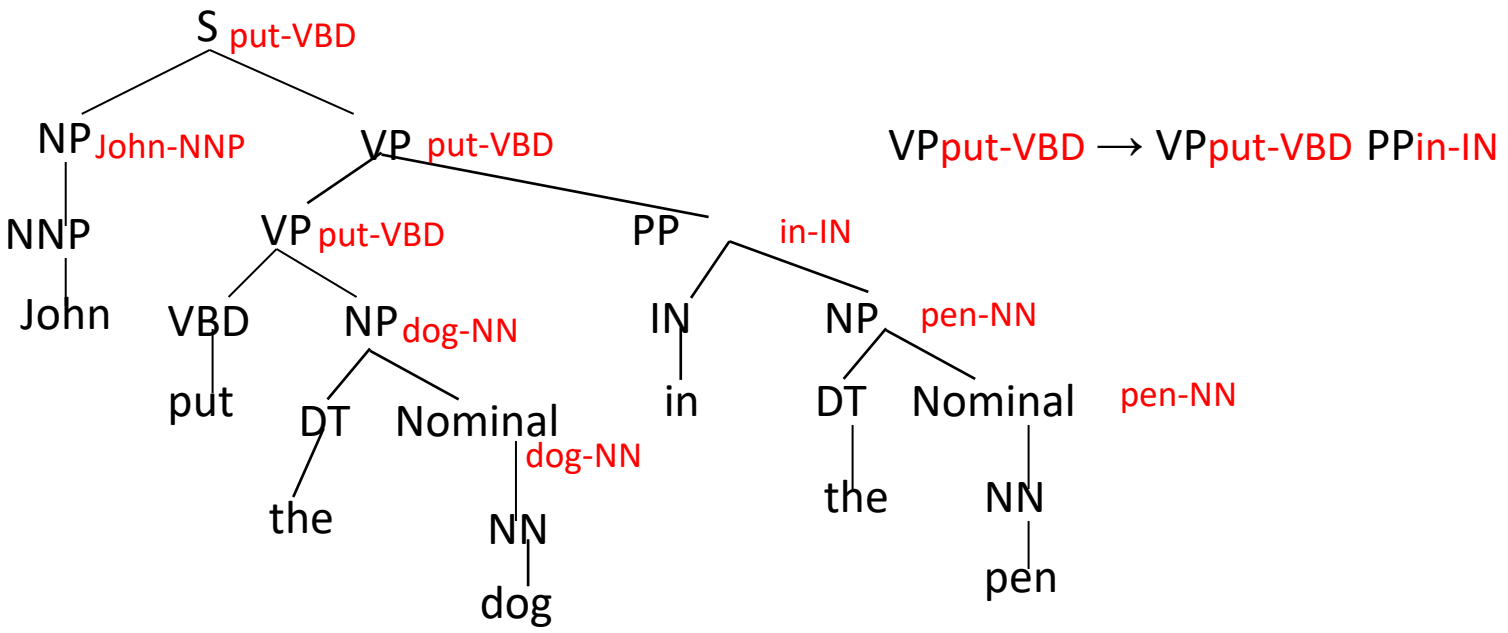
- PCFGs must be lexicalized, i.e. productions must be specialized to specific words by including their head-word in their LHS non-terminals.
- Each non-terminal is associated to headword *w.*
- Syntactic phrases usually have a word in them that is most "central" to the phrase.
- Process Rule
  - Head of a VP is the main verb.
  - Head of an NP is the main noun.
  - Head of a PP is the preposition.
  - Head of a sentence is the head of its VP.

Lexicalized Productions

- Specialized productions can be generated by including the head word and its POS of each non-terminal as part of that non-terminal's symbol.



Lexicalized Productions



Parameterizing Lexicalized Productions

- Accurately estimating parameters on such a large number of very specialized productions requires enormous amounts of treebank data.

- Need some way of estimating parameters for lexicalized productions that makes reasonable independence assumptions so that accurate probabilities for very specific rules can be learned.

Collins' Parser

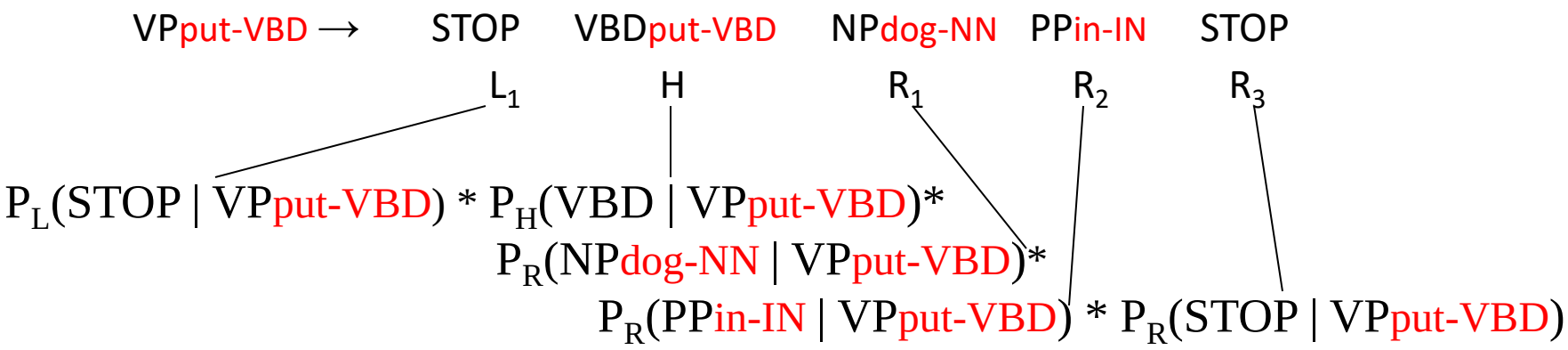
- Collins' (1999) parser assumes a simple generative model of lexicalized productions.

- Models productions based on context to the left and the right of the head daughter.
  - LHS → $L_n L_{n-1} \ldots L_1 H\ R_1 \ldots R_{m-1} R_m$

- First generate the head (H) and then repeatedly generate left (L_i) and right (R_i) context symbols until the symbol STOP is generated.

Sample Production

VPput-VBD → VBDput-VBD NPdog-NN PPin-IN



Estimating Production Generation Parameters

- Estimate P_H, P_L, and P_R parameters from treebank data.

$$P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{put-VBD}}) = \frac{\text{Count}(\text{PP}_{\text{in-IN}} \text{ right of head in a VP}_{\text{put-VBD}} \text{ production})}{\text{Count}(\text{symbol right of head in a VP}_{\text{put-VBD}})}$$

$$P_R(\text{NP}_{\text{dog-NN}} \mid \text{VP}_{\text{put-VBD}}) = \frac{\text{Count}(\text{NP}_{\text{dog-NN}} \text{ right of head in a VP}_{\text{put-VBD}} \text{ production})}{\text{Count}(\text{symbol right of head in a VP}_{\text{put-VBD}})}$$

- Smooth estimates by linearly interpolating with simpler models conditioned on just POS tag or no lexical info.

$$\begin{aligned} \text{sm}P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{put-VBD}}) = {} & \lambda_1 P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{put-VBD}}) \\ & + (1-\lambda_1)\,(\lambda_2 P_R(\text{PP}_{\text{in-IN}} \mid \text{VP}_{\text{VBD}}) + \\ & \quad (1-\lambda_2) P_R(\text{PP}_{\text{in-IN}} \mid \text{VP})) \end{aligned}$$

Parsing Evaluations Metrics

- PARSEVAL metrics measure the fraction of the constituents that match between the computed and human parse trees. If *P* is the system's parse tree and *T* is the human parse tree (the "gold standard"):
  - ***Recall*** = (# correct constituents in *P*) / (# constituents in *T*)
  - ***Precision*** = (# correct constituents in *P*) / (# constituents in *P*)

- ***Labeled Precision*** and ***labeled recall*** require getting the non-terminal label on the constituent node correct to count as correct.

- F_1 is the harmonic mean of precision and recall.

Summary of Probabilistic Parsing

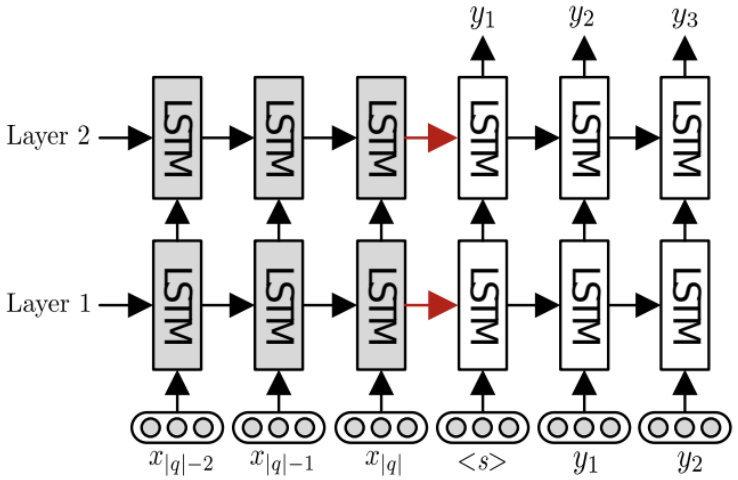
- Statistical models such as PCFGs allow for probabilistic resolution of ambiguities.
- PCFGs can be easily learned from treebanks.
- Lexicalization and non-terminal splitting are required to effectively resolve many ambiguities.
- Current statistical parsers are quite accurate but not yet at the level of human-expert agreement.

Neural-based Syntactic Summary

- Encoder
  - LSTM
  - Transformer
- Decoder
  - Transition-based
  - Chart-based
  - Sequence-to-Sequence

Neural-based Constituency Parsing

- [Li et al. 2016] present a general method based on an attention-enhanced encoder-decoder model. They encode input utterances into vector representations, and generate their logical forms by conditioning the output sequences or trees on the encoding vectors.



Dong, Li, and Mirella Lapata., ACL 2016. Language to logical form with neural attention.

Neural-based Constituency Parsing

- Model Architecture

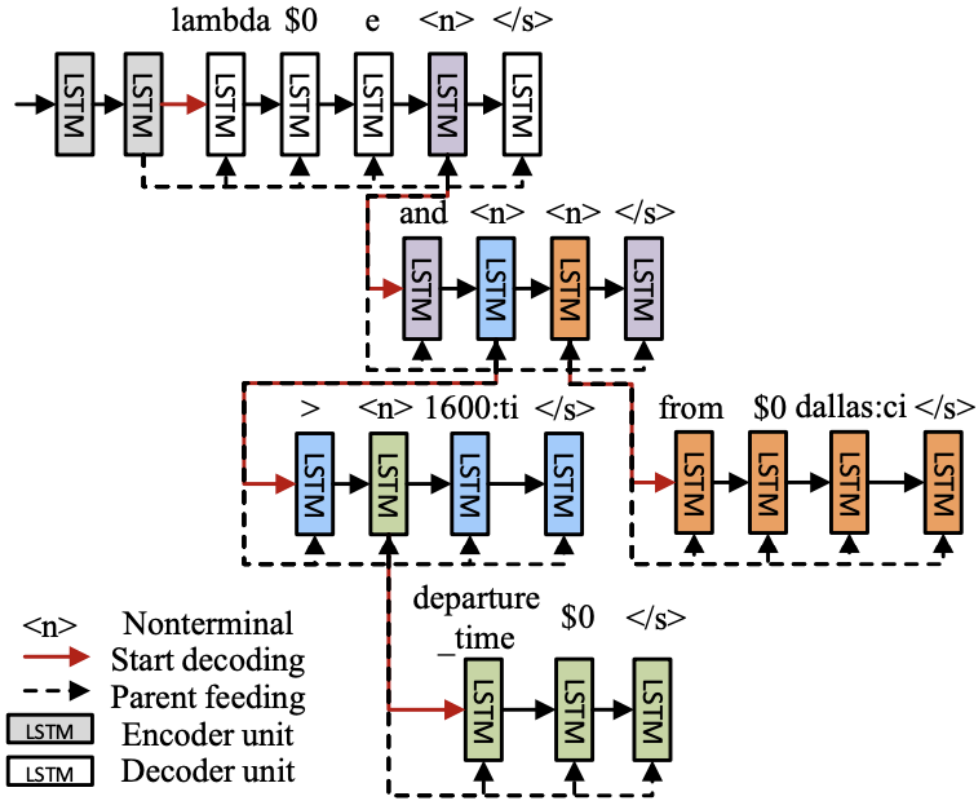


Figure 3: Sequence-to-tree (SEQ2TREE) model with a hierarchical tree decoder.

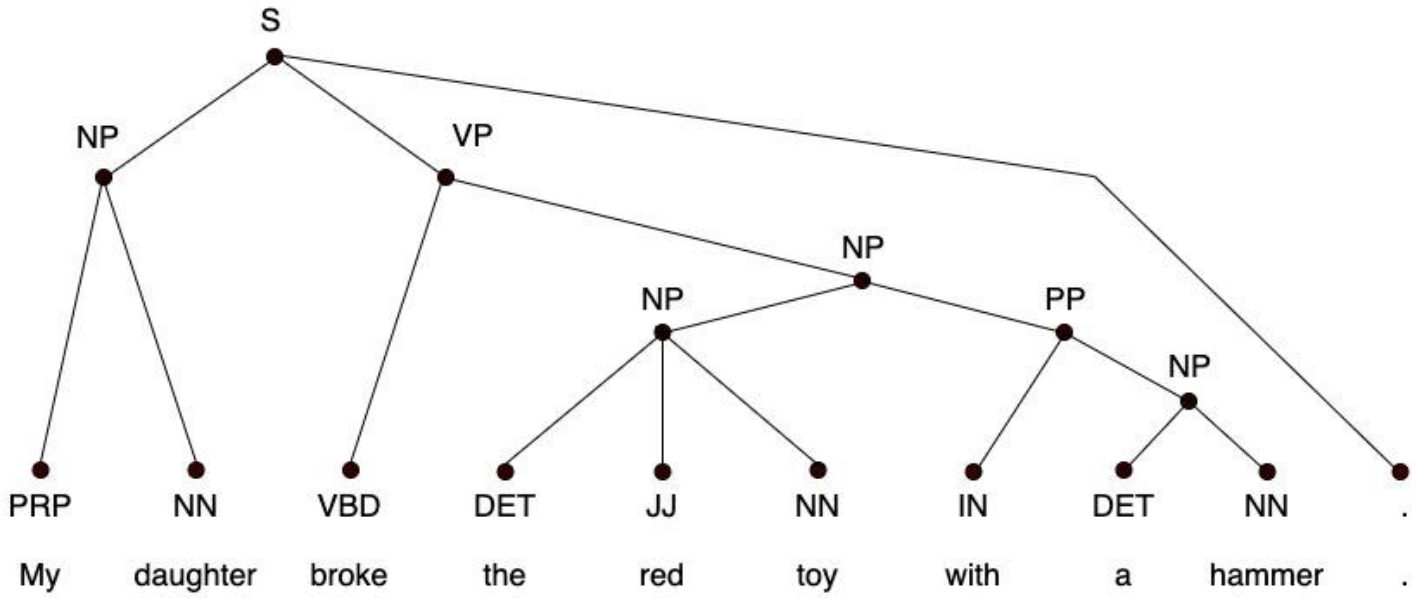
Neural-based Constituency Parsing

- [Gomez-Rodriguez et al. 2018] introduce a method to reduce constituent parsing to sequence labeling.
  - For each word w_t, it generates a label that encodes: (1) the number of ancestors in the tree that the words w_t and w_{t+1} have in common (CA), and (2) the non-terminal symbol at the lowest common ancestor (LCA).
  - They first prove that the proposed encoding function is injective for any tree without unary branches.
  - In practice, the approach is made extensible to all constituency trees by collapsing unary branches.

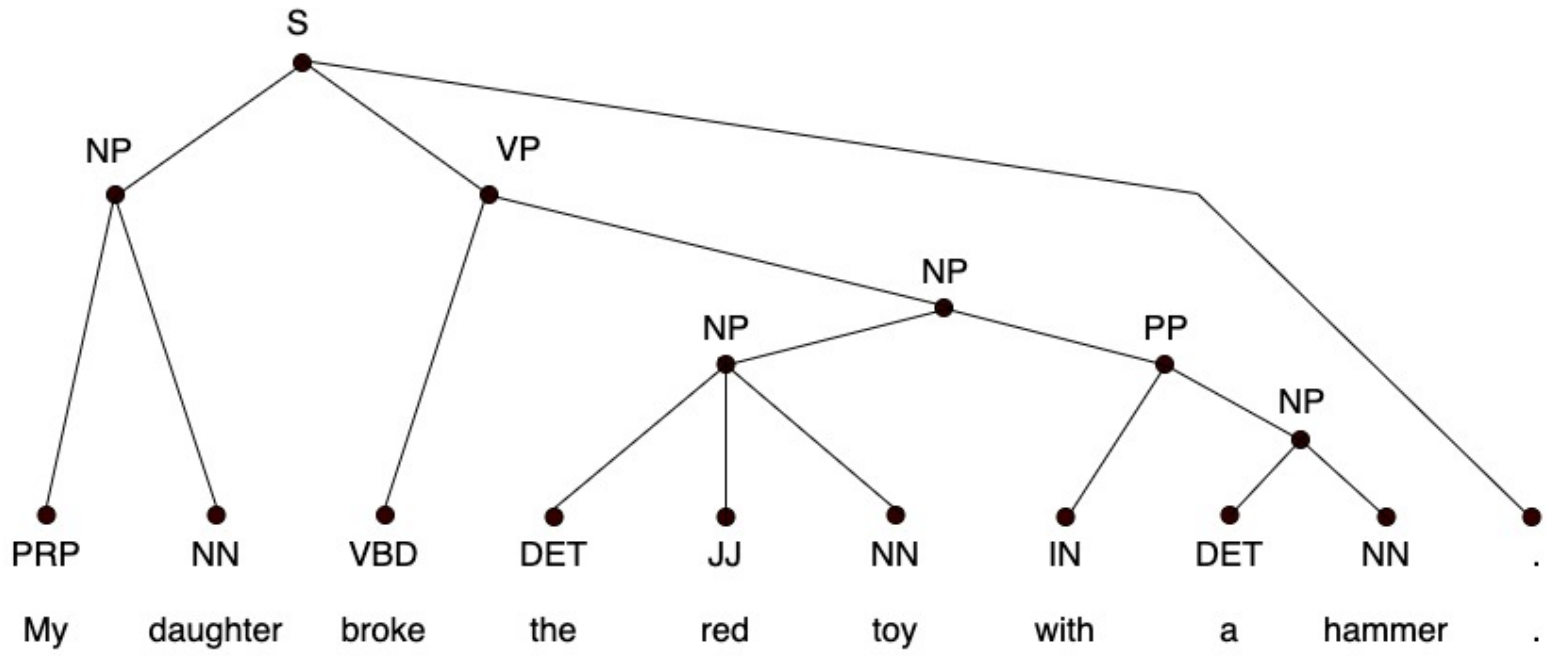
Gómez-Rodríguez, C. and Vilares, D., EMNLP 2018. Constituent parsing as sequence labeling.

Example:
Parsing as Sequence Labeling - 1

- Example of the parsing tree



2



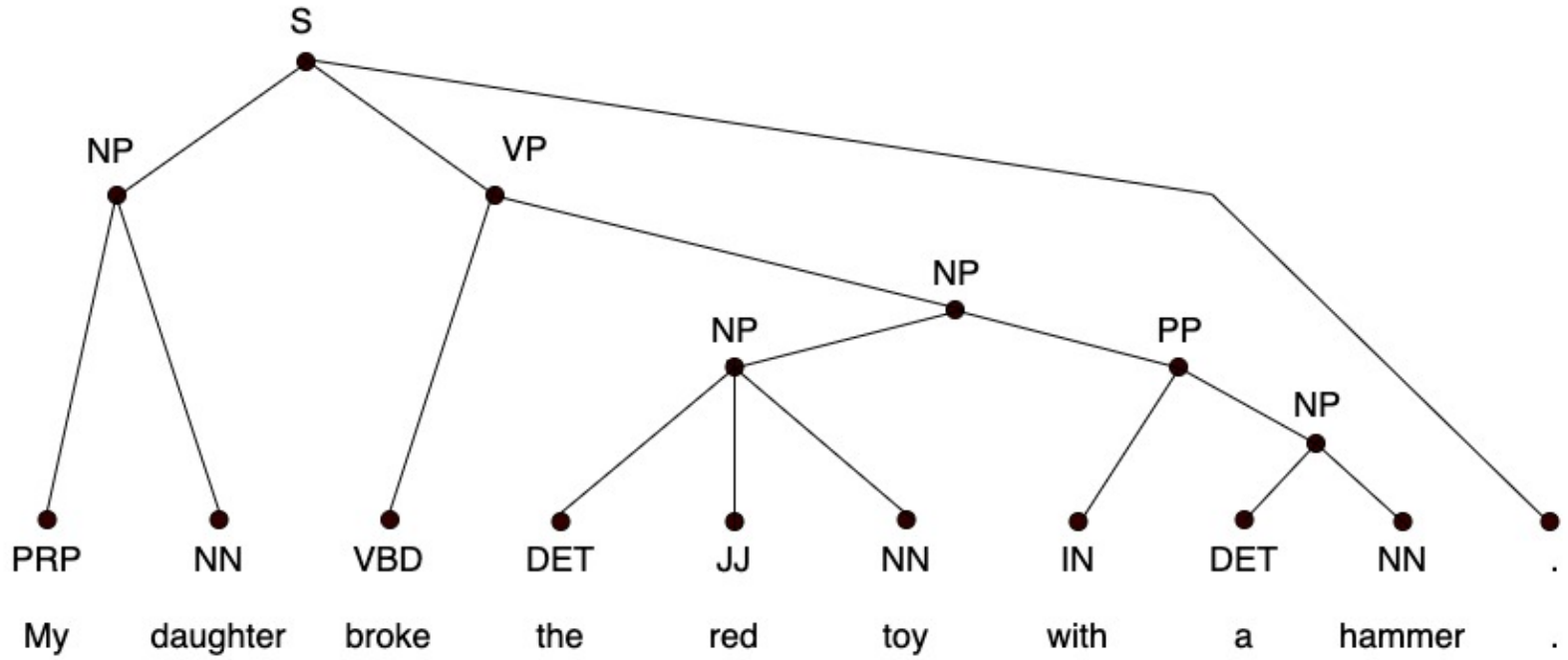
Linearized tree (absolute scale):

2NP

Linearized tree (relative scale):

2NP

3



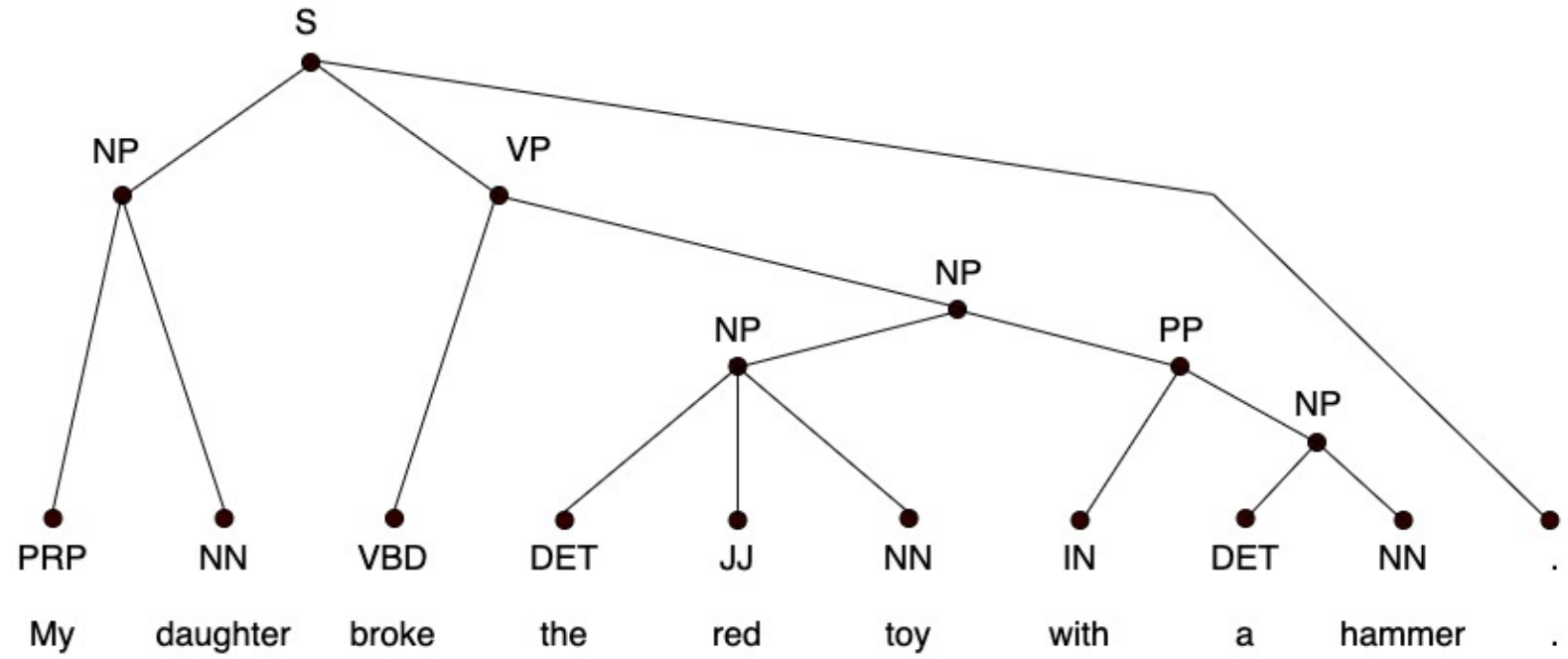
Linearized tree (absolute scale):

2NP 1S

Linearized tree (relative scale):

2NP -1S

4



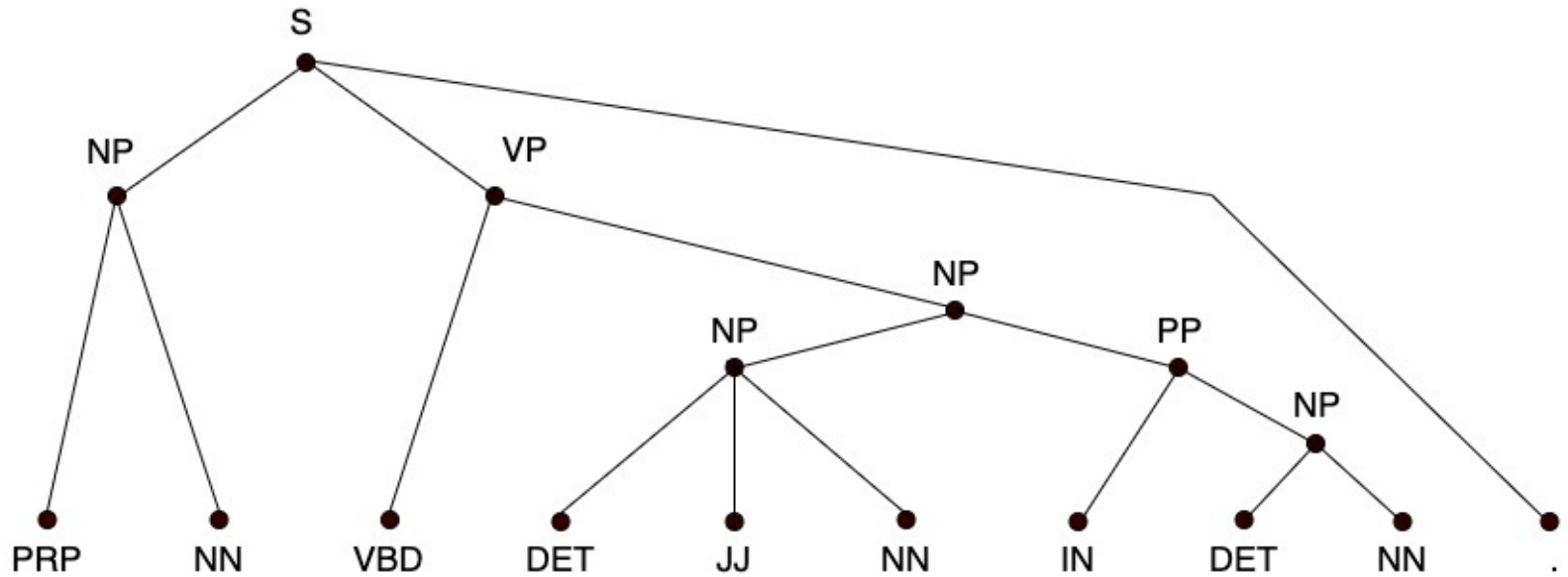
Linearized tree (absolute scale):

2NP 1S 2VP

Linearized tree (relative scale):

2NP -1S 1VP

5

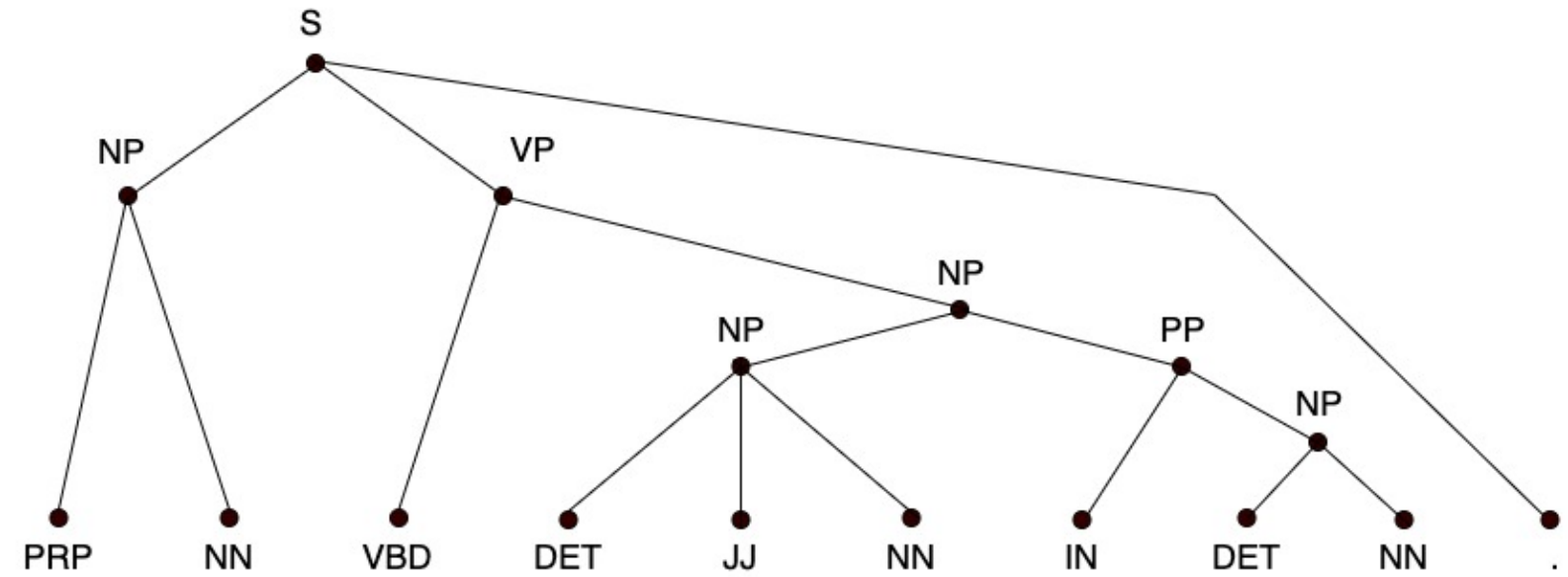


Linearized tree (absolute scale):

2NP 1S 2VP 4NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP

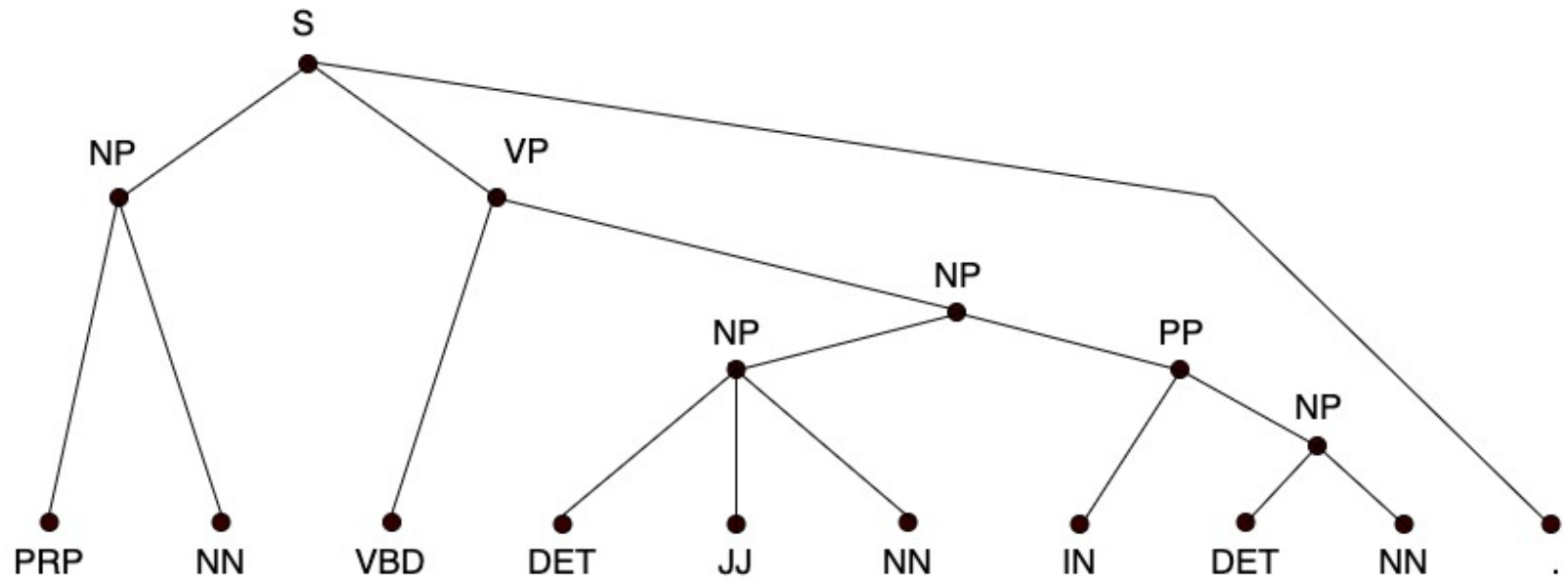


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP

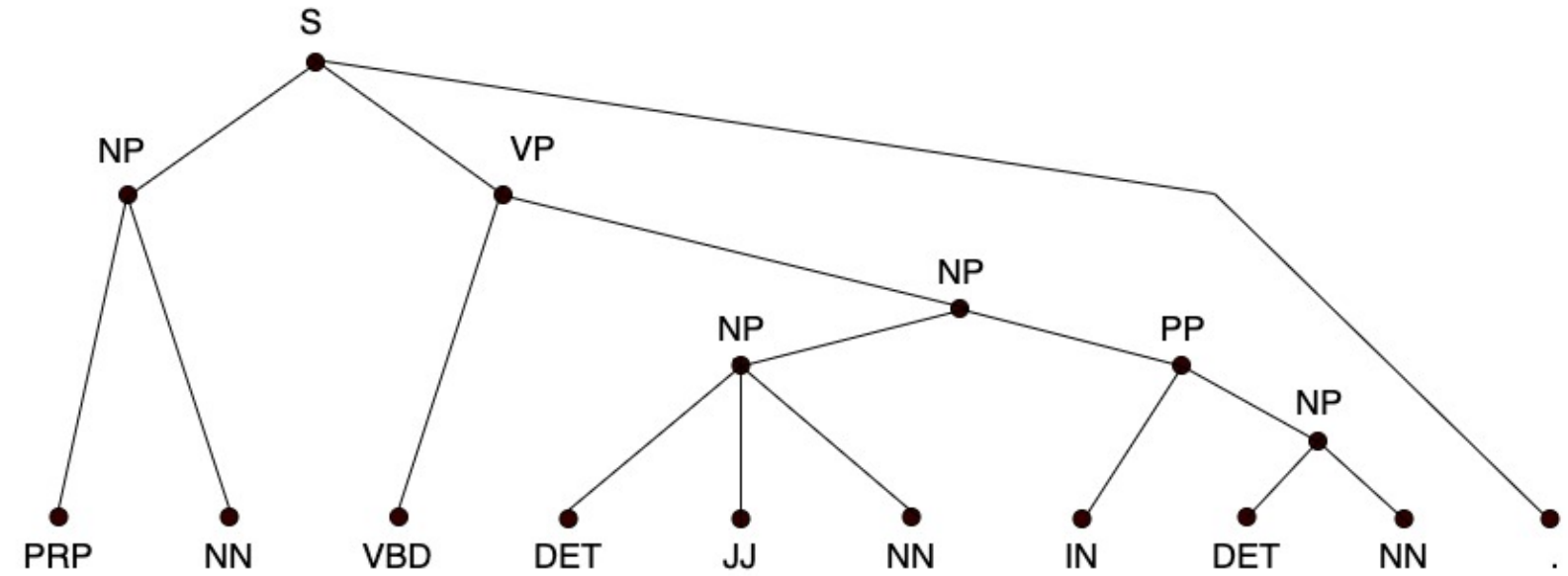


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP

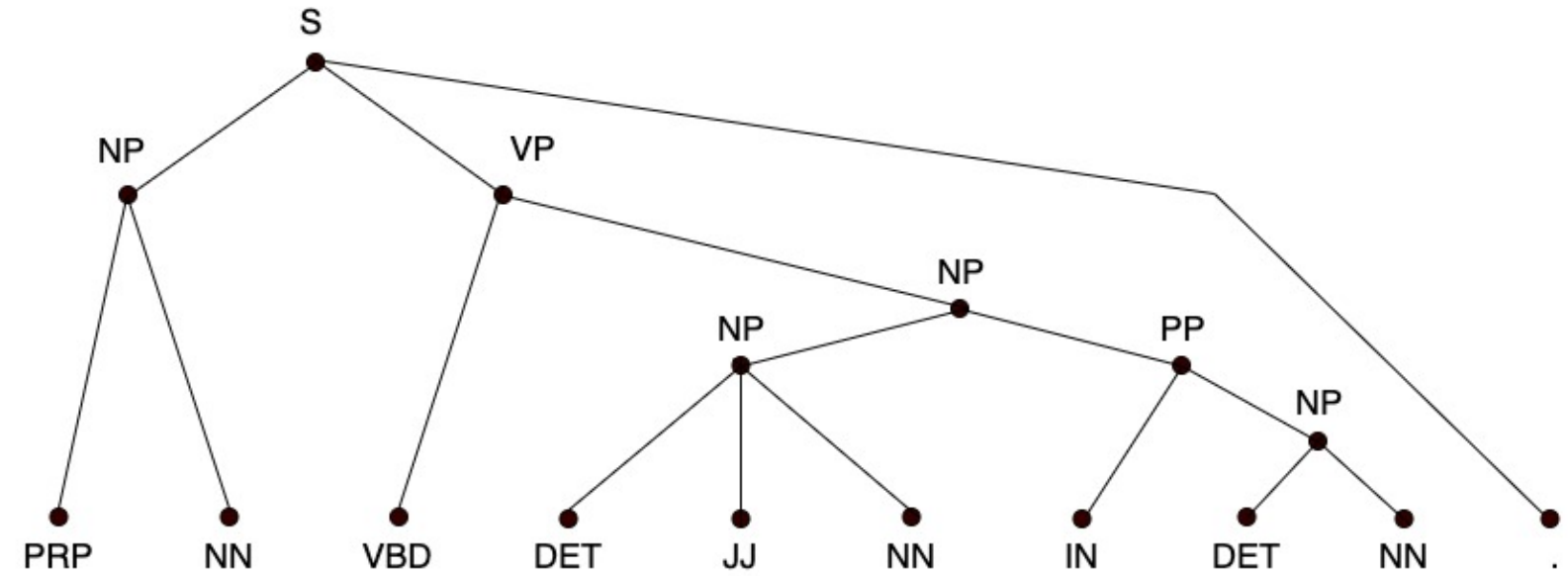


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP 4PP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP 1PP

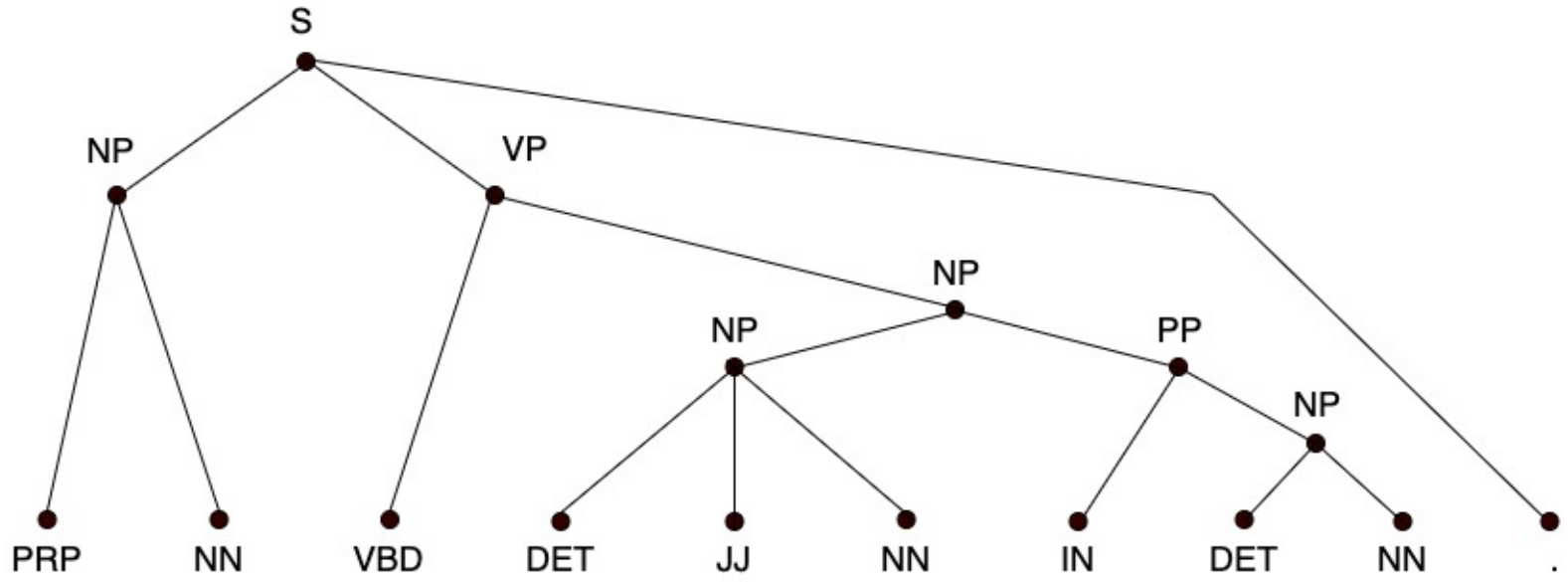


Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP 4PP 5NP

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP 1PP 1NP



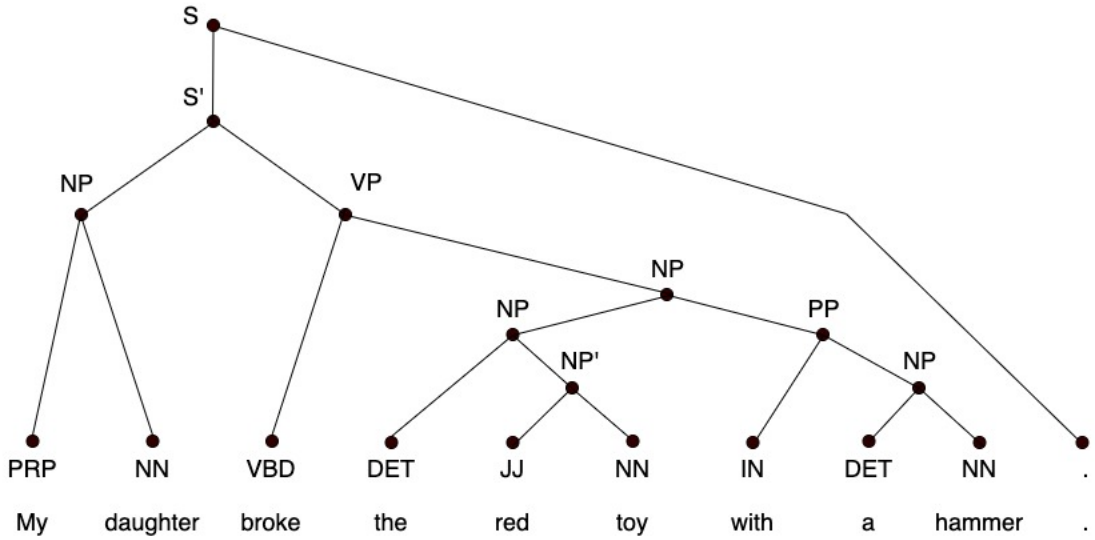
Linearized tree (absolute scale):

2NP 1S 2VP 4NP 4NP 3NP 4PP 5NP 1S

Linearized tree (relative scale):

2NP -1S 1VP 2NP 0NP -1NP 1PP 1NP -4S

- Example of the 2-ary parsing tree



Linearized tree (absolute scale):

3NP 2S' 3VP 5NP 6NP' 4NP 5PP 6NP 1S

Linearized tree (relative scale):

3NP -1S' 1VP 2NP 1NP' -2NP 1PP 1NP -5S

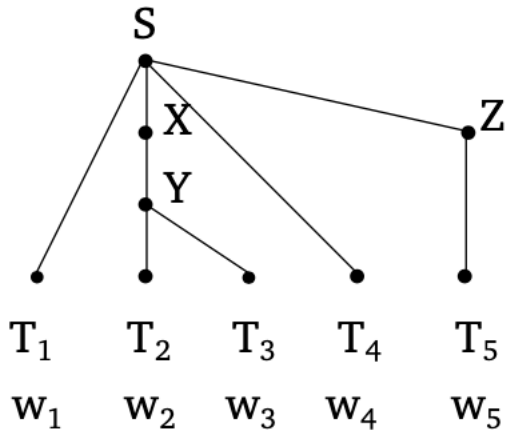
Linearized tree (simplified relative scale):

3NP -S' 1VP 2NP 1NP' -NP 1PP 1NP -S

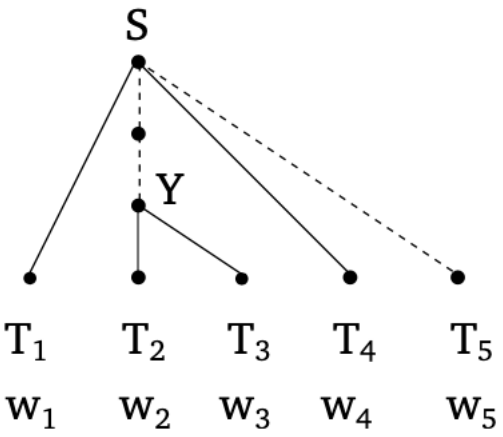
12

- Example of the tree can not be directly linearized

T:



$\Phi^{-1}(\Phi(T))$:



$\Phi(T)$: 1_S 3_Y 1_S 1_S

Today's Class

- Neural-based Syntactic Parsing
- Dependency Syntax
- Dependency Parsing
- Neural-based Dependency Parsing
- Parsing Evaluation
- Parsing Resources

Dependency

- Syntactic structure consists of lexical items, linked by binary asymmetric relations called dependencies.

- [Tesniere 1959]

> The sentence is an *organized whole*, the constituent elements of which are *words*. Every word that belongs to a sentence ceases by itself to be isolated as in the dictionary. Between the word and its neighbors, the mind perceives *connections*, the totality of which forms the structure of the sentence. The structural connections establish *dependency* relations between the words. Each connection in principle unites a *superior* term and an *inferior* term. The superior term receives the name *governor*. The inferior term receives the name *subordinate*.

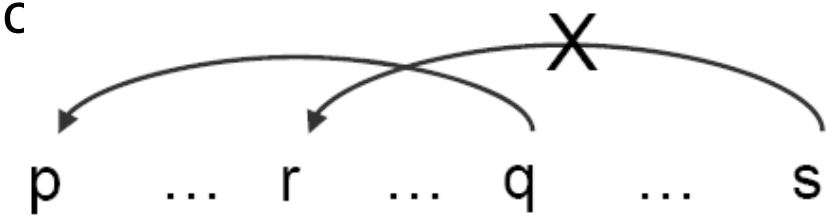
Robinson's axiom

a. One and only one element is independent;

b. All others depend directly on some element;

c. No element depends directly on more than one other; and

d. (Non-crossing constraint): if ***A*** depends directly to ***B*** and some element ***C*** intervenes between them (in linear order of string), then ***C*** depends directly on ***A*** or on ***B*** or to some c



Robinson's axiom

e. An element cannot have dependents lying on the other side of its own governor.(Huang, Changning)

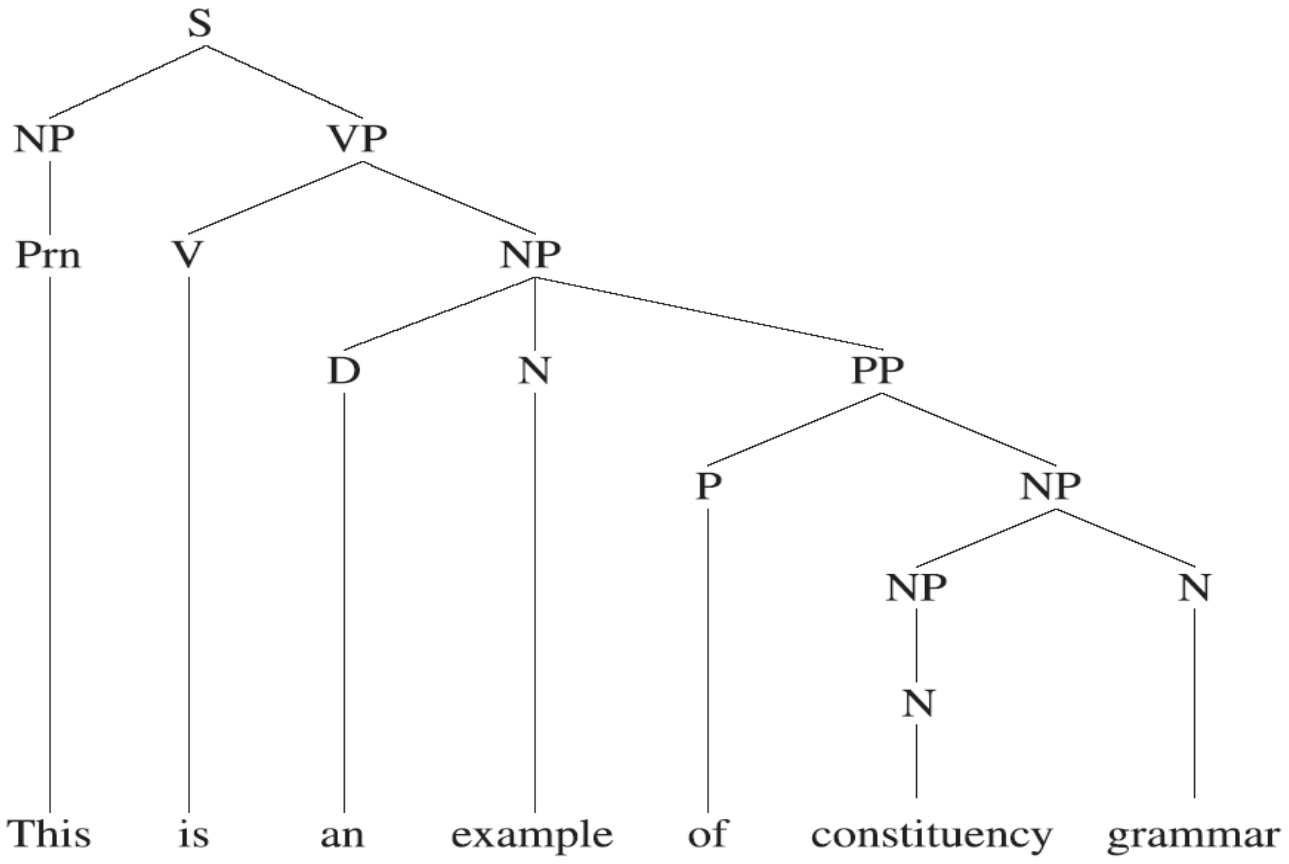
Element:

- Head (Superior, Head, Governor, Regent, 核心词)
  - Plays the role in determining the behavior of the pair.
- Dependent (Inferior, Modifier, Subordinate, 从属词)
  - Modifier, object or complement
  - Pre-dependent, post-dependent
- Link – the relation (Arc)
  - Close to semantic relation
  - With Dependency Type

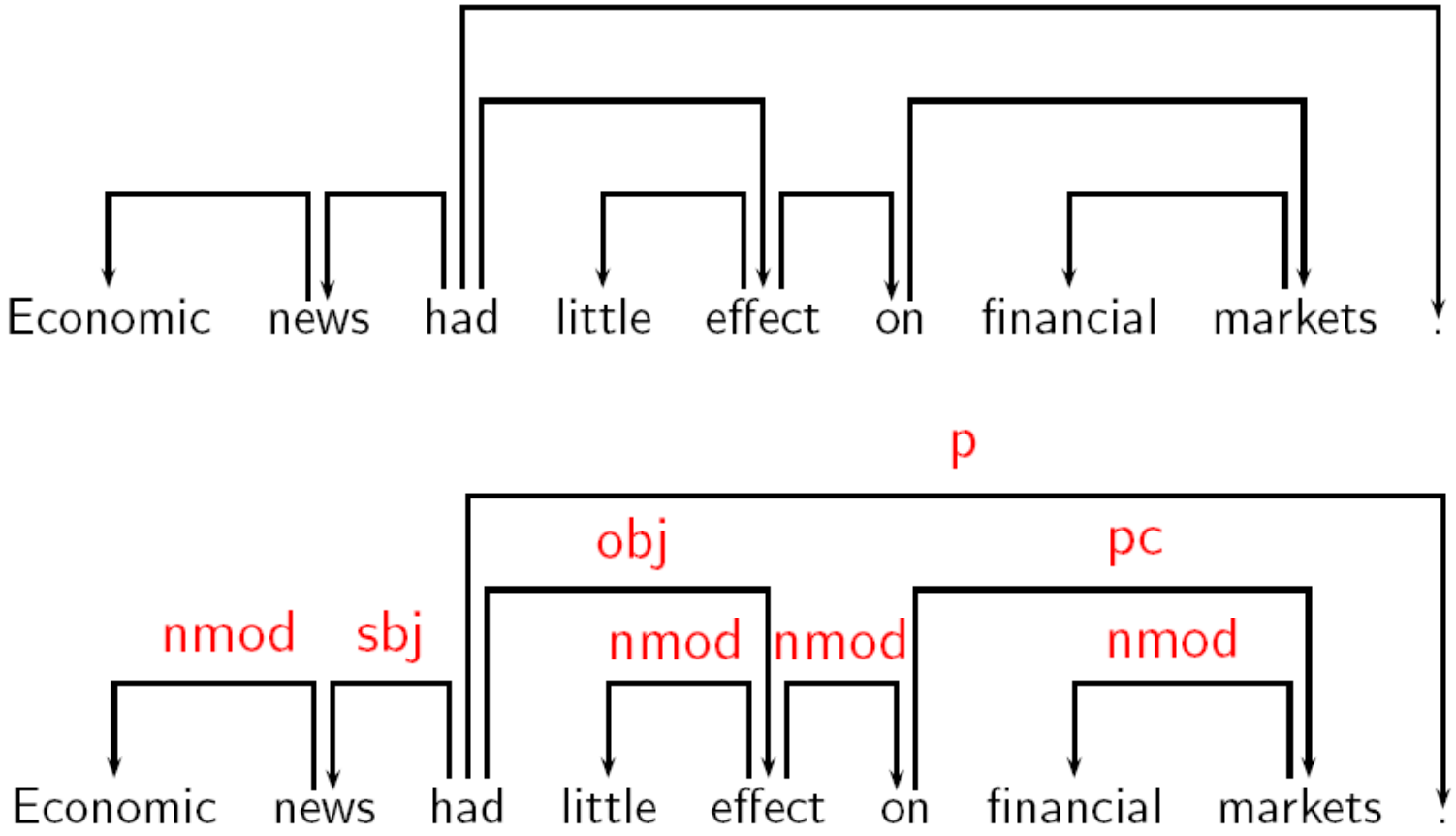
Typical Dependency Types

	Dependency Description	Example
	Adjective and its adverbial modifier	极其/d 惨痛/a *greatly painful*
ADV	Predicate and its adverbial modifier in which the predicate serves as head	沉重/ad 打击/v *heavily strike*
AN	Noun and its adjective modifier	合法/a 收入/n *lawful incoming*
CMP	Predicate and its complement in which the predicate serves as head	医治/v 无效/v *ineffectively treat*
NJX	Juxtaposition structure	公正/a 合理/a *fair and reasonable*
NN	Noun and its nominal modifier	人身/n 安全/n *personal safety*
SBV	Predicate and its subject	财产/n 转移/v *property transfer*
VO	Predicate and its object in which the predicate serves as head	转换/v 机制/n *change mechanism*
VV	Serial verb constructions which indicates that there are serial actions	跟踪/v 报导/v *trace and report*
OT	Others	

Phrase Structure



Dependency Structure



The Comparison

	Phrase structure	Dependency structure
word relation	phrasal constitution	head-dependent
categories	syntactic functional	syntactic structural
new node (Y/N)	multiple nodes per word	one node per word
operation	waiting for complete phrase	word-at-a-time

Some Theoretical Frameworks

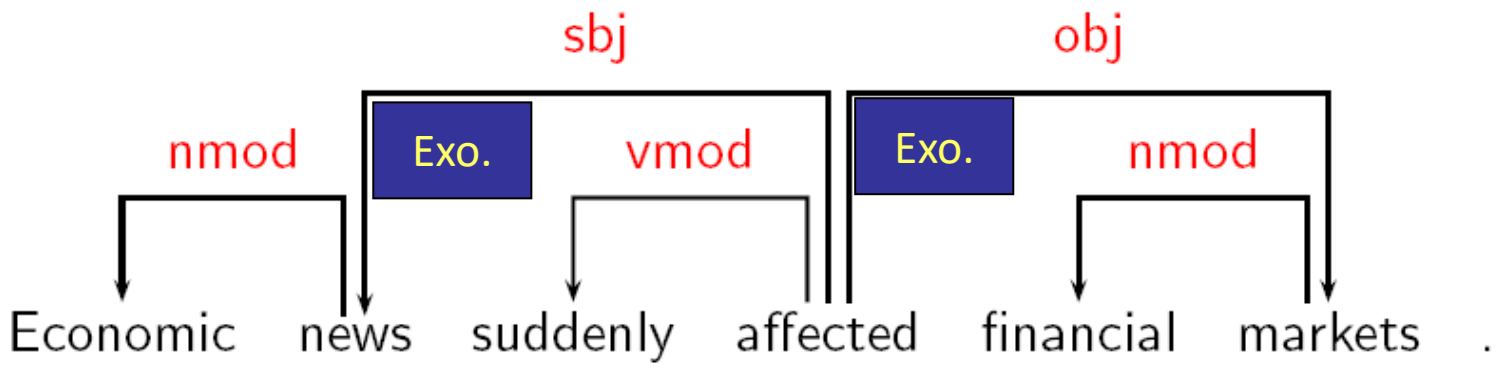
- Word Grammar (WG) [Hudson 1984, Hudson 1990]
- Functional Generative Description (FGD) [Sgall et al. 1986]
- Dependency Unification Grammar (DUG)
  - [Hellwig 1986, Hellwig 2003]
- Meaning-Text Theory (MTT) [Melcuk 1988]
- (Weighted) Constraint Dependency Grammar ([W]CDG)
  - [Maruyama 1990, Harper and Helzerman 1995, Menzel and Schr¨oder 1998, Schr¨oder 2002]
- Functional Dependency Grammar (FDG)
  - [Tapanainen and Jarvinen 1997, Jarvinen and Tapanainen 1998]
- Topological/Extensible Dependency Grammar ([T/X]DG)
  - [Duchier and Debusmann 2001, Debusmann et al. 2004]

Example

outwards

inwards

Construction	Head	Dependent
Exocentric	Verb	Subject (sbj)
	Verb	Object (obj)
Endocentric	Verb	Adverbial (vmod)
	Noun	Attribute (nmod)

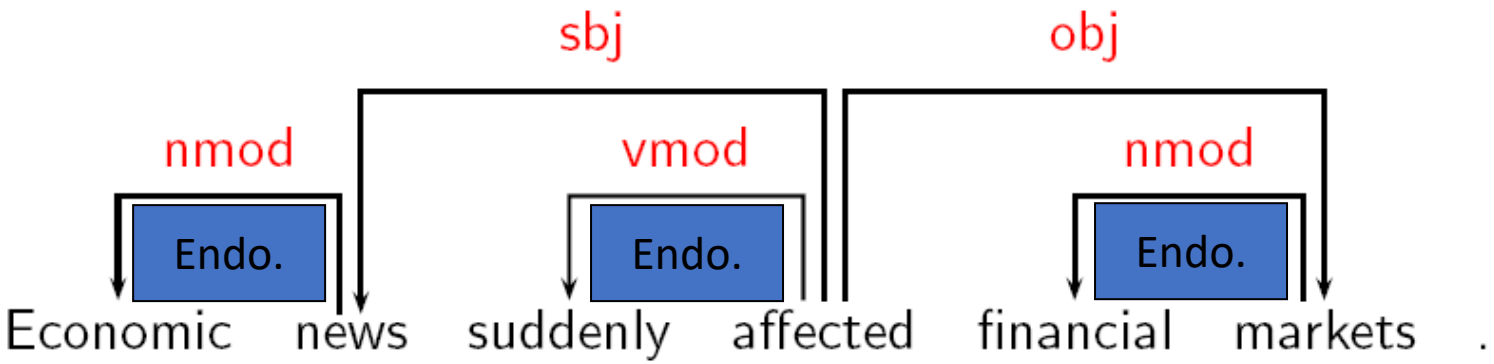


Example

outwards

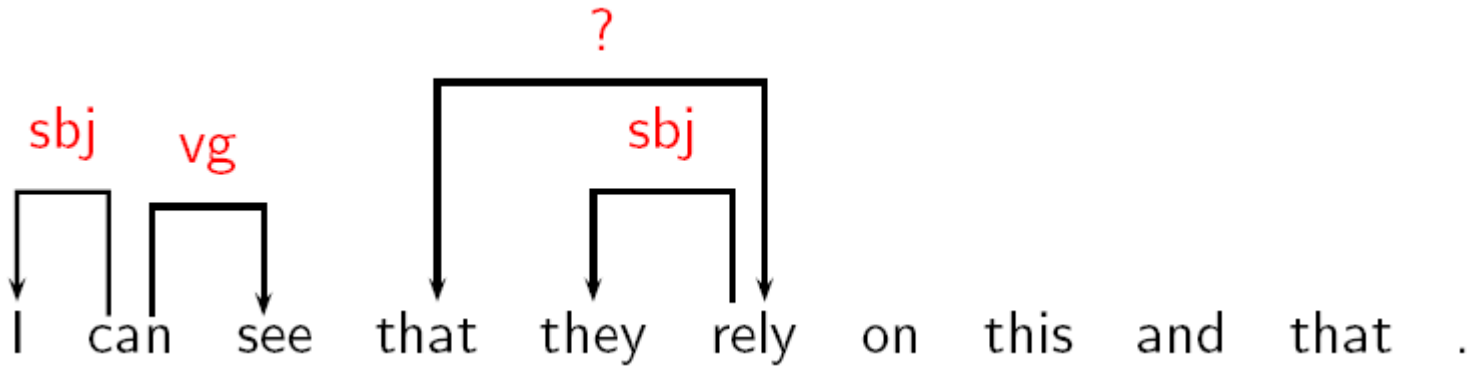
inwards

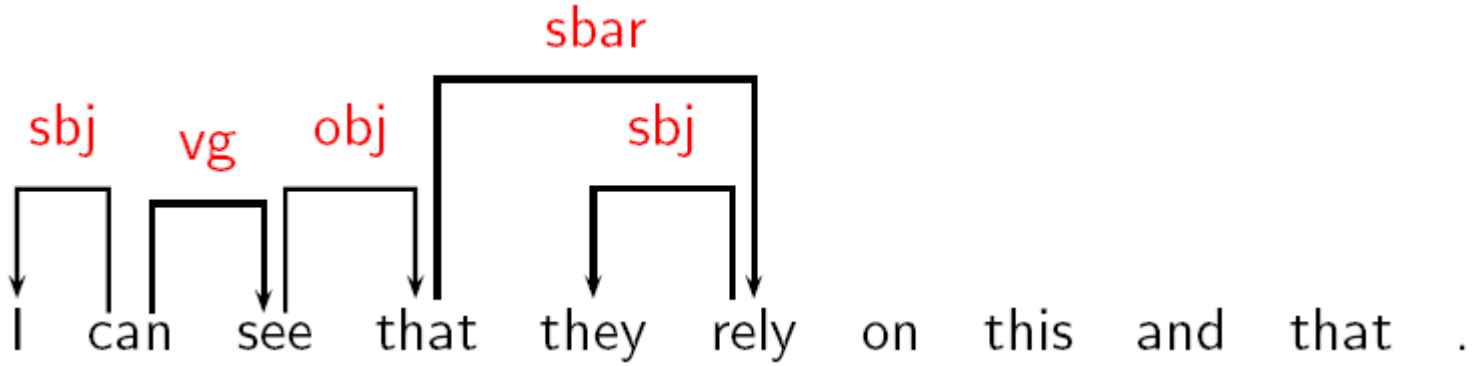
Construction	Head	Dependent
Exocentric	Verb	Subject (sbj)
	Verb	Object (obj)
Endocentric	Verb	Adverbial (vmod)
	Noun	Attribute (nmod)



Tricky Cases

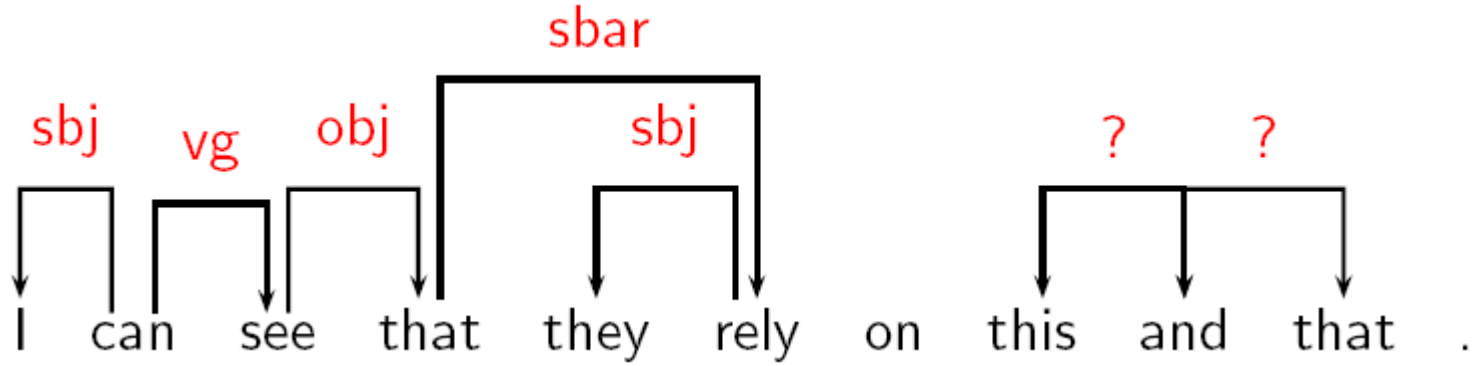
- Subordinate clauses (complementizer ↔ verb)

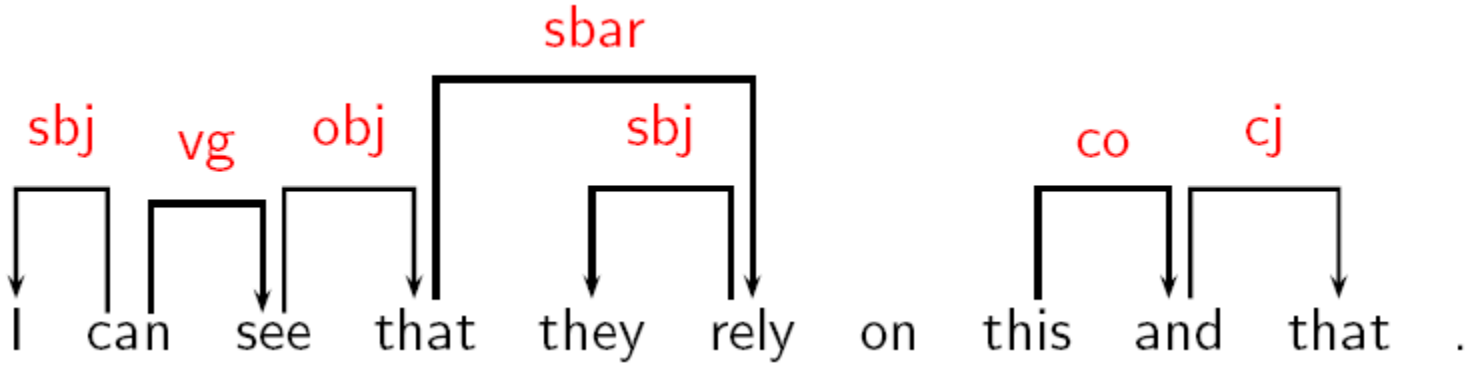




Tricky Cases

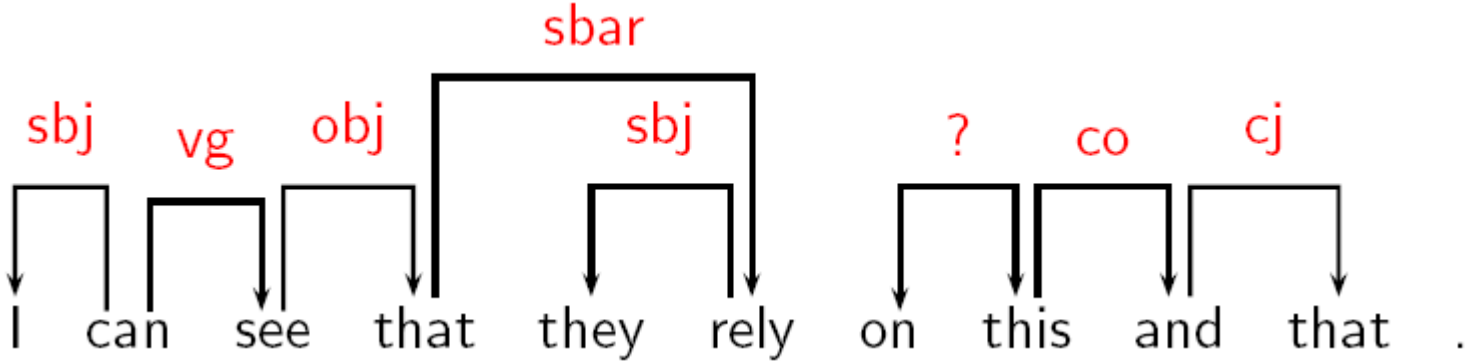
- Coordination (coordinator ↔ conjuncts)

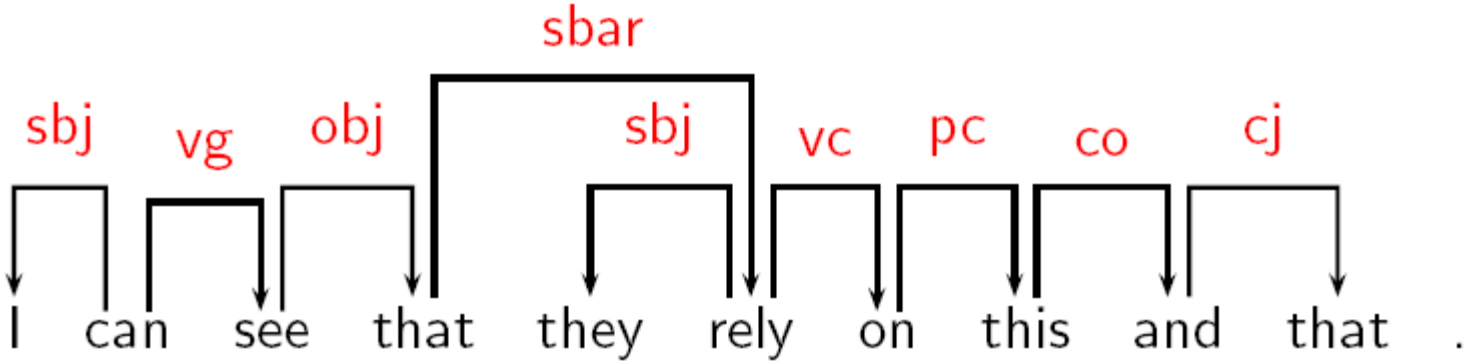




Tricky Cases

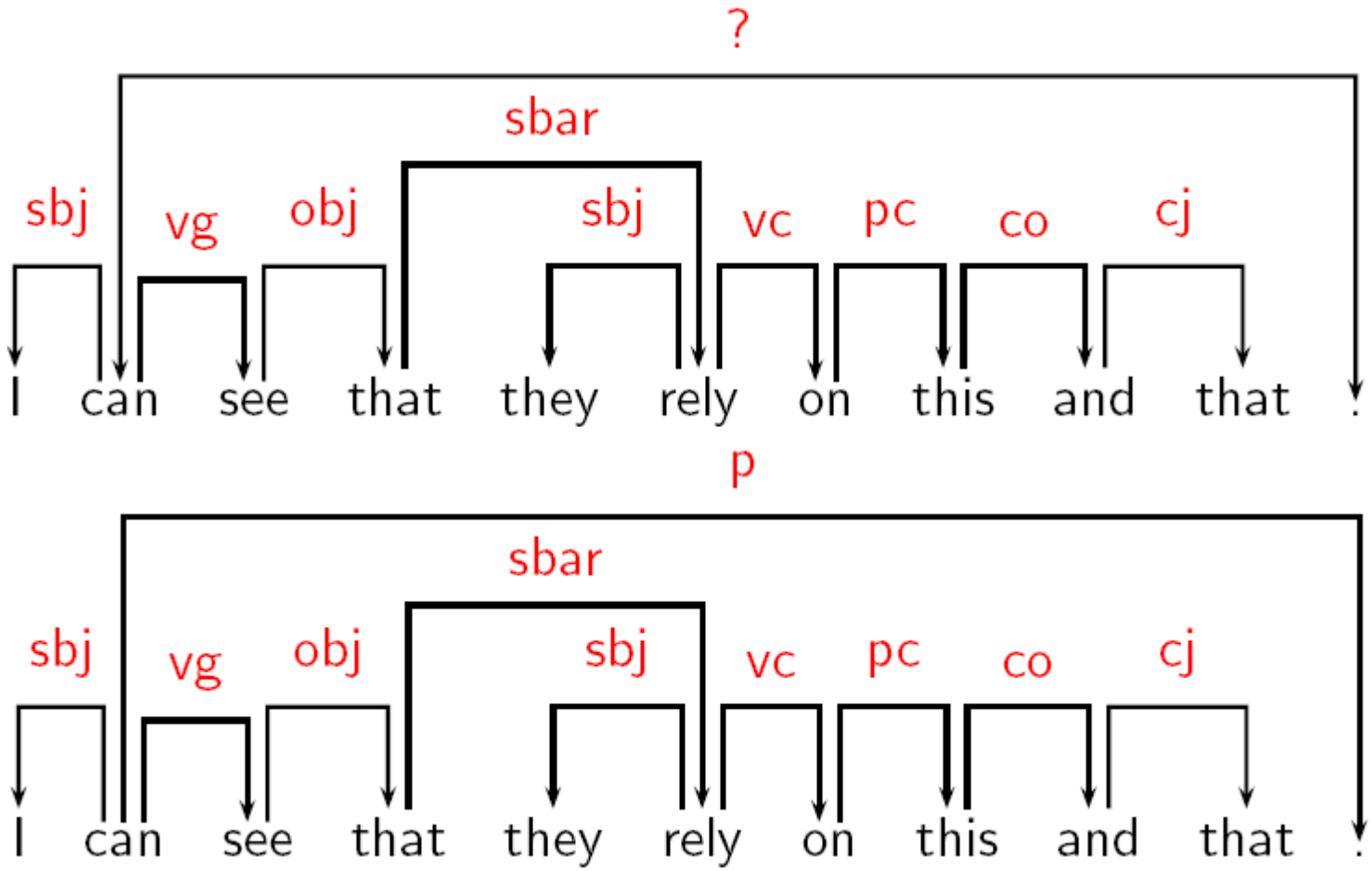
- Prepositional phrases (preposition ↔ nominal)





Tricky Cases

- Punctuation



Dependency Parsing

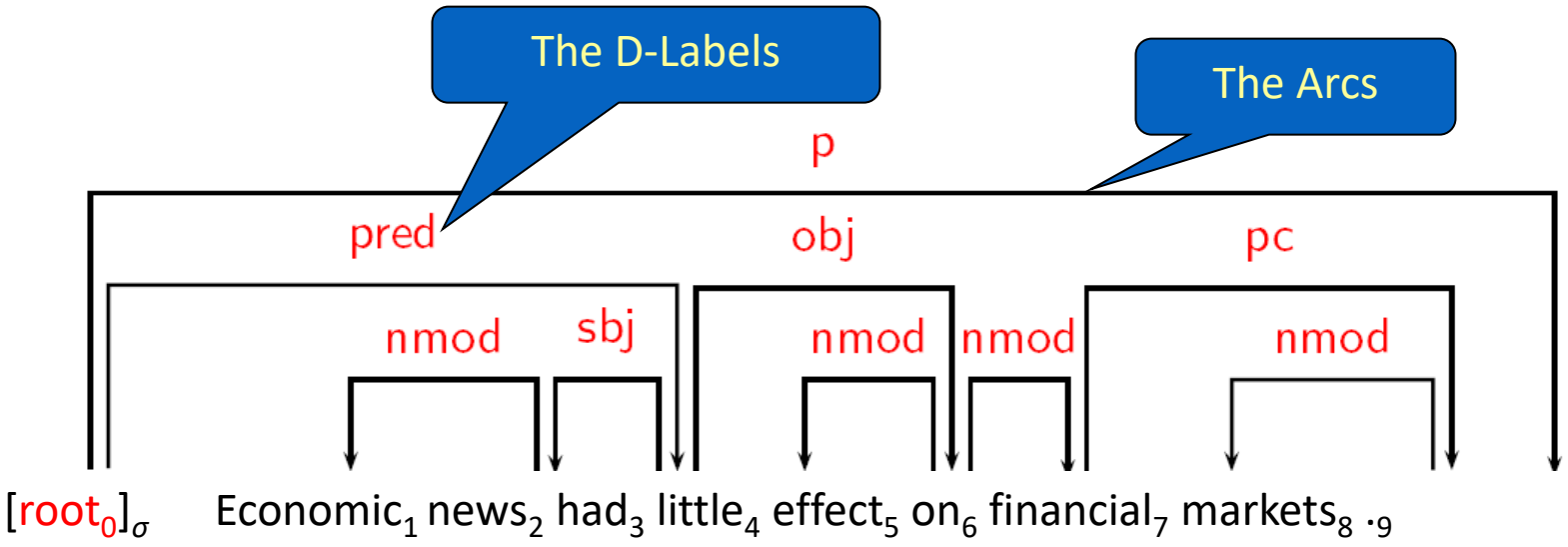
- Input:
  - Sentence $W = w_0, w_1, \ldots, w_n$ with w_0=*root*
- Output:
  - Dependency graph $G = (V, A)$ for W where:
  - $V = \{0, 1, \ldots, n\}$ is the *vertex* set,
  - A is the *arc* set, i.e., $(i, j, k) \in A$ represents a dependency from w_i to w_j with label $l_k \in L$

Example

- Input:

$[\text{root}_0]_\sigma$ Economic_1 news_2 had_3 little_4 effect_5 on_6 financial_7 markets_8 $.9$

- Output:



Approaches

- Grammar-based parsing
  - Context-free dependency grammar
  - Constraint dependency grammar

- Data-driven parsing
  - Transition-based models
  - Graph-based models

Context-free Dependency Grammar

- Basic idea
  - Dependency grammar as lexicalized context-free grammar
  - Standard context-free parsing algorithms (CKY, Earley, etc.)
- Recent developments:
  - Link Grammar [Sleator and Temperley 1991]
  - Earley-style parser with left-corner filtering
  - [Lombardo and Lesmo 1996]
  - Bilexical grammars [Eisner 1996, Eisner 2000]

Constraint Dependency Grammar

- Parsing as constraint satisfaction [Maruyama 1990]:
  - Grammar consists of a set of boolean constraints, i.e. logical formulas that describe well-formed dependency graphs.
  - Constraint propagation removes candidate graphs that contradict constraints (eliminative parsing).

- Recent developments:
  - Weighted Constraint Dependency Grammar [Menzel and Schr¨oder 1998, Foth et al. 2004]
  - Probabilistic Constraint Dependency Grammar [Harper and Helzerman 1995, Wang and Harper 2004]
  - Topological/Extensible Dependency Grammar [Duchier and Debusmann 2001, Debusmann et al. 2004]

Transition-based models

- Basic idea:
  - Define a transition system (state machine, MM) for mapping a sentence to its dependency graph.
  - Learning: Induce a model for predicting the next state transition, given the transition history.
  - Parsing: Construct the optimal transition sequence, given the induced model.

- Characteristics:
  - Local training of a model for optimal transitions
  - Greedy search/inference

Transition Systems

- A transition system for dependency parsing is a quadruple $S = (C, T, c_s, C_t)$,where
  1. C is a set of configurations, each of which contains a buffer β of (remaining) nodes and a set A of dependency arcs,
  2. T is a set of transitions, each of which is a (partial) function $t: C \rightarrow C$,
  3. C_s is an initialization function, mapping a sentence $x = W_0, W_1, \ldots, W_n$ to a configuration with $\beta = [1, \ldots, n]$,
  4. $C_t \subseteq C$ is a set of terminal configurations.

- Note:
  - A configuration represents a parser state.
  - A transition represents a parsing action (parser state update).

Transition Sequences

- Let $S = (C, T, c_s, C_t)$ be a transition system.

- A transition sequence for a sentence $x = W_0, W_1, \ldots, W_n$ in S is a sequence $C_{0,m} = (C_0, C_1, \ldots, C_m)$ of configurations, such that
  1. $c_0 = c_s(x)$,
  2. $C_m \in C_t$,
  3. For every i $(1 \leq i \leq m)$, $C_i = t(c_{i-1})$ for some $t \in T$.

- The parse assigned to x by $C_{0,m}$ is the dependency graph $G_{Cm} = (\{0,1,..,n\}, A_{Cm})$, where A_{Cm} is the set of dependency arcs in C_m.

Lexicalized Dependency Probabilistic Model

- Collins, 1996
- Training: Retrieve the probability of any words with specific dependency through Maximal Likelihood Estimation.
- Parsing (Decoding): Search a dependency tree with the highest product of production probability of all dependent pairs

Probabilistic Dependency Model

- Eisner, 1996
- For each candidate dependency tree, *T*, its production probability is defined as the probability product of the production probability of each node in T

$$Gen(T) = \prod_{x \in T} Gen(x)$$

- Task of Parsing: Lc ndency tree with the highest production probability

Shift-Reduce Dependency Parsing

- Transitions:
  - Left-Arc$_k$:
    $(\sigma|i, j|\beta, A) \rightarrow (\sigma, j|\beta, A \cup \{(j, i, k)\})$
  - Right-Arc$_k$:
    $(\sigma|i, j|\beta, A) \rightarrow (\sigma, i|\beta, A \cup \{(i, j, k)\})$
  - Shift:
    $(\sigma, i|\beta, A) = (\sigma|i, \beta, A)$
- Preconditions:
  - Left-Arc$_k$:
    $\neg[i = 0]$
    $\neg\exists i'\exists k'[(i', i, k') \in A]$
  - Right- Arc$_k$:
    $\neg\exists i'\exists k'[(i', j, k') \in A]$
-

Example: Shift-Reduce Parsing - 1

$[root_0]_\sigma$ [Economic$_1$ news$_2$ had$_3$ little$_4$ effect$_5$ on$_6$ financial$_7$ markets$_8$.$_9$]$_\beta$

We have nothing in the configuration.
Just shift to first word

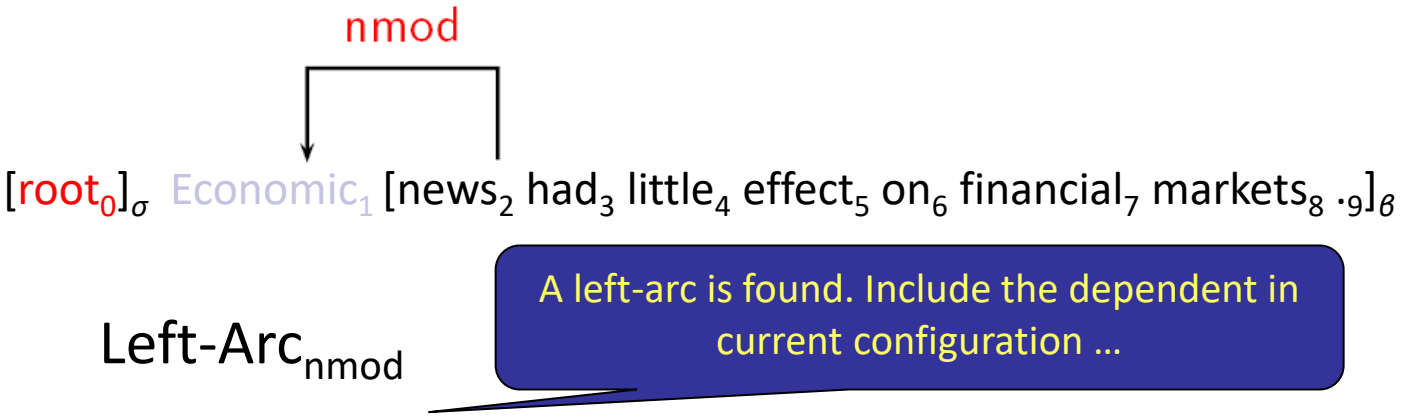
2

$[\text{root}_0\ \text{Economic}_1]_\sigma\ [\text{news}_2\ \text{had}_3\ \text{little}_4\ \text{effect}_5\ \text{on}_6\ \text{financial}_7\ \text{markets}_8\ .{}_9]_\beta$

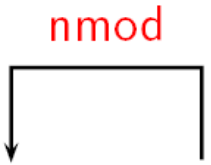
Shift

We have one word in the configuration. Now to search the in/out arcs …

3


nmod
[root0]σ Economic1 [news2 had3 little4 effect5 on6 financial7 markets8 .9]β
Left-Arcnmod
A left-arc is found. Include the dependent in current configuration …

4

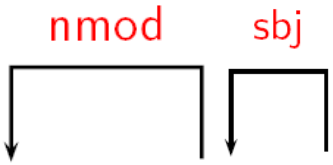


[root$_0$ Economic$_1$ news$_2$]$_\sigma$ [had$_3$ little$_4$ effect$_5$ on$_6$ financial$_7$ markets$_8$.$_9$]$_\beta$

Shift

Current configuration is self-contained. Now shift is required to include the next word …

5

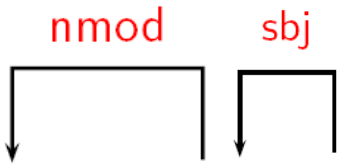


$[root_0]_\sigma$ $Economic_1$ $news_2$ $[had_3$ $little_4$ $effect_5$ on_6 $financial_7$ $markets_8$ $._9]_\beta$

Left-Arc_{sbj}

But the next word holds a *SBJ* left arc to word *news*.
The configuration is reduced to empty again.

6

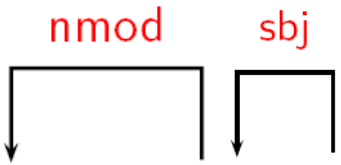


[root$_0$ Economic$_1$ news$_2$ had$_3$]$_\sigma$ [little$_4$ effect$_5$ on$_6$ financial$_7$ markets$_8$.$_9$]$_\beta$

Shift

The *SBJ* left arc creates a new configuration. We now shift to the next word.

7

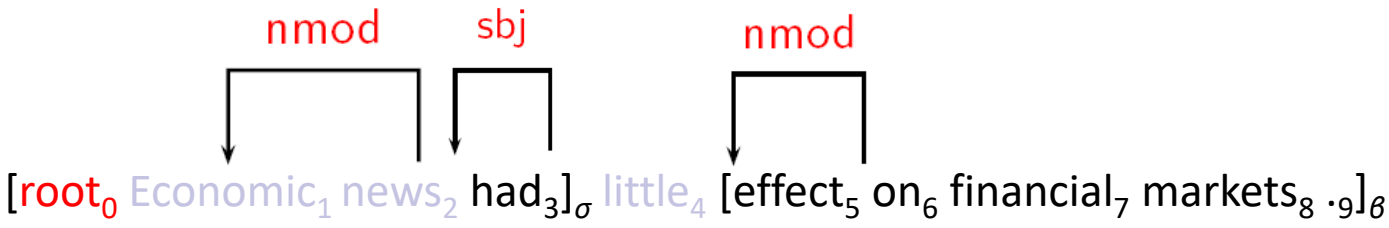


[root$_0$ Economic$_1$ news$_2$ had$_3$ little$_4$]$_\sigma$ effect$_5$ on$_6$ financial$_7$ markets$_8$.$_9$]$_\beta$

Shift

The next word holds no valid arc with current configuration. Then we continue to shift.

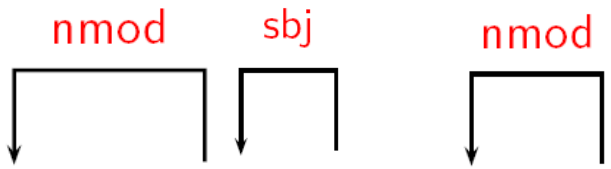
8



Left-Arc$_{nmod}$

Now a NMOD left arc is found. The current configuration is reduced to previous one.

9

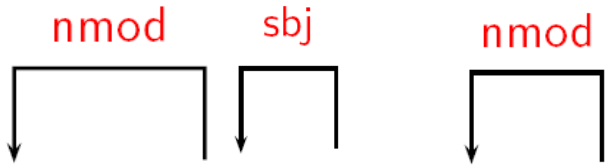


[root$_0$ Economic$_1$ news$_2$ had$_3$ little$_4$ effect$_5$]$_\sigma$ [on$_6$ financial$_7$ markets$_8$.$_9$]$_\beta$

Shift

The valid NMOD left arc is included in the current configuration. Shift to next word.

10

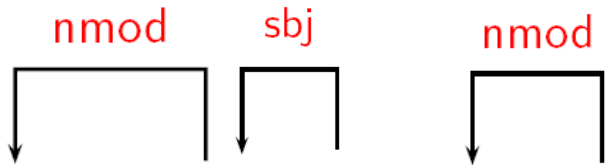


[root$_0$ Economic$_1$ news$_2$ had$_3$ little$_4$ effect$_5$ on$_6$]$_\sigma$ [financial$_7$ markets$_8$.$_9$]$_\beta$

Shift

The word on should hold an arc to its neighbors. It is not word effect. So we continue to shift.

11

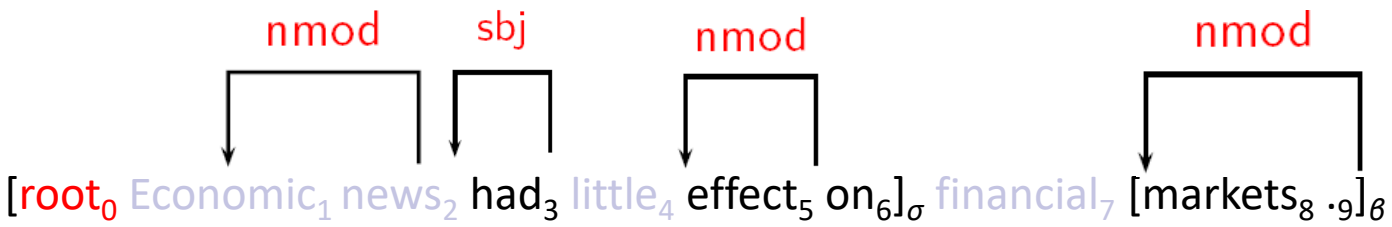


[root$_0$ Economic$_1$ news$_2$ had$_3$ little$_4$ effect$_5$ on$_6$ financial$_7$]$_\sigma$ [markets$_8$.$_9$]$_\beta$

Shift

We include the new word in the configuration. However, the arc from *on* to *financial* is not valid. Shift but reduce to previous configuration.

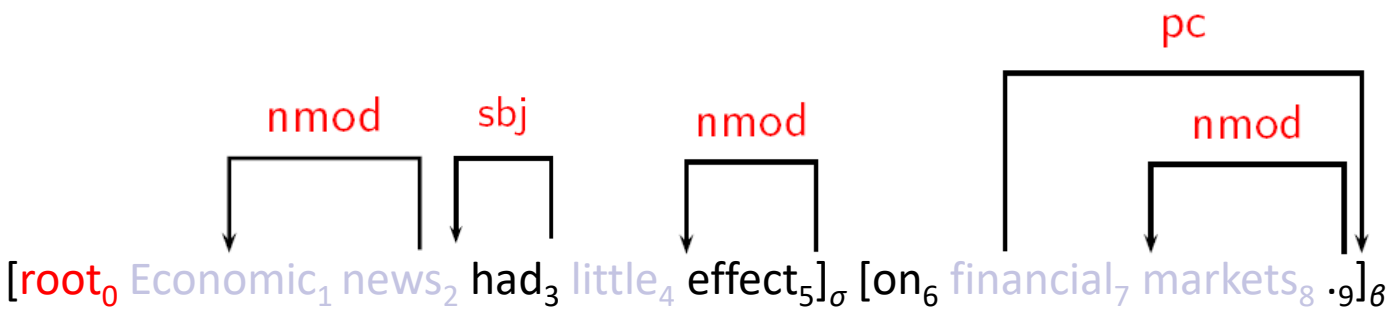
12



Left-Arc$_{nmod}$

New NMOD left arc is found. The remaining word is punctuation. Stop shifting. Search right arc.

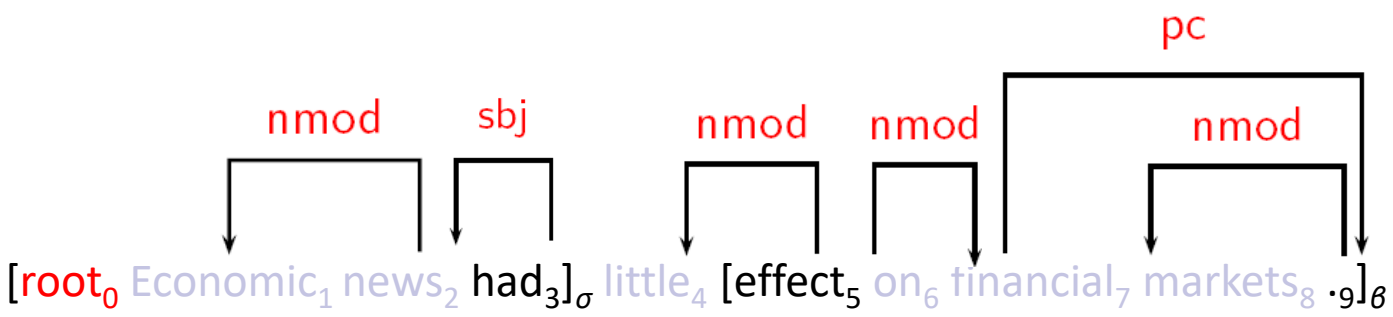
13



Right-Arc$_{pc}$

A PC right arc is found from *on* to *markets*. Reduce configuration. Continue searching right arc.

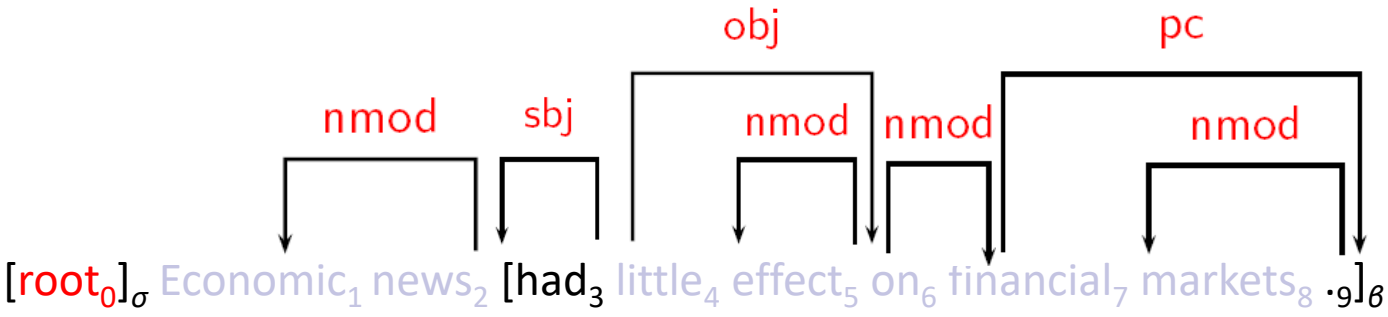
14



Right-Arc$_{nmod}$

A PC right arc is found from *effect* to *on*. Reduce configuration. Continue searching right arc.

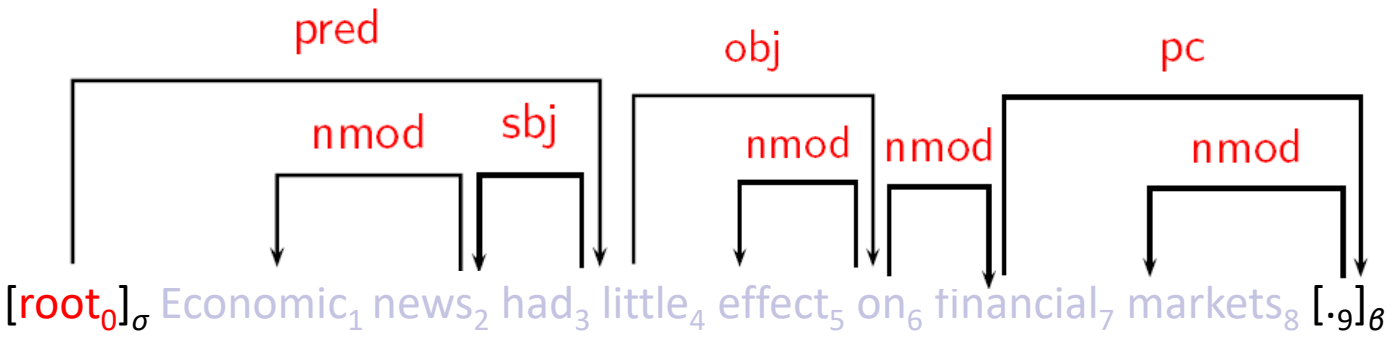
15



Right-Arc$_{obj}$

A PC right arc is found from *had* to *effect*. Reduce configuration. Continue searching right arc.

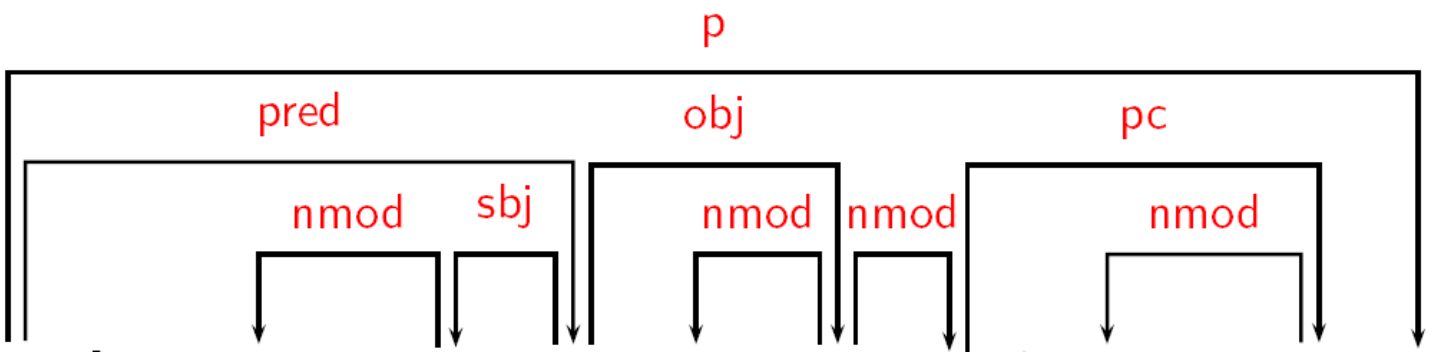
16



Right-Arc$_{pred}$

A PC right arc is found from *root* to *had*. Reduce configuration. Continue searching right arc.

17

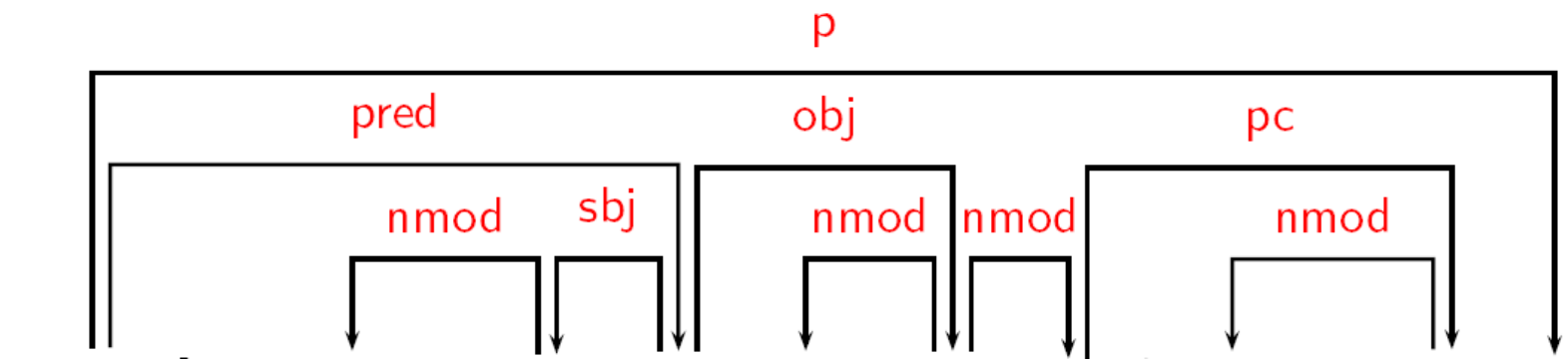


$[]_{\sigma}$ [$root_0$] $_{\beta}$ $Economic_1$ $news_2$ had_3 $little_4$ $effect_5$ on_6 $financial_7$ $markets_8$ $._9$

Right-Arc_p

A PC right arc is found from *root* to *punctuation*. Reduce configuration. Continue searching right arc.

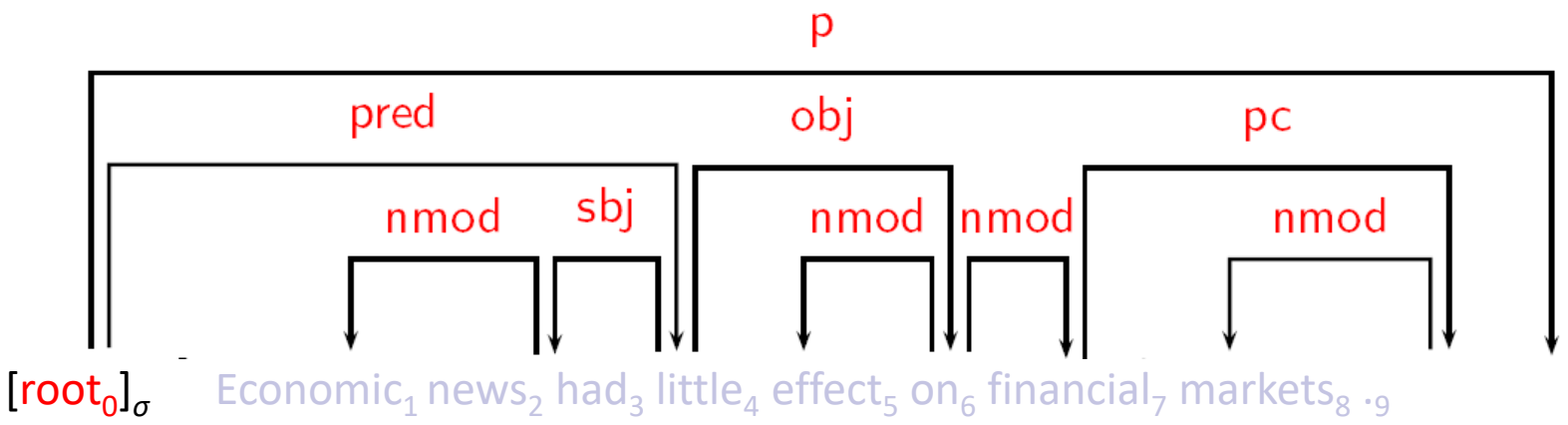
18



Shift

No right arc is found. Remove the empty configuration. Parsing over.

19


p
pred
obj
pc
nmod
sbj
nmod
nmod
nmod
[root0]σ Economic1 news2 had3 little4 effect5 on6 financial7 markets8 .9

Shift

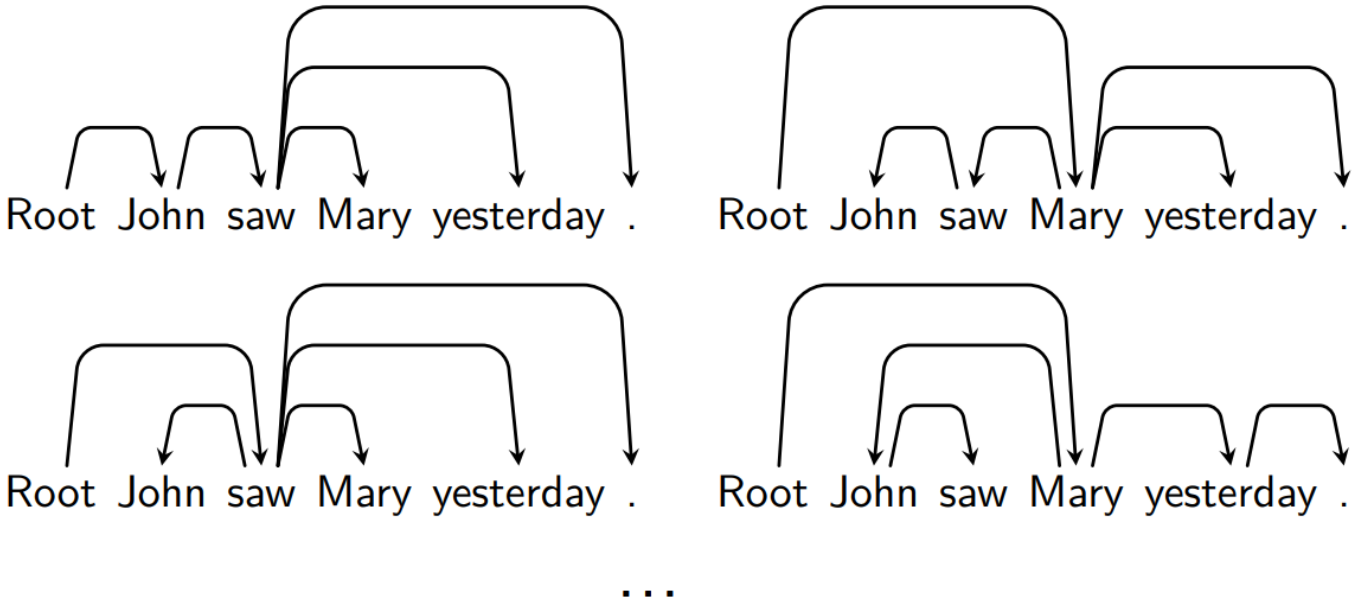
Graph-based Models

- Basic idea:
  - Define a space of candidate dependency graphs for a sentence.
  - Learning: Induce a model for *scoring* an entire dependency graph for a sentence.
  - Parsing: Find the highest-scoring dependency graph, given the induced model.

- Characteristics:
  - Global training of a model for optimal dependency graphs
  - Exhaustive search/inference

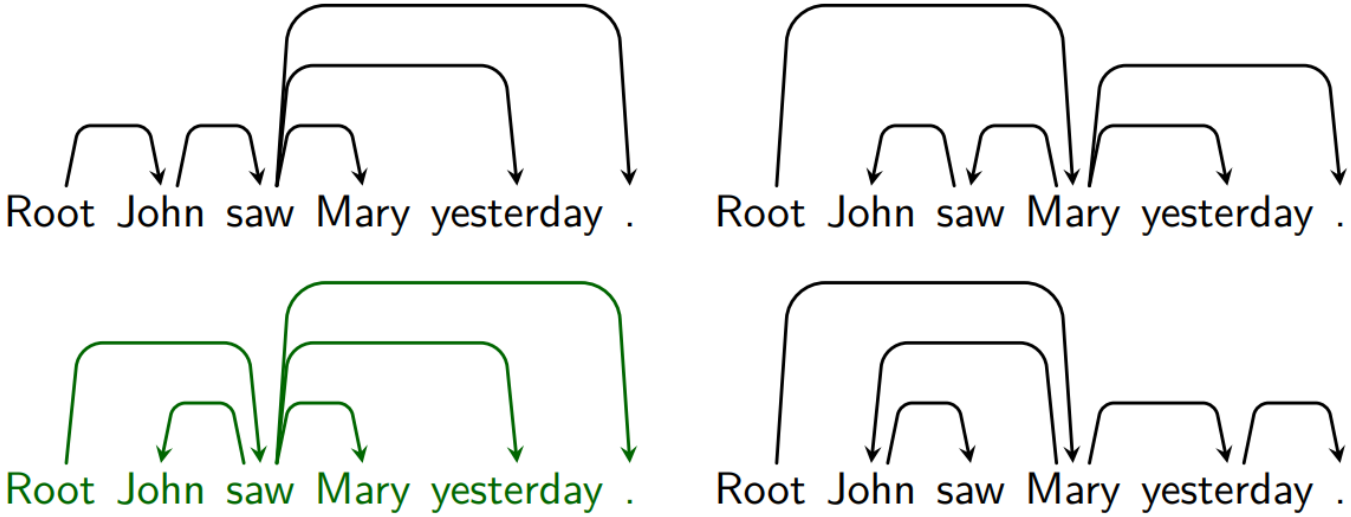
Graph-based Models

- Candidate trees for “John saw mary yesterday”



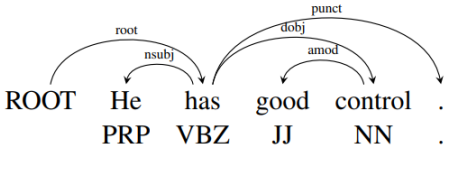
Graph-based Models

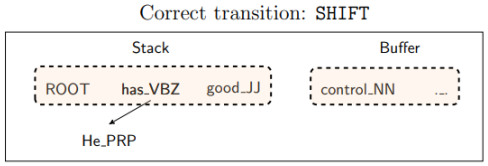
- Candidate trees for “John saw mary yesterday”



Neural-based Dependency Parsing

- Chen, Danqi, and Christopher D. Manning, EMNLP 2014
- VS. traditional transition-based approach
- Let the neural network do the feature combination





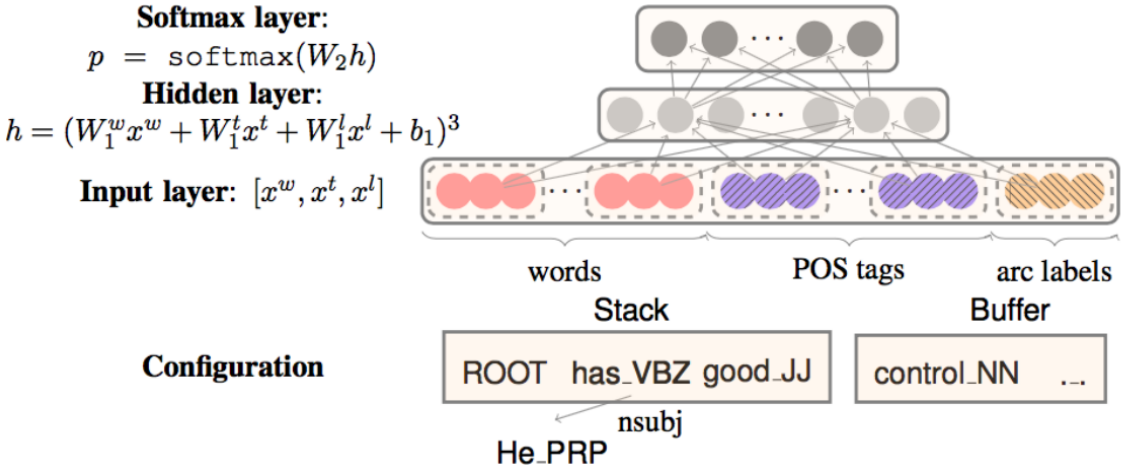
Transition	Stack	Buffer	A
	[ROOT]	[He has good control .]	$\emptyset$
SHIFT	[ROOT He]	[has good control .]	
SHIFT	[ROOT He has]	[good control .]	
LEFT-ARC(nsubj)	[ROOT has]	[good control .]	$A \cup$ nsubj(has,He)
SHIFT	[ROOT has good]	[control .]	
SHIFT	[ROOT has good control]	[.]	
LEFT-ARC(amod)	[ROOT has control]	[.]	$A \cup$amod(control,good)
RIGHT-ARC(dobj)	[ROOT has]	[.]	$A \cup$ dobj(has,control)
...	...	...	...
RIGHT-ARC(root)	[ROOT]	[]	$A \cup$ root(ROOT,has)

Single-word features (9)	
$s_1.w$; $s_1.t$; $s_1.wt$; $s_2.w$; $s_2.t$;	
$s_2.wt$; $b_1.w$; $b_1.t$; $b_1.wt$	
Word-pair features (8)	
$s_1.wt \circ s_2.wt$; $s_1.wt \circ s_2.w$; $s_1.wts_2.t$;	
$s_1.w \circ s_2.wt$; $s_1.t \circ s_2.wt$; $s_1.w \circ s_2.w$	
$s_1.t \circ s_2.t$; $s_1.t \circ b_1.t$	
Three-word feaures (8)	
$s_2.t \circ s_1.t \circ b_1.t$; $s_2.t \circ s_1.t \circ lc_1(s_1).t$;	
$s_2.t \circ s_1.t \circ rc_1(s_1).t$; $s_2.t \circ s_1.t \circ lc_1(s_2).t$;	
$s_2.t \circ s_1.t \circ rc_1(s_2).t$; $s_2.t \circ s_1.w \circ rc_1(s_2).t$;	
$s_2.t \circ s_1.w \circ lc_1(s_1).t$; $s_2.t \circ s_1.w \circ b_1.t$	

Chen, Danqi, and Christopher D. Manning. "A fast and accurate dependency parser using neural networks." Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014.

Neural-based Dependency Parsing

- The Input layer
  - Word embedding matrix E_w, POS tags embedding matrix E_t, Arc label embedding matrix E_l
  - Choose a set of elements based on the stack / buffer positions for each type of information (word, POS or label)
  - Add word feature $x_w = [e^w_{w_1}, e^w_{w_1}, \ldots, e^w_{w_n}]$ to the input layer, Similarly, we add the POS tag features x_t and arc label features x_l to the input layer.



Chen, Danqi, and Christopher D. Manning. "A fast and accurate dependency parser using neural networks." Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014.

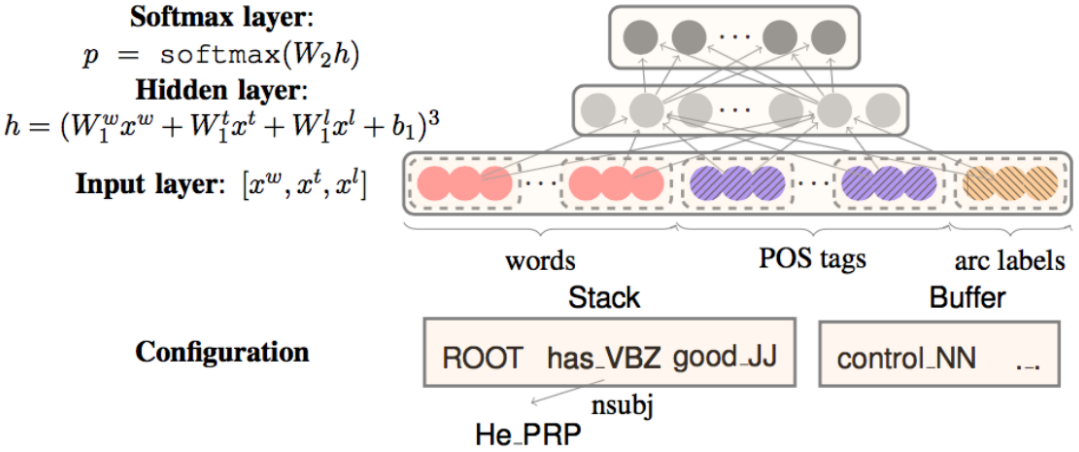
Neural-based Dependency Parsing

- The neural network architecture with one hidden layer
  - map the input layer to a hidden through a cube activation function:

$$h = (W_1^w x_w + W_1^t x_t + W_1^l x_l + b_1)^3$$

- A softmax layer is finally added on the top of the hidden layer for modeling multi-class probabilities

$$p = softmax(W_2 h)$$



Chen, Danqi, and Christopher D. Manning. "A fast and accurate dependency parser using neural networks." Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014.

Neural-based Dependency Parsing

- What Features to Extract?
  - The Top3 words on the stack and buffer (6 features)
  - The two leftmost/rightmost children of the top tow words on the stack (8 features)
  - Leftmost and rightmost grandchildren (4 features)
  - POS tags of all the above (18 features)
  - Arc labels of all children/grandchildren (12 features)

Neural-based Dependency Parsing

- Contributions
  - The first simple, successful neural dependency parser
  - The dense representations let it outperform other greedy parsers in both accuracy and speed
  - This work was further developed and improved by others, particular at Google
    - Bigger , deeper networks with better tuned hyperparameters
    - Beam search
    - Global, conditional random field (CRF)-style inference over the decision sequence

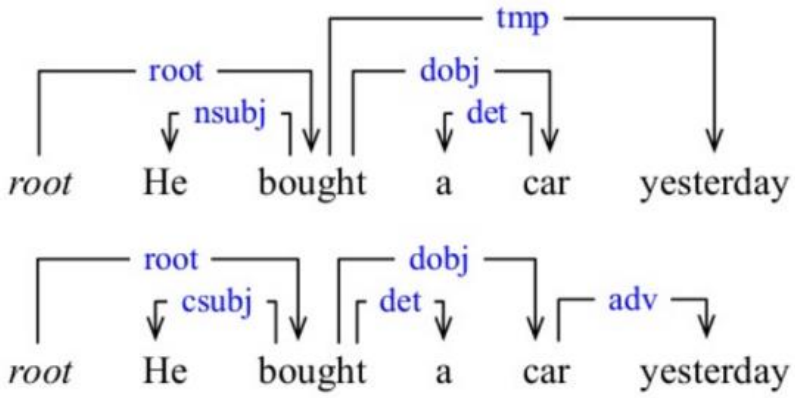
Parsing Evaluation

- Assume each node has exactly one head except for the ***root***

- For each tree, count
  - how many nodes found correct heads:
    - Unlabeled attachment score (UAS)
  - how many nodes found correct labels:
    - Label accuracy (LS)
  - how many nodes found both correct heads and labels:
    - Labeled (LAS)

Parsing Evaluation

- Unlabeled attachment score
  - Mismatches: bought -> a , car -> yesterday (3 / 5 = 60%)
- Label accuracy
  - Mismatches: He - csubj , yesterday - adv (3 / 5 = 60%)
- Labeled attachment score
  - He - csubj , bought -> a , car -> yesterday - adv(2 / 5 = 60%)



Treebanks

- English Penn Treebank: Standard corpus for testing syntactic parsing consists of 1.2 M words of text from the Wall Street Journal (WSJ).
- Typical to train on about 40,000 parsed sentences and test on an additional standard disjoint test set of 2,416 sentences.
- Chinese Penn Treebank: 100K words from the Xinhua news service.
- Other corpora existing in many languages, see the Wikipedia article “Treebank”

Parsing Resources

- Michael Collins ‘s Parser : English
  - http://people.csail.mit.edu/mcollins/code.html
- Dan Bikel ‘s Parser: English / Chinese / Arabic
  - http://www.cis.upenn.edu/~dbikel/software.html#stat-parser
- Stanford Parser: English / Chinese / German
  - http://www-nlp.stanford.edu/software/lex-parser.shtml
  - English / Chinese / German
- David Chiang’s Parser: English / Chinese
  - http://www.isi.edu/~chiang/
- HIT IR Dependency Parser
  - http://ir.hit.edu.cn

The Next Lecture

- Lecture 9

  Text Categorization